

Java Script

Hareketli bir kodlama için javascript...

Bu pdf bir öğrencinin javascript öğrenme sırasında aldığı notlardan ibarettir.

Yazar : Musa Yazlık

2020

JavaScript Eğitimim Notlarım

Version : 1.1

Son güncelleme: 21.08.2020

Yazar : Musa Yazlık

Sosyal Medya ve İletişim Bilgilerim

Web Sitem: <https://musayazlik.com>

Mail adresim: info@musayazlik.com

Telegram: <https://t.me/musayazlik>

Linkedin: <https://www.linkedin.com/in/musayazlik/>

Facebook: <https://www.facebook.com/musayazlik/>

Instagram: https://www.instagram.com/musa_yazlik/

Twitter: <https://twitter.com/musayazlik>

Github: <https://github.com/musayazlik>

Bu belgenin tüm telif hakları Musa Yazlık'a aittir.

Bu belge ticari olmadığı sürece , kesme veya koparma yapmadan çoğatılmasında , dağıtılmasında veya kopyalanmasında hiçbir sakınca yoktur.

İçindekiler

1. Javascript Nedir? Ne değildir?.....	1
1.1 Java Script Nedir?	1
1.2 Neden Javascript Öğrenmeliyim ?.....	1
1.3 Neden Modern Javascript diyoruz ?	1
1.4 Peki bunun anlamı nedir ?	2
1.5 Javascript 'i en popüler dil yapan sadece bu mu ?.....	2
1.6 Peki nedir Angular, React ya da Vuejs kütüphanesi ?	2
1.7 JavaScript Nasıl Kullanılır?	2
2. Çok Kullanılan Bazı Temel Javascript Fonksiyonları	3
2.1 console.log()	3
2.2 document.write()	4
2.3 document.writeln()	4
2.4 typeof	5
2.5 alert()	5
2.6 window.alert()	6
2.7 prompt()	6
2.8 break;	7
2.9 continue;	7
2.10 substr()	8
2.11 replace()	9
2.12 toLowerCase()	9
2.13 toUpperCase()	9
2.14 split().....	10
2.15 innerHTML	10
3. JavaScript String Metotları	11
3.1 length	11
3.2 trim()	12
3.3 slice()	12
3.4 substring().....	13
3.5 substr()	13
3.6 concat()	13
3.7 repeat()	14

3.8 indexOf()	14
3.9 lastIndexOf()	15
3.10 includes()	15
3.11 split()	16
3.12 charAt()	16
3.13 charCodeAt()	17
3.14 fromCharCode()	18
3.15 toLowerCase()	18
3.16 toUpperCase()	18
4. Array(Dizi) Metotları	19
4.1 concat()	19
4.2 toString()	20
4.3 join()	20
4.4 pop()	21
4.5 push()	21
4.6 shift()	22
4.7 unshift()	23
4.8 splice()	23
4.9 slice()	24
4.10 length	25
4.11 reverse()	26
4.12 sort()	26
4.13 indexOf()	27
4.14 some()	27
5. Javascript Bazı Hazır Fonksiyonlar	28
5.1 encodeURI()	28
5.2 decodeURI()	28
5.3 eval()	29
5.4 isFinite()	29
5.5 isNaN()	30
5.6 Number()	30
5.7 parseFloat()	31
5.8 parseInt()	33
5.9 String()	34
5.10 isArray()	34
6. JavaScript Math Nesnesi Metotları	35

6.1 isSafeInteger()	37
6.2 toPrecision()	37
6.3 toExponential()	38
6.4 sign()	38
7. JavaScript Date Nesnesi Metodları	39
7.1 JavaScript ile Günler Türkçe Yazdırma	40
7.2 JavaScript ile Ayları Türkçe Yazdırma	40
8. Javascriptteki Yorum Satırları	41
9. Javascriptteki Yazım Kuralları	42
10. Javascriptte Özel İşaretlerin Kullanımı	43
11. Değişkenler	44
11.1 Değişkenlere Giriş ve Yazım Kuralları	44
11.2 Değişken Türleri	45
11.3 Değişken Kapsamı	45
11.3.1 Yerel Değişken Nedir?	45
11.3.2 Genel Değişken Nedir?	45
12. Düzenli İfadeler	46
12.1 Düzenli ifadelerde kullanılan bazı metotlar	46
12.1.1 match()	46
12.1.2 global	47
12.1.3 ignoreCase	47
12.1.4 multiline	48
12.1.5 test()	49
12.1.6 input	49
12.1.7 source	49
12.1.8 lastIndex	50
12.1.9 leftContext	50
12.1.10 rightContext	51
12.1.11 exec()	51
12.1.12 compile()	52
12.2 Düzenli İfadelerde Konum Belirleyiciler	53
12.2.1 ^ konum belirleyicisi	53
12.2.2 \$ konum belirleyicisi	54
13. Operatörlere Giriş	54
13.1 Aritmetik Operatörleri	54
13.2 Artırma Ve Azaltma Operatörleri	55

13.3 Atama Operatörleri	55
13.4 Karşılaştırma Operatörleri	56
13.4.1 == Eşitse	56
13.4.2 === Hem değeri hem de türü eşitse	56
13.4.3 != Eşit değilse	56
13.4.4 !== Tam Eşit Değilse	57
13.4.5 < Küçükse	57
13.4.6 <= Küçük veya eşitse	57
13.4.7 > Büyükse	57
13.4.8 >= Büyük veya eşitse	58
13.5 Mantıksal Operatörler	58
13.5.1 Ve (&&) Operatörü	58
13.5.2 Veya () Operatörü	58
13.5.3 Değil (!) Operatörü	59
14. Koşul Operatörleri	59
14.1 İf-Else Operatörü	59
14.1.1 Kullanımı 1:	59
14.1.2 Kullanımı 2:	60
14.1.3 Kullanımı 3:	61
14.2 Switch Operatörü	61
15. Döngülere Giriş	63
15.1 For Döngüsü	63
15.2 While Döngüsü	65
15.3 Do-While Döngüsü	67
15.4 For in Döngüsü	67
15.5 İç İç Döngüler	68
15.5.1 Çarpım tablosu uygulaması	68
16. Hata Yönetimi	69
16.1.1 Throw	70
16.1.2 Finally	71
17. DİZİLER	72
17.1 Javascript ile Dizi Nasıl Oluşturulur?	73
17.2 New Anahtar Kelimesi ile Dizi Oluşturma	73
17.3 Dizi Nesnelerine Nasıl Ulaşılır?	74
17.4 Dizi içindeki değer nasıl değiştirilir?	74
17.5 Çok Boyutlu Diziler	75

18. Fonksiyonlar (Functions)	76
18.1 Fonksiyonlar nedir?	76
18.2 Fonksiyonların genel özellikleri	76
18.3 Parametrelili Fonksiyon	76
18.4 Parametresiz Fonksiyon	77
19. JAVASCRIPTTE NESNELER	77
20. Dom(Document Object Model) Olayları	80
20.1 getElementById()	80
20.2 getElementsByName()	81
20.3 getElementByTagName()	82
20.4 getElementByClassName()	83
20.5 document.body	83
20.6 document.head	84
20.7 characterSet	84
20.8 title	85
20.9 documentURI	85
20.10 location	86
20.11 domain	86
20.12 lastModified	87
20.13 hasFocus()	87
20.14 activeElement()	88
20.15 scripts	88
20.16 dir	89
20.17 classList	89
20.18 tagName	90
20.19 childElementCount	91
20.20 hasChildNodes	91
20.21 children	92
20.22 firstElementChild	93
20.23 firstChild	94
20.24 lastElementChild	94
20.25 lastChild	94
20.26 nextElementSibling	94
20.27 nextSibling	95
20.28 previousElementSibling	96
20.29 previousSibling	96

20.30 offsetParent	96
20.31 contains()	97
20.32 textContent	98
20.33 compareDocumentPosition()	98
20.34 parentElement	99
20.35 parentNode	100
20.36 appendChild	101
20.37 removeChild	101
20.38 replaceChild()	101
20.39 cloneNode()	102
20.40 insertBefore()	103
20.41 Attribute Özellikleri	104
20.41.1 setAttribute()	104
20.41.2 createAttribute()	105
20.41.3 setAttributeNode()	105
20.41.4 getAttribute	106
20.41.5 getAttributeNode	106
20.41.6 removeAttribute()	107
20.41.7 removeAttributeNode()	107
20.41.8 hasAttribute()	108
20.41.9 hasAttributes()	109
20.42 createElement()	109
20.43 createTextNode()	110
20.44 removeChild()	110
20.45 appendChild()	111
20.46 replaceChild()	112
20.47 querySelector()	113
20.48 querySelectorAll()	114
20.49 addEventListener()	114
20.50 removeEventListener	115
20.51 Attribute Nesne Özellikleri	116
20.51.1 length	116
20.51.2 name	117
20.51.3 value	117
20.51.4 specified	118
20.51.5 getNamedItem()	119

20.51.6 setNamedItem()	119
20.51.7 removeNamedItem()	120
20.51.8 item()	120
20.52 clientLeft	121
20.53 clientTop	121
20.54 clientWidth	122
20.55 clientHeight	122
20.56 offsetLeft	123
20.57 offsetTop	123
20.58 offsetWidth	124
20.59 offsetHeight	124
20.60 scrollLeft	125
20.61 scrollTop	125
20.62 scrollWidth	126
20.63 scrollHeight	126
21. JavaScript DOM Events	127
21.1 Javascript Fare Olayları (Mouse Events) Türleri	127
21.1.1 onclick()	127
21.1.2 ondblclick()	128
21.1.3 oncontextmenu()	128
21.1.4 onmouseover()	129
21.1.5 onmouseout()	130
21.1.6 onmouseup()	130
21.1.7 onmousedown()	130
21.1.8 onmousemove()	131
21.1.9 onmouseenter	132
21.1.10 onmouseleave	133
21.1.11 onwheel	134
21.2 MouseEvent Özellikleri Ve Metotları	134
21.2.1 altkey	134
21.2.2 shiftkey	135
21.2.3 ctrlkey	135
21.2.4 button	135
21.2.5 which	136
21.2.6 buttons	137
21.2.7 clientX && clientY	137

21.2.8 pageX && page Y	138
21.2.9 detail	139
21.3 JS- DOM Events - Klavye Olayları	140
21.3.1 onkeydown	140
21.3.2 onkeypress	140
21.3.3 onkeyup	140
21.3.4 keycode.....	141
21.3.5 key	141
22. JavaScript Bom Olayları	142
22.1 Bom Nedir?	142
22.2 Window Arabirimi.....	142
22.2.1 open().....	142
22.2.2 opener()	144
22.2.3 close()	145
22.2.4 closed	146
22.2.5 stop().....	147
22.2.6 print()	148
22.2.7 confirm()	149
22.2.8 screenX ve screenY	150
22.2.9 innerWidth ve innerHeight	151
22.2.10 pageXOffset & pageYOffset	152
22.2.11 scrollX & scrollY	153
22.2.12 resizeBy()	153
22.2.13 moveBy()	154
22.2.14 scrollBy	155
22.3 Screen Arabirimi.....	155
22.3.1 width & height	155
22.3.2 availwidth & availheight	156
22.3.3 colorDepth	157
22.3.4 pixelDepth.....	157
22.4 Navigator Arabirimi.....	158
22.4.1 appCodeName	158
22.4.2 language	158
22.4.3 userAgent	159
22.4.4 onLine	159
22.4.5 cookieEnabled.....	159

22.4.6 javaEnabled()	160
22.5 Location Arabirimi	160
22.5.1 pathname	160
22.5.2 search	161
22.5.3 protocol	162
22.5.4 assign()	162
22.5.5 replace()	163
22.5.6 reload()	164
22.6 Zamanlayıcılar	164
22.6.1 setTimeout()	164
22.6.2 clearTimeout	165
22.6.3 setInterval	165
22.6.4 clearInterval	166
23. Kapanış Yazısı	168
24. Kaynakça	169

1. Javascript Nedir? Ne değildir?

1.1 Java Script Nedir?

Javascript web sayfalarını interaktif hâle getirebilmek için geliştirilmiş bir kodlama dilidir. Kısaltması JS'dir. Sunucu tabanlı değildir; kullanıcı tabanlıdır. Bu yüzden hosting hizmeti veren sunucuları yormazlar.

1995 senesinde Java adlı yeni bir programlama dili piyasaya sürülmekteydi. NetScape firması ise bu betik dilin daha popüler olması adına ismini JavaScript koyarak piyasaya sürdü. Java ile JavaScript arasında isim benzerliği dışında bir ilgi yoktur.

JavaScript daha çok internet sayfalarında anket, quiz, uyarı penceresi gibi interaktif uygulamalarda kullanılır. Örneğin; kullanıcı herhangi bir formda onayla tuşuna bastı; fakat şifresini yazmayı unuttu. Kullanıcı "Şifrenizi belirlemeyi unuttunuz!" bildirimi olan bir açılır pencere uyarısı alabilir. İşte bunu sağlayan JavaScript'tir.

1.2 Neden Javascript Öğrenmeliyim ?

Javascript son zamanlarda **en popüler diller** arasında yerini aldı hatta **javascript'in en popüler programlama dili** olduğunu söyleyebiliriz. Peki JavaScript 'i son zamanlarda bu kadar **popüler yapan neden nedir** ?

Çoğu web geliştirici için Javascript halen sadece tarayıcıda yani client tarafında çalışan ve web içeriklerini hareketli hale getiren bir script dili olarak biliniyor. Evet Javascript 'i bu yönüyle halen kullanmaktayız.

Web sitemizde bir slider 'a mı ihtiyacımız var ya da bir butona tıkladığımızda bir popup kutusu mu açılsın ya da tarayıcıda çalışan bir program mı ihtiyacımız var ? Bu durumda yıllardır kullandığımız yapı tabi ki **Javascript**... Hatta **Javascript 'i asekron veri iletiminde** bile kullanıyoruz. Hatta **Jquery** 'nin aslında **Javascript** olduğunda biliyoruz.

Javascript' e sadece tarayıcıda çalışan bir dil olarak baksakta javascript gerçekten kuvvetli ve bir o kadar da zevkli bir dildir. Peki bu işlemleri zaten Javascript ile yıllardır yapıyoruz **son zamanlarda Javascript' in bu kadar popüler olmasındaki neden nedir** ?

1.3 Neden Modern Javascript diyoruz ?

Bu nedenlerin en başında tabi ki **NodeJs** yapısı geliyor. **NodeJs** en basit şekilde Javascript kodlarını makine koduna çeviren bir programdır.

Hangi programlama dilini kullanırsak kullanalım yazdığımız kodların bilgisayar tarafından anlaşılır olması için yazdığımız kodların **makine koduna** çevrilmesi yani kodlarımızın derlenmesi gerekiyor çünkü bilgisayarın anladığı ve konuştuğu tek dil budur.

NodeJs 'den önce yazdığımız Javascript kodlarını sadece tarayıcı içinde bulunan Javascript motoru ile anlaşılır hale getirebiliyorduk ancak **NodeJs Javascript in sadece tarayıcı**

içinde çalışmasının önüne geçti yani **Google 'in chrome Javascript V8 engine** 'ini tarayıcı içinden çıkarıp bir paket halinde kullanıcılara sundu.

Yani artık kullanıcılar NodeJs paketini bilgisayarlarına kurup javascript kodlarını çalıştırabiliyorlar...

1.4 Peki bunun anlamı nedir ?

Javascript in kullanım alanı tarayıcı dışında da mümkün hale geldi ve artık **Javascript 'i bir server tabanlı programlama dili** gibi kullanabiliyoruz örneğin **asp.net ,php** ya da **java** gibi...

Bu gerçekten kulağa hoş geliyor. Bildiğimiz kodlama yapısı artık bize yeni bir **programlama dili** sundu. **Javascript ile server tarafında programlama** yapabiliyor olmaz gerçekten güzel bir olay.

1.5 Javascript 'i en popüler dil yapan sadece bu mu ?

Tabiki de hayır....

**** Npm** yani **node package manager** ile bir çok hazır javascript kütüphanesini projemizi saniyeler içinde indirebiliyoruz. Üstelik npm i kullanmak için nodejs projesi geliştirmek zorunda değiliz. Her türlü projemize **hazır js ve css kütüphanelerini kolaylıkla indirebiliyoruz.**

****** ve tabi ki en önemlisi **Angular, React ve Vuejs** 'in **Javascript dilini kullanarak** geliştirilmesi diyebiliriz. Yani Javascript i biliyorsak **Angular, React** ve **Vuejs** kolaylıkla öğrenebiliriz.

1.6 Peki nedir Angular, React ya da Vuejs kütüphanesi ?

Angular, React ya da Vuejs kütüphanesi ile mobil uygulama gibi çalışan web siteleri geliştirebiliyoruz. Hatta yazdığımız kodları native uygulamalara yani **adroid** ya da **ios uygulamalarına dönüştürebiliyoruz...** Bu belkide Javascript i Javascript yapan en önemli özellik...

Yani android için Java, IOS için swift bilmemize hiç gerek yok javascript kullanarak bunların hepsini zaten yapabiliriz.

Güçlü bir Javascript bilgisine sahip olmak son zamanlarda yazılım adına yapmamız gereken en önemli adımdır.

******* Eeee düşmanımızı tanıdığımıza göre bu javascripti nasıl projelerimizde kullanacağız onu öğrenelim. 😊

1.7 JavaScript Nasıl Kullanılır?

JavaScript HTML kodları içerisinde `<script></script>` tagları içerisinde kullanılacağı gibi harici bir .js uzantılı dosya olarak da sayfaya sonradan çağrılabilir.

JavaScript'in HTML İçerisinde Kullanımı

```
11 <body>
12
13 <h1>Merhaba Dünya</h1>
14
15 <script>
16 /* Java script kodları */
17 </script>
18 </body>
```

JavaScript'in Harici Dosyadan Çağırılması

```
11 <body>
12
13 <h1>Merhaba Dünya</h1>
14
15 <script type="text/javascript" src="dosya.js"></script>
16 </body>
```

Genel olarak javascript kodları harici bir javascript dosyasında kodlanır. Hiç yoksa ben size onu öneririm. 😊

2. Çok Kullanılan Bazı Temel Javascript Fonksiyonları

 Buradaki notları çok fazla kullanacağınızın garantisini verebilirim. 

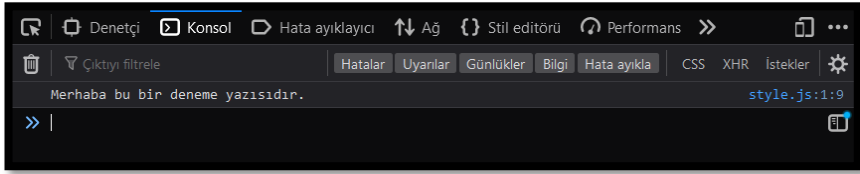
2.1 console.log()

Girilen kodların çıktısının konsolda görünmesini sağlar.

Örnek:

```
style.js  X
1 console.log("Merhaba bu bir deneme yazısıdır.");
```

Çıktısı:



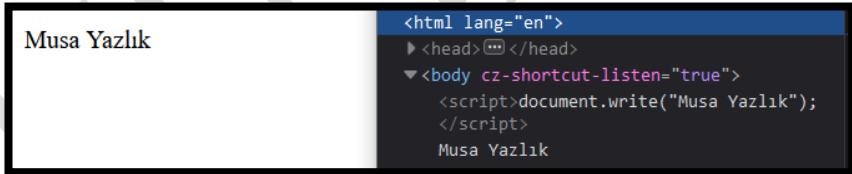
2.2 document.write()

Dökümana çıktı yapar. Javascriptte çok kullanılan bir kodlamadır.

Örnek:



Çıktısı:



2.3 document.writeln()

document.write() ile aynı işlevi yapar tek farkı işlem sonucunda yeni satıra geçiş yapar.

Örnek:



Çıktısı:

```
Musa Yazlık  
Merhaba Dünya
```

2.4 typeof

Değişkenlerin türlerini öğrenmek amaçlı kullanılır.

Örnek:

```
14 var nesne1 = "merhaba";  
15 var nesne2 = 20;  
16 var nesne3 = true;  
17 var nesne4 = null;  
18  
19 console.log(typeof nesne1);  
20 console.log(typeof nesne2);  
21 console.log(typeof nesne3);  
22 console.log(typeof nesne4);
```

Çıktısı:



2.5 alert()

Ekran uyarı metni vermek amaçlı kullanılan bir fonksiyondur.

Örnek:

```
<script>  
  alert("Uyarı : Bir şeyi yanlış yapmış olmalısın...");  
</script>
```

Çıktısı:

Uyarı : Bir şeyi yanlış yapmış olmalısın...

☐ Bu sayfanın ek iletişim kutuları oluşturmamasının önle

Tamam

2.6 window.alert()

alert ile aynı işlevi görür. Tek varkı nesnesel bir alart işlemi gerçekleştirir.

Örnek:

```
<script>  
window.alert("Musa Yazlık");  
</script>
```

Çıktısı:

Musa Yazlık

Tamam

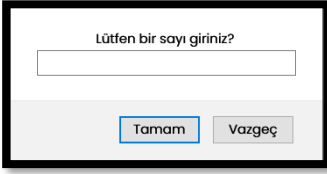
2.7 prompt()

Kullanıcıdan bir veri almak amaçlı kullanılan bir fonksiyondur.

Örnek:

```
<script>  
prompt("Lütfen bir sayı giriniz? ");  
</script>
```

Çıktısı:



Lütfen bir sayı giriniz?

Tamam Vazgeç

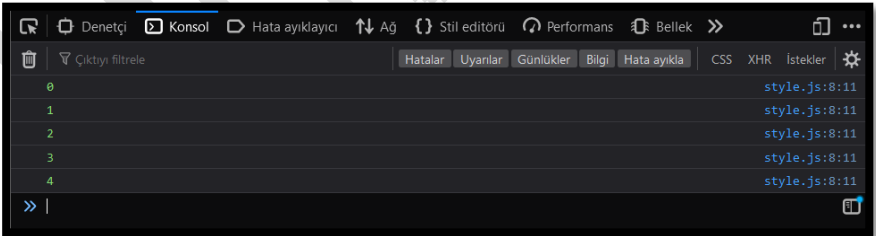
2.8 break;

Break ifadesi döngüden çıkmak için kullanılır.

Örnek:

```
<script>
for (var i = 0; i < 10; i++) {
  if (i == 5) {
    break;
  }
  console.log(i);
  //0,1,2,3,4
}
</script>
```

Çıktısı:



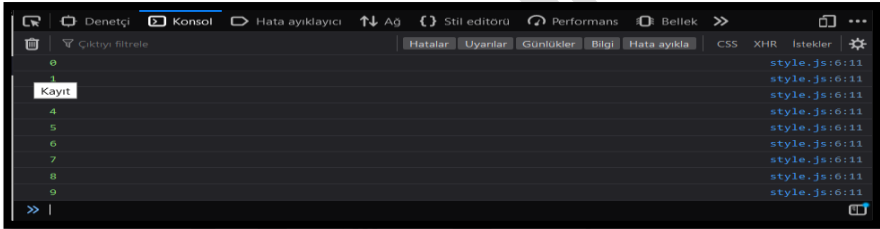
2.9 continue;

Continue deyimi, belirli bir koşul olduğunda döngüdeki geçerli yinelemeyi durdurur ve bir sonraki yineleme ile döngüyü devam ettirir.

Örnek:

```
<script>
for (i = 0; i < 10; i++) {
  if (i === 3) {
    continue;
  }
  console.log(i);
  // 0,1,2,4,5,6,7,8,9
}
</script>
```

Çıktısı:



2.10 substr()

substr(değer1,değer2): İstenilen aralıkta kesme işlemi yapar.

değer1 → Başlangıç değeridir.

değer2 → Kaç karakter alınacağına karar verilir.

Örnek:

```
<script>
var metin = "Merhaba bu bir deneme yazıdır.";
document.write(metin.substr(6,10));
</script>
```

Çıktısı:

a bu bir d

2.11 replace()

Belirlenen değeri belirtilen değer ile değiştirmek amaçlı kullanılır. Ama içeriğe tam olarak müdaale etmez. yeni bir değer olarak döndürür.

Örnek ve Çıktısı:

```
<script>
const str = 'JavaScript heyecan verici bir dildir.';

// replace() metodu büyük küçük duyarlıdır.
console.log(str.replace('JavaScript', 'PHP')); // 'PHP heyecan verici bir dildir.'

// büyük küçük duyarlılığı olmadan yapmak için regular exp kullanmalısınız.
console.log(str.replace(/JAVASCRIPT/i, 'PHP')); // 'PHP heyecan verici bir dildir.'

// replace() metodu yalnızca ilk eşleşen karakteri değiştirir.
console.log(str.replace('e', 'E')); // 'JavaScript heyecan verici bir dildir.'

// tüm karakterleri değiştirmek için regular exp kullanmalısınız.
console.log(str.replace(/e/g, 'E')); // 'JavaScript heyecan verici bir dildir.'
</script>
```

2.12 toLowerCase()

Büyük harfle yazılmış olan bütün karakterleri küçük harfe dönüştürür.

Örnek:

```
<script>
var metin = "MERHABA deneme YaZIsI."
document.write(metin.toLowerCase());
</script>
```

Çıktısı:

merhaba deneme yazısı.

2.13 toUpperCase()

Küçük harfle yazılmış olan bütün karakterleri büyük harfe dönüştürür.

Örnek:

```
<script>
var metin = "MERHABA deneme YAZISI."
document.write(metin.toUpperCase());
</script>
```

Çıktısı:

MERHABA DENEME YAZISI.

2.14 split()

Verilen string değerini istenilen bölme noktanına göre parçalara ayırarak diziye dönüştürür. İlk parametre neye göre parçalayacağını ikinci değer ise dizinin eleman sayısını sınırlar. İkinci değer boş bırakılabilir. O zaman verilen bütün stringi diziye dönüştürür.

Örnek:

```
<script>
var metin = "Merhaba bu bir deneme yazısıdır. Merhaba dünya"
document.write(metin.split(" ", 5));
</script>
```

Çıktısı:

Merhaba,bu,bir,deneme,yazısıdır.

2.15 innerHTML

Document Nesnesinin içeriğini değiştirmek, yeni bir içerik girmek veya nesnenin içeriğine ulaşmak amacıyla kullanılır.

Örnek:

```
<body>

<div id="demo"></div>
<script>
var girilecek_icerik = "Girilecek içerik yazısı burada.";
document.getElementById('demo').innerHTML = girilecek_icerik;
</script>

</body>
```

document.getElementById('demo').innerHTML → Yazarakta nesnenin içeriğine ulaşabilirsiniz.

Çıktısı:

Girilecek içerik yazısı burada.

```
<body>
<div id="demo">Girilecek içerik yazısı burada.</div>
<script>
var girilecek_icerik = "Girilecek içerik yazısı
burada.";
document.getElementById('demo').innerHTML =
girilecek_icerik;
</script>
```

3. JavaScript String Metotları

3.1 length

String bir ifadenin uzunluğunu yani karakter sayısını döndürür. Eğer belirttiğiniz string ifade değeri boş ise 0 döndürecektir.

Örnek:

```
<script>
var metin = "Merhaba Dünya";
document.write("Metnini Karakter sayısı: " + metin.length);
</script>
```

Çıktısı:

Metinini Karakter sayısı: 13

3.2 trim()

Değişken içeriğinde varsa başındaki ve sonundaki boşlukları siler ve değeri geri döndürür.

Örnek:

```
<script>
var mesaj = "Benim adım ";
var kısa_kelime = "Musa Yazlık";
var sonuc = mesaj + kısa_kelime;
var trim_sonuc = mesaj + kısa_kelime.trim();
console.log(sonuc);
console.log(trim_sonuc);
</script>
```

Çıktısı:

```
Benim adım      Musa Yazlık
Benim adım Musa Yazlık
```

>>

3.3 slice()

Kendisine verilen string bir metnin istenilen aralığını kesip değer olarak geri döndürür. İki parametre alır. 1. parametre başlangıç indisi, 2. parametre bitiş indisi. Parametre belirtilmez ise tüm içeriği kopyalar. 1. parametre ikinci parametreden büyük olamaz. Olamaması durumunda bir çıktı ve hata verme vermez.

Örnek:

```
<script>
var metin = "Lorem ipsum dolor sit amet consectetur, adipiscing elit.";
document.write("Kesip alınan parça: " + metin.slice(12,26));
</script>
```

Çıktısı:

Kesip alınan parça: dolor sit amet

3.4 substring()

slice ile aynı işlemi yapar tek farkı 1. parametre 2. parametreden büyük olduğunda 1. parametre ile 2. parametrenin yerini değiştirerek işlem yapar. O şekilde bir çıktı verir. Diyelim (26,12) olarak işlem verilsin. (12,26) olarak çıktı verir. Bu metot için örneğe gerek duymadım.

3.5 substr()

substr(değer1,değer2) istenilen aralıkta kesme işlemi yapar.

değer1 → Başlangıç değeridir.

değer2 → Kaç karakter alınacağına karar verilir.

Örnek:

```
<script>
var metin = "Merhaba bu bir deneme yazıdır.";
document.write(metin.substr(6,10));
</script>
```

Çıktısı:

a bu bir d

3.6 concat()

İki veya daha fazla stringi birleştirmeye yarar. Aynı şekilde dizilerde de kullanılır. Sonsuz parametre alabilir.

Örnek:

```
<script>
var str1 = "Merhaba";
var str2 = "Dünya";
var str3 = "Musa Yazlık";
document.write(str1.concat(str2, str3));
</script>
```

Çıktısı:

Merhaba Dünya Musa Yazlık

3.7 repeat()

İstenilen string değerinin istenildiği kadar yazılmasını sağlar.

Örnek:

```
<script>
var str1 = "Merhaba";
document.write(str1.repeat(5));
</script>
```

Çıktısı:

"Merhaba" yazısının 5 kez yazılmasını istedik.

MerhabaMerhabaMerhabaMerhabaMerhaba

3.8 indexOf()

String olarak verilen metnin içerisinde parametre olarak verilen değişkeni arar ve bulunduğu ilk değişkenin baş indis numarasını geri döndürür. Bulunamaz ise -1 değerini döndürür. Küçük büyük harf duyarlıdır. İki parametre alır. 1. parametre alanacak değişken, ikinci parametre aramanın başlanacağı indis numarası (2. parametrenin girilmesi zorunlu değildir. Girilmez ise 0 inci indisten aramaya başlar.)

Örnek:

```
<script>
var str1 = "Lorem ipsum dolor sit, amet musa consectetur adipiscing elit.";
document.write(str1.indexOf("musa") + " indiste bulundu.");
</script>
```

Çıktısı:

28 indiste bulundu.

3.9 lastIndexOf()

indexOf ile aynı işlemi yapar tek farkı son eşleşmeyi sağlayan değerın indis numarasını geri döndürür.

Örnek:

```
<script>
var str1 = "Lorem ipsum dolor sit, amet musa consectetur adipiscing musa elit.";
document.write(str1.lastIndexOf("musa") + " indiste bulundu.");
</script>
```

Çıktısı:

57 indiste bulundu.

3.10 includes()

String metnin içerisinde aranan kelimenin olup olmadığını kontrol eder. Sonucu true veya false olarak geri döndürür. Küçük büyük harf duyarlıdır. İki parametre alır. 1. parametre alıncak kelime, ikinci parametre aramanın başlanacağı indis numarası (2. parametrenin girilmesi zorunlu değildir. Girilmez ise 0. indisten aramaya başlar).

Örnek:

```
<script>
var str1 = "Lorem ipsum dolor sit, amet musa consectetur ahmet adipiscing musa elit.";
document.write(str1.includes("Ahmet") + "<br>");
document.write(str1.includes("ahmet"));
</script>
```

Çıktısı:

```
false
true
```

3.11 split()

Verilen string değerini istenilen bölme noktanına göre parçalara ayırarak diziye dönüştürür. İlk parametre neye göre parçalayacağını ikinci değer ise dizinin eleman sayısını sınırlar. İkinci değer boş bırakılabilir. O zaman verilen bütün stringi diziye dönüştürür.

Örnek:

```
<script>
var metin = "Merhaba bu bir deneme yazısıdır. Merhaba dünya"
document.write(metin.split(" ", 5));
</script>
```

Çıktısı:

```
Merhaba,bu,bir,deneme,yazısıdır.
```

3.12 charAt()

Kendisine parametre olarak verilen değer doğrultusunda değişken içeriği olan karakteri bularak geri döndürür. Dizilerdeki gibi bir işlev görür. Parametre olarak verilen değer metinden büyük olursa boş bir değer döndürür. Null veya undefined dönmez.

Örnek:

```
<script>
var metin = "Merhaba Dünya";
document.write("Metnin 3 üncü karakteri: " + metin.charAt(3) + "<br>");
document.write("Metnin 6 ıncı karakteri: " + metin.charAt(6) + "<br>");
document.write("Metnin 9 üncü karakteri: " + metin.charAt(9) + "<br>");
document.write("Metnin 12 inci karakteri: " + metin.charAt(12) + "<br>");
</script>
```

Çıktısı:

Metnin 3 üncü karakteri: h
Metnin 6 ıncı karakteri: a
Metnin 9 üncü karakteri: ü
Metnin 12 inci karakteri: a

3.13 charCodeAt()

Metinde alınan karakterin unicode değerini geri döndürür.

Örnek:

```
<script>
var metin = "Merhaba Dünya";
document.write("Metnin 5 üncü karakteri: " + metin.charAt(5) + "<br>");
document.write("5 inci Karakterin Unicode Değeri: " + metin.charCodeAt(5) + "<br>");
</script>
```

Çıktısı:

Metnin 5 üncü karakteri: b
5 inci Karakterin Unicode Değeri: 98

3.14 fromCharCode()

Kendisine parametre olarak verilen unicode değerin karakter sonucunu geri döndürür. Çoktu parametre alabilir.

Örnek:

```
<script>  
document.write(75 unicode değerine sahip karakter: "+String.fromCharCode(75) + "<br>");  
</script>
```

Çıktısı:

75 unicode değerine sahip karakter: K

3.15 toLowerCase()

Büyük harfle yazılmış olan bütün karakterleri küçük harfe dönüştürür.

Örnek:

```
<script>  
var metin = "MERHABA deneme Yazısı."  
document.write(metin.toLowerCase());  
</script>
```

Çıktısı:

merhaba deneme yazısı.

3.16 toUpperCase()

Küçük harfle yazılmış olan bütün karakterleri büyük harfe dönüştürür.

Örnek:

```
<script>
  var metin = "MERHABA deneme Yazisi."
  document.write(metin.toUpperCase());
</script>
```

Çıktısı:

MERHABA DENEME YAZISI.

4. Array(Dizi) Metotları

4.1 concat()

Concat metodu eklendiği dizi ile parametre olarak aldığı dizi(leri) birleştirerek yeni bir dizi döndürür. Kısacası **dizileri birleştirmek** için kullanılır. Birden fazla diziyi tek bir dizi haline getirir yani birleştirir. Birleştirmek için dizi sınırı yoktur ancak en az 1 dizi diğer bir dizi ile birleştirilebilir.

Örnek:

```
<script>
  var diller = ["türkçe", "ingilizce"];
  var meyveler = ["muz", "çilek", "armut", "ayva"];
  var hayvanlar = ["at", "ayr", "arı", "aslan"];

  var yeniDizi = diller.concat(meyveler, hayvanlar);

  document.write(diller + "<br>");
  document.write(meyveler + "<br>");
  document.write(hayvanlar + "<br>");
  document.write(yeniDizi + "<br>");
</script>
```

Çıktısı:

```
türkçe,ingilizce  
muz,çilek,armut,ayva  
at,ay,arı,aslan  
türkçe,ingilizce,muz,çilek,armut,ayva,at,ay,arı,aslan
```

4.2 toString()

toString metodu, kendisine tanımlanan değişken değerini string olarak değiştirerek döndürür.

Örnek:

```
<script>  
var sayi = 22;  
var cikti = sayi.toString();  
document.write("sayi : " + sayi + "<br>");  
document.write("sayi veri türü : " + typeof (sayi) + "<br>");  
document.write("cikti : " + cikti + "<br>");  
document.write("cikti veri türü : " + typeof (cikti));  
</script>
```

Çıktısı:

```
sayi : 22  
sayi veri türü : number  
cikti : 22  
cikti veri türü : string
```

Not: Bir diziye toString uyguladığımızda içindeki değeri virgül ile ayrılmış olarak bir dizi değeri döndürür.

4.3 join()

join metodu bir array içerisinde yer alan bütün elemanları birleştirerek string bir ifade olarak geri döndürür. Elemanlar varsayılan olarak virgül ile ayırıcı ile ayrılır. İsteğe bağlı olarak elementleri birbirinden ayırmak için farklı ayıraçlar da kullanılabilir.

Örnek:

```
<script>
var deneme = ["musa", 20, true, "selam"];

document.write(deneme.join() + "<br>")
document.write(deneme.join("+") + "<br>")
document.write(deneme.join("Ahmet") + "<br>")
</script>
```

Çıktısı:

```
musa,20,true,selam
musa+20+true+selam
musaAhmet20AhmettrueAhmettselam
```

4.4 pop()

Pop metodu dizideki son değeri siler ve o değeri bir değer olarak geri döndürür.

Örnek:

```
<script>
var deneme = ["musa", 20, true, "selam"];
document.write("Dizimizin ilk hali: " + deneme + "<br>");
var silinen_deger = deneme.pop();
document.write("Dizideki silinen son değer: " + silinen_deger + "<br>");
document.write("Dizinin son hali: " + deneme);
</script>
```

Çıktısı:

```
Dizimizin ilk hali: musa,20,true,selam
Dizideki silinen son değer: selam
Dizinin son hali: musa,20,true
```

4.5 push()

Dizinin sonuna eleman eklemek amaçlı kullanılır.

Örnek:

```
<script>
var deneme = ["musa", 20, true, "selam"];
document.write("Dizimizin ilk hali: " + deneme + "<br>");
deneme.push("merhaba", 20, 45, 42);
document.write("Dizinin son hali: " + deneme);
</script>
```

Çıktısı:

Dizimizin ilk hali: musa,20,true,selam
Dizinin son hali: musa,20,true,selam,merhaba,20,45,42

4.6 shift()

Dizinin ilk öğesini kaldırır ve diğer tüm öğeleri daha düşük bir indise düşürür. Kaldırılan öğeyi geri döndürür.

Örnek:

```
<script>
var deneme = ["musa", 20, true, "selam"];
document.write("Dizimizin ilk hali: " + deneme + "<br>");
var silinen_ilk_oge = deneme.shift();
document.write("Silinen ilk öge: " + silinen_ilk_oge + "<br>");
document.write("Dizinin son hali: " + deneme);
</script>
```

Çıktısı:

Dizimizin ilk hali: musa,20,true,selam
Silinen ilk öge: musa
Dizinin son hali: 20,true,selam

4.7 unshift()

Dizinin başına istenilen öğe veya öğeleri ekler ve diğer tüm öğeleri daha yüksek bir indise yükseltir. Dizinin eleman sayısını geri döndürür.

Örnek:

```
<script>
var deneme = ["musa", 20, true, "selam"];
document.write("Dizimizin ilk hali: " + deneme + "<br>");
var eleman_sayisi = deneme.unshift("merhaba", 20, 45, 24);
document.write("Dizinin Eleman sayısı: " + eleman_sayisi + "<br>");
document.write("Dizimizin son hali: " + deneme + "<br>");
</script>
```

Çıktısı:

```
Dizimizin ilk hali: musa,20,true,selam
Dizinin Eleman sayısı: 8
Dizimizin son hali: merhaba,20,45,24,musa,20,true,selam
```

4.8 splice()

splice() metodu diziyi eleman ya da elemanlar eklenebilmesini sağladığı gibi, silinebilmesini de sağlar. splice() ile diziden silinen değeri geri döndürür.

Örnek:

```
<script>
var dizi = ["ali", "veli", "kırklareli", "emret", "komutanım"];

document.write("orjinal dizi: " + dizi + "<br>");
var sil = dizi.splice(2, 1); // 2. indisten başla 1 tane sil, silinecek adet sayısı belirtilmezse 2. indisten
başlar sona kadar siler.

document.write("silinen değer: " + sil + "<br>");
document.write("silindikten sonraki dizi: " + dizi + "<br>");

var ekle = dizi.splice(1, 0, "ahmet", "mehmet"); // 1. indisten başla (değeri 1. indisin önüne koyar), değer
silme (0 kayarak değer silme diyorum), sonradan yazılanları diziyi 1. indisten itibaren ekle

document.write("eklenen değer: " + ekle + "<br>"); // eklenen değeri döndürmez dedik!
document.write("ekledikten sonraki dizi: " + dizi + "<br>");

// aynı anda hem silip hem ekleme yapalım

var oncesilsongraekle = dizi.splice(1, 2, "murat"); // 1. indisten sonra (1. indis dahil) 2 değer silip güncel
değer i ekledi
document.write("silinen değerler: " + oncesilsongraekle + "<br>");
document.write("ekledikten sonraki dizi: " + dizi + "<br>");
</script>
```

Çıktısı:

orjinal dizi : ali,veli,kırkclareli,emret,komutanım
silinen değer : kırkclareli
silindikten sonraki dizi : ali,veli,emret,komutanım
eklenen değer :
ekledikten sonraki dizi : ali,ahmet,mehmet,veli,emret,komutanım
silinen değerler : ahmet,mehmet
ekledikten sonraki dizi : ali,murat,veli,emret,komutanım

4.9 slice()

slice() metodu, kendisine parametre olarak verilen dizi aralığında bulunan değerleri kopyalar ve yeni bir dizi oluşturarak geri döndürür.

Örnek:

```
<script>
var dizi = ["Ali", "Veli", "Kırkclareli", "Foto", "Can", "Telefon"];
document.write("Orjinal dizi : " + dizi + "<br>");

// 2. indisten başlayarak (2. indis dahil) sonuna kadar olan tüm değerleri aldı ve yeni dizi oluşturdu.
var kopya = dizi.slice(2);
document.write("1. kopya dizi : " + kopya + "<br>");

// 2. indisten başlayarak (2. indis dahil) 5. indise kadar (5 hariç) olan tüm değerleri aldı ve yeni dizi oluşturdu.
var kopya2 = dizi.slice(2, 5);
document.write("2. kopya dizi : " + kopya2 + "<br>");

// 2. indisten başlayarak (2. indis dahil) sonuncu değer hariç tüm değerleri aldı ve yeni dizi oluşturdu.
var kopya3 = dizi.slice(2, -1);
document.write("3. kopya dizi : " + kopya3 + "<br>");

// sondan başlayarak 4. indis ile sondan 2. indise kadar olan değerleri al ve yeni dizi oluştur. (-4 dahil, -2 hariç oluyor)
var kopya4 = dizi.slice(-4, -2);
document.write("4. kopya dizi : " + kopya4 + "<br>");

// sondan başlayarak 4. indis ile en sona kadar olan değerleri al ve yeni dizi oluştur. (-4 dahil)
var kopya5 = dizi.slice(-4);
document.write("5. kopya dizi : " + kopya5 + "<br>");
</script>
```

Çıktısı:

Orjinal dizi : Ali,Veli,Kırklareli,Foto,Can,Telefon

1. kopya dizi : Kırklareli,Foto,Can,Telefon

2. kopya dizi : Kırklareli,Foto,Can

3. kopya dizi : Kırklareli,Foto,Can

4. kopya dizi : Kırklareli,Foto

5. kopya dizi : Kırklareli,Foto,Can,Telefon

4.10 length

String bir ifadenin uzunluğunu yani karakter sayısını döndürür. Eğer belirttiğiniz string ifade değeri boş ise 0 döndürecektir. Aynı şekilde dizinin eleman sayısını da verir.

Not: Javascript de **length** özelliği boşluk, nokta, virgül gibi özel karakterleri de sayar yani her şeyi sayar.

Örnek:

```
<script>
var stringifade = "ali veli kırklareli emret komutanım diyeceksiniz";
var dizi = [15, 24, 56, "Merhaba", "Ahmet", true, false]

document.write("String ifade içeriği: <br> " + stringifade + "<br> <br>");
document.write("Dizi içeriği: <br> " + dizi + "<br> <br>");

var string_uzunluk = stringifade.length;
var dizi_uzunluk = dizi.length;

document.write("String ifadenin karakter uzunluğu : " + string_uzunluk + "<br>");
document.write("Dizinin uzunluğu : " + dizi_uzunluk);
</script>
```

Çıktısı:

String ifade içeriği:
ali veli kırklareli emret komutanım diyeceksiniz.

Dizi içeriği:
15,24,56,Merhaba,Ahmet,true,false

String ifadenin karakter uzunluğu : 49
Dizinin uzunluğu : 7

4.11 reverse()

Dizi elemanlarını tersten sıralar . Dizinin ilk elemanı son eleman olur, son elemanı ilk eleman olur.

Örnek:

```
<script>
var dizi = [15, 24, 56, "Merhaba", "Ahmet", true, false];
document.write("Normal Dizinin Hali : " + dizi + "<br>");
var ters_dizi = dizi.reverse();
document.write("Ters Çevrilmiş Dizinin Hali : " + ters_dizi + "<br>");
</script>
```

Çıktısı:

Normal Dizinin Hali : 15,24,56,Merhaba,Ahmet,true,false
Ters Çevrilmiş Dizinin Hali : false,true,Ahmet,Merhaba,56,24,15

4.12 sort()

String olan dizi elemanlarını alfabetik sıraya göre sıralar. Sayılar için sort fonksiyonu kullanılmaz. Girilen String ifadelerin ilk karakterlerine bakarak alfabetik sıralar. İlk karakteri aynı olan string ifadelerin ikinci karakterine göre sıralamaya yapar. İkinci karakterde aynı ise 3. karaktere bakar. İşlem bu şekilde devam ederek sıralamayı yapar.

Örnek:

```
<script>
var dizi = ["Ahmet", "Fatma", "Selim", "Kemal", "Zeynep", "Mustafa", "Musa"];
var sirali_dizi = dizi.sort();
document.write("Normalde Dizi: " + dizi + "<br>");
document.write("Sıralı Dizi: " + sirali_dizi);
</script>
```

Çıktısı:

Normalde Dizi: Ahmet,Fatma,Kemal,Musa,Mustafa,Selim,Zeynep
Sıralı Dizi: Ahmet,Fatma,Kemal,Musa,Mustafa,Selim,Zeynep

4.13 indexOf()

Argüman olarak verilen elemanın dizi de ilk görüldüğü index'i verir. Yani aranan argümandan bir den fazla bulunuyorsa ilk bulunduğunun index ini verir. Eğer bu eleman array de bulunmuyorsa -1 değeri döner.

Örnek:

```
<script>
var dizi = ["Ahmet", "Fatma", "Selim", "Kemal", "Zeynep", "Mustafa", "Musa"];
document.write(dizi.indexOf("Selim") + "<br>");
document.write(dizi.indexOf("Çetin"));
</script>
```

Çıktısı:

```
2
-1
```

4.14 some()

Bu method ile yapacağımız şey çok basit. Belirli bir array içerisinde vereceğimiz koşula uyan tek bir eleman dahi bulsa true dönecek olan bir method.

Burada some methodu bir callback ve index alıyor parametre olarak. Callback fonksiyonu içerisindeki koşul true dönene kadar çalışır. Truthy dediğimiz değeri bulduğu an çalışmayı durdurur ve true döner. Eğer öyle bir değer bulamazsa false döner.

Örnek:

```
<script>
var years = [1989, 2000, 2018, 1970];
// 4 adet sene içeren bir arrayimiz olsun.

var sonuc1 = years.some(function (year) {
    return year >= 2000; // 2000'den büyük olan yıllardan varsa
});
var sonuc2 = years.some(function (year) {
    return year <= 1900; // 1900'den küçük olan yıllardan varsa
});

document.write(sonuc1 + "<br>");
document.write(sonuc2);
</script>
```

Çıktısı:

```
true  
false
```

5. Javascript Bazı Hazır Fonksiyonlar

5.1 encodeURIComponent()

Bir url yi kodlamak amaçlı kullanılır. Url nin şifrelenmiş olarak gözükmesini sağlar.

Örnek:

```
<script>  
var url = "https://lorem-impüs-öçş.com";  
document.write(url + "<br>");  
document.write(encodeURIComponent(url) + "<br>");  
</script>
```

Çıktısı:

```
https://lorem-impüs-öçş.com  
https://lorem-%C4%B1mp%C3%BCs-%C3%B6%C3%A7%C5%9F.com
```

5.2 decodeURI()

encodeURIComponent ile kodlanmış url yi normal hale çevirir.

Örnek:

```
<script>  
var url = "https://lorem-impüs-öçş.com";  
document.write(url + "<br>");  
document.write(encodeURIComponent(url) + "<br>");  
document.write(decodeURI(url))  
</script>
```

Çıktısı:

```
https://lorem-impüs-öçş.com  
https://lorem-%C4%B1mp%C3%BCs-%C3%B6%C3%A7%C5%9F.com  
https://lorem-impüs-öçş.com
```

5.3 eval()

JavaScript eval metodu, aldığı karakter dizisi parametresini, dinamik olarak JavaScript kaynak kodlarına çevirerek çalıştırılmasını sağlar. **eval** metodu JavaScript'in temel fonksiyonlarından biridir ve herhangi bir nesnesi ile ilişkisi yoktur.

eval metodunun aldığı karakter dizisi parametresi bir JavaScript deyimini (komutu, fonksiyonu, nesnesi, özelliği vs.) ifade ediyorsa JavaScript kodlarına çevrilip çalıştırılır. Eğer karakter dizisi bir veya daha fazla operatör kodunu içeriyorsa operatörün işlemi gerçekleştirilir. Bu yüzden özellikle matematiksel işlemleri içeren/ifade eden karakter dizileri kullanılmamalıdır. **eval** otomatik olarak matematiksel işlemi gerçekleştirecektir.

Örnek:

```
<script>  
var a = 10;  
var b = 20;  
c = 'a+b';  
document.write(c + "<br>");  
document.write(eval(c))  
</script>
```

Çıktısı:

```
a+b  
30
```

5.4 isFinite()

Global **isFinite()** fonksiyonu girilen değerin sonlu sayı olup olmadığını kararlaştırır. Gerekliyse, parametre ilk önce sayıya çevrilir. Bu fonksiyonu bir sayının sonlu olup olmadığını

kararlařtırmak için kullanabilirsiniz. Eğer argümanın değeri NaN, pozitif sonsuz veya negatif sonsuz ise, metod **false** döndürür; deęilse, **true** döndürür.

Örnek:

```
<script>
isFinite(SonsuzSayı); // false
isFinite(NaN); // false
isFinite(-SonsuzSayı); // false

isFinite(0); // true
isFinite(2e64); // true
isFinite(null); // true

isFinite("0");
// true, daha güçlü bir ifade olan Number.isFinite("0") ile kullanılsaydı false olacaktı.
</script>
```

5.5 isNaN()

Bir değerin NaN olup olmadığını belirler. Eğer NaN ise true değeri döndürür.

Örnek:

```
<script>
function milliseconds(x) {
  if (isNaN(x)) {
    return 'Bir sayı değildir';
  }
  return x * 1000;
}

document.write(milliseconds("5748F") + "<br>");
document.write(milliseconds("5"));
</script>
```

Çıktısı:

Bir sayı değildir
5000

5.6 Number()

String olarak girilmiş bir değeri sayıya yani number a çevirir.

Örnek:

```
<script>
var deneme = "123";
sayi1 = Number(deneme);
document.write(typeof sayi1);
</script>
```

5.7 parseFloat()

parseFloat() metodu, kendisine parametre olarak verilen değerlere göre verilen değerleri ondalıklı sayı haline dönüştürür ve yeni bir değişken oluşturarak geri döndürür. Artı, eksi, nokta haricinde bir değer gördüğünde işlemi durdurur. Girilen değerın veri türü ne olursa olsun **number** veri türüne dönüşür. Ancak girilen değer kabul ettiği değerler haricinde olursa NaN sonucunu verir.

Örnek:

```
<script>
var sayi1 = 22;
var sayi2 = "22";
var sayi3 = -30;
var sayi4 = "+239";
var sayi5 = "Sayı 5";
var sayi6 = "5 6 7 8 9";
var sayi7 = 13.2;

var ciktil = parseFloat(sayi1);
var ciktil2 = parseFloat(sayi2);
var ciktil3 = parseFloat(sayi3);
var ciktil4 = parseFloat(sayi4);
var ciktil5 = parseFloat(sayi5);
var ciktil6 = parseFloat(sayi6);
var ciktil7 = parseFloat(sayi7);

// sayi 1
document.write("sayi1 : " + sayi1 + "<br>");
document.write("sayi2 veri türü : " + typeof (sayi2) + "<br>");
document.write("ciktil : " + ciktil + "<br>");
document.write("ciktil veri türü : " + typeof (ciktil) + "<br>");

// sayi 2
document.write("sayi2 : " + sayi2 + "<br>");
document.write("sayi2 veri türü : " + typeof (sayi2) + "<br>");
document.write("ciktil2 : " + ciktil2 + "<br>");
document.write("ciktil2 veri türü : " + typeof (ciktil2) + "<br>");

// sayi 3
document.write("sayi3 : " + sayi3 + "<br>");
document.write("sayi3 veri türü : " + typeof (sayi3) + "<br>");
document.write("ciktil3 : " + ciktil3 + "<br>");
document.write("ciktil3 veri türü : " + typeof (ciktil3) + "<br>");
```

```
// sayi 4
document.write("sayi4: " + sayi4 + "<br>");
document.write("sayi4 veri türü: " + typeof (sayi4) + "<br>");
document.write("cikti4: " + cikti4 + "<br>");
document.write("cikti4 veri türü: " + typeof (cikti4) + "<br>");

// sayi 5
document.write("sayi5: " + sayi5 + "<br>");
document.write("sayi5 veri türü: " + typeof (sayi5) + "<br>");
document.write("cikti5: " + cikti5 + "<br>");
document.write("cikti5 veri türü: " + typeof (cikti5) + "<br>");

// sayi 6
document.write("sayi6: " + sayi6 + "<br>");
document.write("sayi6 veri türü: " + typeof (sayi6) + "<br>");
document.write("cikti6: " + cikti6 + "<br>");
document.write("cikti6 veri türü: " + typeof (cikti6) + "<br>");

// sayi 7
document.write("sayi7: " + sayi7 + "<br>");
document.write("sayi7 veri türü: " + typeof (sayi7) + "<br>");
document.write("cikti7: " + cikti7 + "<br>");
document.write("cikti7 veri türü: " + typeof (cikti7) + "<br>");

</script>
```

Çıktısı:

sayi1 : 22
sayi1 veri türü : number
ciktil : 22
ciktil veri türü : number

sayi2 : 22
sayi2 veri türü : string
cikti2 : 22
cikti2 veri türü : number

sayi3 : -30
sayi3 veri türü : number
cikti3 : -30
cikti3 veri türü : number

```
sayi4 : +239  
sayi4 veri türü : string  
cikti4 : 239  
cikti4 veri türü : number
```

```
sayi5 : Sayı 5  
sayi5 veri türü : string  
cikti5 : NaN  
cikti5 veri türü : number
```

```
sayi6 : 5 6 7 8 9  
sayi6 veri türü : string  
cikti6 : 5  
cikti6 veri türü : number
```

```
sayi7 : 13.27  
sayi7 veri türü : number  
cikti7 : 13.27  
cikti7 veri türü : number
```

5.8 parseInt()

parseInt() metodu, kendisine ondalıklı olarak verdiğiniz değerin ondalık kısmını siler ve kalan kısımdan yeni bir değişken oluşturarak döndürür. **parseInt()**, veri türü ne olursa olsun girilen değeri **number** veri türüne dönüştürecektir. Yani string bir değer de girseniz işlem sonucu number olacaktır. Buna ek olarak, fonksiyona girdiğiniz ikinci parametre ile dönüştürmek istediğiniz sayı sistemine dönüşüm yaptırabilirsiniz. Eğer ikinci parametre girilmezse default olarak onluk sayı sistemine dönüşüm olur.

Örnek:

```
<script>  
var sayi = 78.5643;  
document.write("Sayı Değeri: " + sayi + "<br>");  
document.write("Veri Türü: " + typeof sayi + "<br><br>");  
var tamsayi = parseInt(sayi);  
document.write("Tam Sayı Değeri: " + tamsayi + "<br>");  
document.write("Tam Sayı Veri Türü: " + typeof tamsayi + "<br><br><br>");  
// string veri türü örneği  
var sayi = "132.79";  
document.write("Sayı Değeri: " + sayi + "<br>");  
document.write("Veri Türü: " + typeof sayi + "<br><br>");  
var tamsayi = parseInt(sayi);  
document.write("Tam Sayı Değeri: " + tamsayi + "<br>");  
document.write("Tam Sayı Veri Türü: " + typeof tamsayi);  
</script>
```

Çıktısı:

Sayı Değeri: 78.5643

Veri Türü : number

Tam Sayı Değeri : 78

Tam Sayı Veri Türü : number

Sayı Değeri: 132.79

Veri Türü : string

Tam Sayı Değeri : 132

Tam Sayı Veri Türü : number

5.9 String()

String dışında verilen bütün veri tiplerini string tipine dönüştürür.

Örnek:

```
<script>
var sayi = 123456;
document.write(typeof String(sayi));
</script>
```

Çıktısı:

string

5.10 isArray()

Girilen parametrenin bir dizi olup olmadığını kontrol eder. true veya false olarak geri sonuç döndürür.

Örnek:

```
<script>
var ornek1 = ["Mavi", "Siyah", "Yeşil"];
var ornek2 = 10
kontrol = Array.isArray(ornek1);
kontrol2 = Array.isArray(ornek2);
document.write(kontrol + "<br>");
document.write(kontrol2);
</script>
```

Çıktısı:

```
true
false
```

6. JavaScript Math Nesnesi Metotları

Aşağıdaki özellikler math nesnesinin alt özellikleridir. Bu yüzden math. koyduktan sonra istediğiniz alt özelliği kullanınız. Örnek: math.ceil(), math.random()

ceil(x) → Ondalıkli sayıyı kendisinden sonraki ilk tam sayıya yuvarlar.

floor(x) → ondalıklı sayının ondalıklı kısmını kaldırır.(aşağı yuvarlar)

random() → 0-1 arasında rastgele sayı üretir.

min(a,b,c,d,...,z) → Kendisine parametre olarak gönderilen değerler arasında en küçük değeri getirir.

max(a,b,c,d,...,z) → Kendisine parametre olarak gönderilen değerler arasında en büyük değeri getirir.

round(x) → Ondalıkli sayıyı en yakın tam sayıya yuvarlar.

sqrt(x) → Kendisine parametre olarak gönderilen sayının karekökünü verir.

cbrt(x) → Kendisine parametre olarak gönderilen sayının küp kökünü verir.

pow(x,y) → x taban, y üs olacak şekilde sayının kuvvetini hesaplar.

abs(x) → Sayının mutlak değerini verir.

exp(x) → e^x degerini getirir.

trunc(x) → Kendisine parametre olarak verilen ondalıklı sayının ondalıklı kısmını kaldırarak tam sayı değerini verir.

log(x) → e tabanındaki doğal logaritmasını verir.

E → Bu özellik matematikteki euler sabitini verir.

PI → Bu özellik matematikteki Pi sabitini verir.

SQRT2 → Bu özellik matematikteki 2 sayısının karekökünü verir.

SQRT1_2 → Bu özellik matematikteki $1/2$ sayısının karekökünü verir.

isInteger() → İçerisine girilen değerin tam sayı olup olmadığını true veya false olarak geri döndürür.

isFinite() → Kendisine parametre olarak verilen değerin maksimum sayı değeri ve minimum sayı değerinin arasında olup olmadığını kontrol eder ve true ve false olarak geri döndürür.

sin(x) → Kendisine parametre olarak verilen değerin sinüs değerini geriye döndürür.

sinh(x) → Kendisine parametre olarak verilen değerin hiperbolik sinüs değerini geriye döndürür.

asin(x) → Kendisine parametre olarak verilen -1 ile 1 arasındaki sayı değerinin ark sinüsünü radyan cinsinden bularak, bulunduğu değeri geriye döndürür.

asinh(x) → Kendisine parametre olarak verilen değerin hiperbolik ark sinüs değerini geriye döndürür.

cosh(x) → Kendisine parametre olarak verilen değerin hiperbolik cosinüs değerini geriye döndürür.

acos(x) → Kendisine parametre olarak verilen -1 ile 1 arasındaki sayı değerinin ark cosinüsünü radyan cinsinden bularak, bulunduğu değeri geriye döndürür.

acosh(x) → Kendisine parametre olarak verilen değerin hiperbolik ark cos değerini geriye döndürür.

tan(x) → Kendisine parametre olarak verilen değerin tanjan değerini geriye döndürür.

tanh(x) → Kendisine parametre olarak verilen değerin hiperbolik tanjan değerini geriye döndürür.

atan(x) → Kendisine parametre olarak verilen -1 ile 1 arasındaki sayı değerinin ark tanjantını radyan cinsinden bularak, bulunduğu değeri geriye döndürür.

atanh(x) → Kendisine parametre olarak verilen değerin hiperbolik ark tanjan değerini geriye döndürür.

Toplu Örnekler :

```
<script>
alert(Math.random()); //sonuç: 0-1 arasında rastgele ondalıklı sayı

alert(Math.min(10,-7,15,10,-3)); // sonuç: -7

alert(Math.max(10,-7,15,10,-3)); //sonuç: 15

alert(Math.round(4.7)); // sonuç: 5
alert(Math.round(4.4)); // sonuç: 4

alert(Math.ceil(4.4)); // sonuç: 5

alert(Math.abs(-20)); // sonuç: +20

alert(Math.exp(5)); // sonuç: 148.413159103

alert(Math.log(3)); // sonuç: 1.0986122886681096

alert(Math.floor(4.7)); // sonuç: 4

alert(Math.sqrt(4)); // sonuç: 2

alert(Math.pow(2,5)); // sonuç: 32

alert(Math.floor(Math.random() * 11)); // 0-10 arasında rastgele sayı üretir.

alert(Math.floor(Math.random() * 11 + 20)); // 20-30 arasında rastgele sayı üretir

alert(Math.abs(-210.6)); // Sonuç: 210.6

</script>
```

6.1 isSafeInteger()

Kendisine parametrik olarak verilen değişken içeriğinin güvenli bir tam sayı olup olmadığını kontrol ederek boolean(true,false) veri türünde sonucu geriye döndürür.

Örnek:

```
<script>
var sayi1 = 55;
var sayi2 = 4892311894651981 * 1471315665196;

document.write("Sayımız: " + sayi1 + " Sonuc: " + Number.isSafeInteger(sayi1) + "<br>")
//Sayi 2 tam sayı olmasına rağmen çok büyük bir sayı olması onu güvensiz yapar. Bu yüzden sonuç false döner.
document.write("Sayımız: " + sayi2 + " Sonuc: " + Number.isSafeInteger(sayi2) + "<br>")
</script>
```

Çıktısı:

```
Sayımız: 55 Sonuc: true
Sayımız: 7.198135129626183e+27 Sonuc: false
```

6.2 toPrecision()

Kendisine parametre olarak verilen değer doğrultusunda değişken içerisindeki sayının kaç basamak olacağını belirleyerek yeni bir içerik oluşturur ve oluşturduğu içeriği geriye döndürür. (Nokta hariç.)

Örnek:

```
<script>
var sayi1 = 14575;
var sayi2 = 154.547847;

document.write("Sayımız: " + sayi1 + " Sonuc: " + sayi1.toPrecision(3) + "<br>")
document.write("Sayımız: " + sayi2 + " Sonuc: " + sayi2.toPrecision(5) + "<br>")
</script>
```

Çıktısı:

```
Sayımız: 14575 Sonuc: 1.46e+4  
Sayımız: 154.547847 Sonuc: 154.55
```

6.3 toExponential()

Kendisine parametre olarak belirlenen değer doğrultusunda içeriğindeki sayının ondalık hanesinde kaç basamak olacağını belirleyerek, kalan değeri üstel hale getirerek yeni bir içerik oluşturur ve oluşturduğu içeriği geriye döndürür.

Örnek:

```
<script>  
var sayi1 = 4.986;  
var sayi2 = 8.547847;  
document.write("Sayımız: " + sayi1 + " Sonuc: " + sayi1.toExponential(2) + "<br>");  
document.write("Sayımız: " + sayi2 + " Sonuc: " + sayi2.toExponential(5) + "<br>");  
</script>
```

Çıktısı:

```
Sayımız: 4.986 Sonuc: 4.99e+0  
Sayımız: 8.547847 Sonuc: 8.54785e+0
```

6.4 sign()

Kendisine parametre olarak verilen herhangi bir sayı değerinin işaretini bularak sonucunu geriye döndürür. sayı pozitif bir sayı ise 1, negatif bir sayı ise -1, 0 ise 0, ve eğer bir sayı değil ise NaN değerini geriye döndürür.

Örnek:

```
<script>
var sayil = 54;
var sayi2 = -20;
var sayi3 = 0;
var sayi4 = "Musa";

document.write("Sayımız: " + sayil + " Sonuc: " + Math.sign(sayil) + "<br>");
document.write("Sayımız: " + sayi2 + " Sonuc: " + Math.sign(sayi2) + "<br>");
document.write("Sayımız: " + sayi3 + " Sonuc: " + Math.sign(sayi3) + "<br>");
document.write("Sayımız: " + sayi4 + " Sonuc: " + Math.sign(sayi4) + "<br>");
</script>
```

Çıktısı:

```
Sayımız: 54 Sonuc: 1
Sayımız: -20 Sonuc: -1
Sayımız: 0 Sonuc: 0
Sayımız: Musa Sonuc: NaN
```

7. JavaScript Date Nesnesi Metodları

var tarih = new Date() → Date fonksiyonu ile o anki tüm tarihi yıl ay gün saat dakika saniyesine kadar olan veriyi tarih değişkenine aktarır.

getFullYear() → 4 basamaklı yılı verir.

getMonth() → 0(Ocak) – 11 (Aralık) arasında sayı

getDay → 0(Pazar) – 6 (Cumartesi)

getDate() → 1-31 arasında sayı

getHours() → 0-23 //saat

getMinutes() → 0-59 //dakika

getSeconds() → 0-59 //saniye

getMilliseconds() → 0-99 //milisaniye

getTime() → 1 Ocak 1970 yılından şuana kadar geçen milisaniye cinsinden sayı döndürür.

setDate() → Aktif ayın tarihini değiştirir.

setFullYear() → Yılı değiştirir.

setHours() → 0-23 arasında saati değiştirir.

setMilliseconds() → 0-999 arasında milisaniyeyi değiştirir.

setMinutes() → 0-59 arasında dakikayı değiştirir.

setMonth() → 0-11 arasında ayı değiştirir.

setSeconds() → 0-59 arasında saniyeyi değiştirir.

setTime() → Milisaniye cinsinden yeni tarih üretir.(1970 yılından başlayarak)

toString() → Date nesnesi ile üretilen değeri string formatına dönüştürür.

Örnekler:

Get Örnekleri :

Örneklerin çıktılarını yorum satırı olarak verildi.

```
<script>
var tarih = new Date("10/25/2014 14:25:05"); //Sat Oct 25 2014 14:25:05
alert(tarih.getFullYear()); //2014
alert(tarih.getDay()); //6
alert(tarih.getDate()); //25
alert(tarih.getMinutes()); //25
alert(tarih.getSeconds()); //05
alert(tarih.getMilliseconds()); //0
alert(tarih.getTime()); //1414236305000 (1970den belirtilen tarihe kadar geçen milisaniye)
</script>
```

Set Örnekleri

Set örnekleri sonradan eklenecek...

7.1 JavaScript ile Günler Türkçe Yazdırma

getDay() komutunun günleri 0(Pazar) -6 (Cumartesi) olacak şekilde aldığı biliyoruz. Günleri ekrana Türkçe yazdırma için günler adında bir dizi oluşturup ilgili index(sıra numarası) tarafından gösterilen dizi elemanını ekrana yazdıracağız.

Örnek:

```
<script>
var gunler = ["pazar", "pazartesi", "salı", "çarşamba", "perşembe", "cuma", "cumartesi"];
var tarih = new Date("2012-10-24");
alert(gunler[tarih.getDay()]); //çarşamba
</script>
```

7.2 JavaScript ile Aylar Türkçe Yazdırma

getMonth() komutunun günleri 0(Ocak) - 12(Aralık) olacak şekilde aldığı biliyoruz. Günleri ekrana Türkçe yazdırma için günler adında bir dizi oluşturup ilgili index(sıra numarası) tarafından gösterilen dizi elemanını ekrana yazdıracağız.

```
<script>
var aylar = ["ocak", "şubat", "mart", "nisan", "mayıs", "haziran", "temmuz", "ağustos", "eylül", "ekim",
"kasım", "aralık"];
var tarih = new Date("2012-10-24");
alert(aylar[tarih.getMonth()]); // ekim
</script>
```

==> Daha fazla Date metot'u için bakmanızı önerdiğim kaynak: [Tıkla](#)

8. Javascriptteki Yorum Satırları

Yorum satırları ne işe yarar? Yorum satırları kodlamadaki bilgilendirme alanlarıdır. İçerisine yazılan her hangi bir şeyi tarayıcılar kodlama olarak algılanmaz. Yorum satırı candır bolca kullanın...😁

İki tür yorum satırı kodlaması vardır.

1. Satır başına // koyarak.
 2. /* */ kullanarak
1. kullanım sadece tek satırlı yorum satırı olarak gözükmelerini sağlar.
2. kullanım ise birden çok satırlı yorum satırı olarak göstermenizi sağlar.

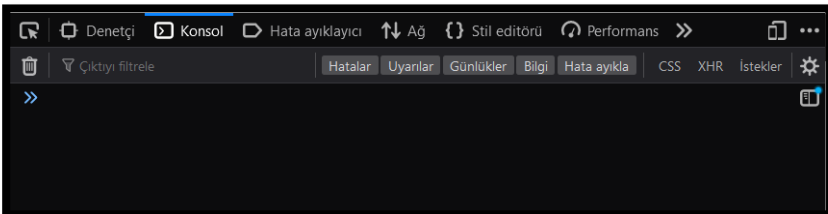
Yani kısacası tek satırlık yorum alanı yapacak iseniz 1. kullanımı kullanmalısınız. Çoklu satırlı yorum alanı yapacak iseniz 2. kullanımı kullanmalısınız. Yorum satırı yapmak bu kadar basit. 😊

Örnek:

```
1 // Tek satırlık yorum satırı. console.log("Deneme")
2
3 /*
4
5 Çoklu satırlı yorum satırı
6
7 console.log("Deneme1");
8 console.log("Deneme2");
9 console.log("Deneme3");
10
11
12 */
```

Çıktısı:

Çıktıda gözüktüğü gibi bir çıktı yok. console.log() fonksiyonu kod olarak algılanmadı.



9. Javascriptteki Yazım Kuralları

⚠ JavaScript'teki bütün ifadeler noktalı virgül ile bitmelidir. Eğer noktalı virgül koymadığınız takdirde beklenmeyen hatalar veya sonuçlar ile karşılaşılabilir. Aslında bir değişken tanımlarken satırın sonuna noktalı virgül koymak şart değildir. JS yorumlayıcılar satırının sonunu escape character marifetiyle bulur. Fakat ileride .js dosyalarımızın boyutlarını küçülmek için kullanacağımız JSMinifier, YUI Compressor tarzı uygulamalar satırın sonundaki escape character 'i (n) sildiği için kodunuzda hata çıkma olasılığı yüksektir.

⚠ Rezerv Edilmiş Kelimeler

abstract	arguments	await*
break	byte	case
char	class*	const
debugger	default	delete
double	else	enum*
export*	extends*	false
finally	float	for
goto	if	implements
in	instanceof	int
let*	long	native
null	package	private

public

return

short

super*

switch

synchronized

throw

throws

transient

try

typeof

var

volatile

while

with

Yukarıda belirtilen rezerv edilmiş kelimeler dışında değişkeninize istediğiniz ismi verebilirsiniz. (JavaScript'te bütün değişkenler harf, alt çizgi (_) veya dolar (\$) işareti ile başladığını unutmayın)

⚠ JavaScript büyük-küçük karakterler konusunda hassas bir dildir. Bu nedenle **değişken**

Değişken

aynı şey değildir. Eğer herhangi problem yaşamak istemiyorsanız bu kurala mutlaka dikkat edin.

⚠ Ne kadar boşluk kullanır sanız kullanın boşluklar dikkate alınmaz.

⚠ Kesinlikle değişken isimlerinde türkçe karakter kullanmayınız.(ı,ş,ö,ğ,...)

⚠ Değişken tanımlar iken (var) ifadesi kullanılmalı.

10. Javascriptte Özel İşaretlerin Kullanımı

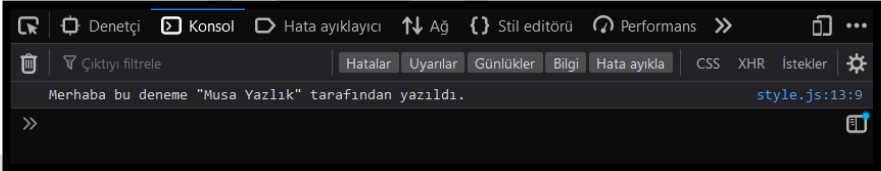
JavaScript kodlarında eğer özel karakterler kullanma ihtiyacı duyarsak özel karakterin başına \ işaretini eklememiz gerekir. Eklenebilecek özel karakterleri de tablo halinde verirsek:

Kod	Çıktısı
\'	Tek tırnak
\"	Çift Tırnak
\f	besleme oluşturmak
\&	ve işareti
\b	backspace
\\	ters slash
\t	tab
\n	Yeni Satır ekleme
\r	satır başı

Örnek: bir kullanım:

```
<script>
console.log("Merhaba bu deneme \"Musa Yazlık\" tarafından yazıldı.")
</script>
```

Çıktısı:



11. Değişkenler

11.1 Değişkenlere Giriş ve Yazım Kuralları

Şimdi de javascripte değişken tanımlar iken hangi kurallara uymamız gerektiğini öğreneceğiz.

1. Değişken isimlerinde harf,rakam,alt çizgi ve dolar işareti olabilir.
2. Değişken isimleri harf,alt çizgi veya dolar işareti ile başlayabilir rakam ile başlayamaz.
3. Değişken isimleri BÜYÜK küçük harf duyarlıdır. (a ve A farklı değişkenlerdir)
4. Değişken isimleri arasında boşluk bırakılmaz.
5. Javascript anaktar(Rezerv Edilmiş) kelimeleri değişken adı olarak kullanılamaz.

- Değişken isimlerinde türkçe karakter kullanılabilir ancak kullanılması tavsiye edilmez. Ben ise kesinlikler kullanmıyorum. İleride sıkıntı çıkarabiliyor.

11.2 Değişken Türleri

Kodlamalarda değişkenleri mutlaka bir türü bulunur. Varsayılan olarak bir çok programlama dili değişkenlere string türü vermekte. Javascript te değişkenlerin tipini öğrenebilmek için **typeof** fonksiyonunu kullanıyoruz. Şimdi ise javascripte değişken türlerini göreceğiz.

- String (Yazı tipi)
- Number (Sayı rakam tipi)
- Boolean (true,false)
- Null (Boş)

11.3 Değişken Kapsamı

Javascriptte kapsama alanına göre iki tür değişken kapsamı vardır.

- Yerel değişken
- Genel Değişken

Not: Burada biraz fonksiyonlara gireceğiz ama onları kafaya takmayın. İleriki sayfalarda fonksiyonlara detaylı bir şekilde gireceğiz.

11.3.1 Yerel Değişken Nedir?

Bir JavaScript fonksiyonu içinde değişken tanımlıyorsanız bunun kapsamı yerel olur. Bir fonksiyon içinde tanımlanan değişken **yerel değişken** olarak adlandırılır. Yani bu değişkene sadece fonksiyonun içinden erişebilirsiniz. Fonksiyon dışından bu değişkene erişemezsiniz.

```
<script>
// Burada yerelan kodlar ad değişkenine erişemez.
function fonksiyonum() {
  var ad = "Ali Veli";
  // Bu satırlar arasında yerelan bir JavaScript kodu
  // ad değişkenini kullanabilir.
}
</script>
```

11.3.2 Genel Değişken Nedir?

Eğer bir değişken fonksiyon dışında tanımlanıyorsa bu Global kapsama sahiptir. Fonksiyonların dışında tanımlanan değişken **Global Değişken** olarak adlandırılır. Global Kapsamın anlamı, o sayfadaki tüm fonksiyonlar bu değişkene erişebilir ve kullanabilir demektir.

```

<script>
//Global değişken
var ad = "Ali Veli";
// Her iki fonksiyonda ad değişkenine erişebilir
// Onu kullanabilir.
function fonksiyon1() {
    alert(ad);
}
function fonksiyon2() {
    alert(ad);
}
</script>

```

12. Düzenli İfadeler

Belirli bir kalıba göre arama yapmak için kullanılan Javascript düzenli ifadeler, search, replace, test, exec ve düzenli ifade özelliklerinin kullanımı ile ilgili bilgiler yer alıyor. JavaScript düzenli ifadeler arama kriteri oluşturur ve metin veri türü üzerinde arama ve değiştirme yapmak için kullanılır.

i → Karakter diziliminin karşılaştırılmasında büyük harf / küçük harf ayırımının yapılıp yapılmayacağını tanımlamasında kullanılır.

g → Karakter diziliminin tamamının karşılaştırılma kontrol edilip edilmeyeceğini tanımlamada kullanılır.

m → Karakter diziliminin birden fazla satırı varsa tüm satırlarının karşılaştırmada kontrol edilip edilmeyeceğini tanımlamada kullanılır.

12.1 Düzenli ifadelerde kullanılan bazı metotlar

12.1.1 match()

Kendisine parametre olarak verilen düzenli ifadeli değer doğrultusunda değişken içeriğini kontrol eder ve aranan değerlerin eşleşmesi durumunda eşleşmeyi sağlayan değişken içeriği değerlerinden yeni bir dizi oluşturarak, oluşturduğu dizi değerini geri döndürür. Aranan değer bulunamaz ise null değeri geri döner.

Örnek:

```

<script>
var mesaj = "Lorem ipsum dolor, sit amet consectetur musa adipiscing elit."
var aranan = new RegExp("musa"); // /musa/ bu şekilde de yazılabilir.
var aranan1 = new RegExp("mehmet");
var yaz = mesaj.match(aranan);
var yaz1 = mesaj.match(aranan1);
document.write(yaz + "<br>");
document.write(yaz1);
</script>

```

Çıktısı:

```
musa  
null
```

12.1.2 global

Düzenli ifade içerisinde **g** ayarının kullanılıp kullanılmadığını kontrol ederek boolean(true,false) veri türünde geriye dönürür.

Örnek:

```
<script>  
var mesaj = "Lorem ipsum dolor, sit amet consectetur musa adipiscing elit."  
var aranan = new RegExp("musa", "g"); // musa/ bu şekilde de yazılabilir.  
var aranan1 = new RegExp("musa");  
var yaz = mesaj.match(aranan);  
var yaz1 = mesaj.match(aranan1);  
document.write(aranan.global + "<br>");  
document.write(aranan1.global);  
</script>
```

Çıktısı:

```
true  
false
```

12.1.3 ignoreCase

Düzenli ifade içerisinde **i** ayarının kullanılıp kullanılmadığını kontrol ederek boolean(true,false) veri türünde geriye dönürür.

Örnek:

```
<script>
var mesaj = "Lorem ipsum dolor, sit amet consectetur musa adipiscing elit."
var aranan = new RegExp("musa","i"); // /musa/ bu şekilde de yazılabilir.
var aranan1 = new RegExp("musa");
var yaz = mesaj.match(aranan);
var yaz1 = mesaj.match(aranan1);
document.write(aranan.ignoreCase + "<br>");
document.write(aranan1.ignoreCase);
</script>
```

Çıktısı:

```
true
false
```

12.1.4 multiline

Düzenli ifade içerisinde **m** ayarının kullanılıp kullanılmadığını kontrol ederek boolean(true,false) veri türünde geriye dönürür.

Örnek:

```
<script>
var mesaj = "Lorem ipsum dolor, sit amet consectetur musa adipiscing elit."
var aranan = new RegExp("musa","im"); // /musa/ bu şekilde de yazılabilir.
var aranan1 = new RegExp("musa");
var yaz = mesaj.match(aranan);
var yaz1 = mesaj.match(aranan1);
document.write(aranan.multiline + "<br>");
document.write(aranan1.multiline);
</script>
```

Çıktısı:

```
true
false
```

12.1.5 test()

Kendisine parametre olarak verilen değişken içeriğinde düzenli ifade ile aranan değer eşleşmesinin olup olmadığını kontrol ederek boolean(true,false) veri türünde sonucu geriye döndürür.

Örnek:

```
<script>
var mesaj = "Lorem ipsum dolor, sit amet consectetur musa adipisicing elit."
var aranan = new RegExp("musa"); // /musa/ bu şekilde de yazılabilir.
var eslesme_var_mi = aranan.test(mesaj)
document.write(eslesme_var_mi + "<br>");
</script>
```

Çıktısı:

true

12.1.6 input

Değişken içeriğinde düzenli ifade ile aranan değer eşleşmesi durumunda eşleşmeyi sağlayan değişkenin tüm değerini geriye döndürür.

Örnek:

```
<script>
var mesaj = "Lorem ipsum dolor, sit amet consectetur musa adipisicing elit."
var aranan = new RegExp("musa"); // /musa/ bu şekilde de yazılabilir.
aranan.test(mesaj)
var eslesme_var_mi = RegExp.input
document.write(eslesme_var_mi + "<br>");
</script>
```

Çıktısı:

Lorem ipsum dolor, sit amet consectetur musa adipisicing elit.

12.1.7 source

Düzenli ifade içerisinde bulunan aranan desen değerini geriye döndürür.

Örnek:

```
<script>
var mesaj = "Havalar nasıl?"
var aranan = new RegExp("musa"); // /musa/ bu şekilde de yazılabilirdi.
var desen = aranan.source
document.write(desen + "<br>");
</script>
```

Çıktısı:

musa

12.1.8 lastIndex

Değişken içeriğinde düzenli ifade ile aranan değer eşleşmesi durumunda ilk eşleşmeyi sağlayan değişken içeriğinin bir sonraki indis/index değerini geriye döndürür.

Örnek:

```
<script>
var mesaj = "Lorem ipsum domusdolor sit amet"
var aranan = new RegExp("musa", "g"); // /musa/ bu şekilde de yazılabilirdi.
aranan.test(mesaj)
document.write(aranan.lastIndex + "<br>");
</script>
```

Çıktısı:

19

12.1.9 leftContext

Değişken içeriğindeki düzenli ifade ile aranılan değer eşleşmesi durumunda eşleşmeyi sağlayan değişken içeriğinin solunda kalan tüm değerleri geriye döndürür. İlk sağlanan değer sol tarafını alır.

Örnek:

```
<script>
var mesaj = "Lorem ipsum dolor sit musa amet"
var aranan = new RegExp("musa", "g"); // /musa/ bu şekilde de yazılabilir.
aranan.test(mesaj)
document.write(RegExp.leftContext + "<br>");
</script>
```

Çıktısı:

Lorem ipsum dolor sit

12.1.10 rightContext

Değişken içeriğindeki düzenli ifade ile aranan değerin eşleşmesi durumunda eşleşmeyi sağlayan değişken içeriğinin sağında kalan tüm değerleri geriye döndürür. İlk sağlanan değerin sağ tarafını alır.

Örnek:

```
<script>
var mesaj = "Lorem ipsum dolor sit musa amet"
var aranan = new RegExp("musa", "g"); // /musa/ bu şekilde de yazılabilir.
aranan.test(mesaj)
document.write(RegExp.rightContext + "<br>");
</script>
```

Çıktısı:

amet

12.1.11 exec()

Kendisine parametre olarak verilen değişken içeriğinde düzenli ifade ile aranan değerlerin eşleşmesi durumunda eşleşmeyi sağlayan değişken içeriği değerinden yeni bir dizi oluşturularak, oluşturduğu dizi değerini geriye döndürür.

NOT: g ayarı kullanılacak ise exec metodu her çağrıldığında lastIndex değeri otomatik olarak eşleşen değerden sonraki değeri alacaktır.

Örnek:

```
<script>

var metin = "musa Musa MuSa MUSA";
var ifadel = RegExp("musa", "g");

// g ifadesini kullandığımızda
document.write(ifadel.exec(metin) + "<br>");
document.write(ifadel.exec(metin) + "<br>");
document.write(ifadel.exec(metin) + "<br>");
document.write(ifadel.exec(metin) + "<br>" + "<hr>");

var ifade2 = RegExp("musa", "i");

// g ifadesini kullanmadığımızda
document.write(ifade2.exec(metin) + "<br>");
document.write(ifade2.exec(metin) + "<br>");
document.write(ifade2.exec(metin) + "<br>");
document.write(ifade2.exec(metin) + "<br>" + "<hr>");

</script>
```

Çıktısı:

```
musa
Musa
MuSa
MUSA

musa
musa
musa
musa
```

Not: g ifadesi kullanılmaz ise lastIndex her aramda 0 olarak algılanacaktır.

12.1.12 compile()

Önceden oluşturulmuş düzenli ifadenin içerisindeki desenin değiştirilmesini sağlar. Yani düzenli ifadede aranan değeri değiştirir.

Örnek:

```
<script>
var metin = "Lorem ipsum dolor ahmet sit amet consectetur adipiscing elit musa. Voluptates, doloribus!";
var ifade = RegExp("Ahmet","i");
var yaz1 = metin.match(ifade);
document.write(yaz1 + "<br>");
ifade.compile("musa","i");
var yaz2 = metin.match(ifade);
document.write(yaz2);
</script>
```

Çıktısı:

```
ahmet
musa
```

12.2 Düzenli İfadelerde Konum Belirleyiciler

12.2.1 ^ konum belirleyicisi

Düzenli ifade dahilinde kontrol edilecek olan değişken içeriğinin diziliminin başlangıcı ile düzenli ifade arasında eşleşme aramak için kullanılır.

Örnek:

```
<script>
var metin = "Lorem ipsum dolor ahmet sit amet consectetur adipiscing elit musa. Voluptates, doloribus!";
var ifade = RegExp("^Ahmet","i");
var ifade2 = RegExp("^Lorem","i");
var yaz1 = ifade.test(metin);
var yaz2 = ifade2.test(metin);
document.write(yaz1 + "<br>");
document.write(yaz2 + "<br>");
</script>
```

Çıktısı:

```
false
true
```

12.2.2 \$ konum belirleyicisi

Düzenli ifade dahilinde aranan değerin istenilen içeriğin en sonunda test metodu ile olup olmadığını kontrol eder. Var ise true yok ise false değeri döndürür.

Örnek:

```
<script>
var mesaj = "deneme mesajı var";
var deger = new RegExp("var$");
var sonuc = deger.test(mesaj);
document.write(sonuc+"<br>");

var mesaj1 = "Bu bir deneme Mesajıdır";
var deger1 = new RegExp("mesaj$", "T");
var sonuc1 = deger1.test(mesaj1);
document.write(sonuc1);
</script>
```

Çıktısı:

```
true
false
```

13. Operatörlere Giriş

13.1 Aritmetik Operatörleri

- + → Toplama
- → Çıkarma
- * → Çarpma
- / → Bölme
- % → Kalan Bulma

Örnek:

```
1 // Toplama işlemi
2
3 var sayi1 = 70;
4 var sayi2 = 6;
5
6 console.log("Toplama işlemi Sonucu: ", sayi1 + sayi2);
7
8 // Çıkarma işlemi
9
10 console.log("Çıkarma işlemi Sonucu: ", sayi1 - sayi2);
11
12 // Çarpma işlemi
13
14 console.log("Çarpma işlemi Sonucu: ", sayi1 * sayi2);
15
16 // Bölme işlemi
17
18 console.log("Bölme işlemi Sonucu: ", sayi1 / sayi2);
19
20 // Kalanı Bulma(Mod)
21
22 console.log("Toplama işlemi Sonucu: ", sayi1 % sayi2);
23
```

Console Output:

İşlem	Sonuç
Toplama İşlemi Sonucu	76
Çıkarma İşlemi Sonucu	64
Çarpma İşlemi Sonucu	420
Bölme İşlemi Sonucu	11.666666666666667
Toplama İşlemi Sonucu	4

13.2 Artırma Ve Azaltma Operatörleri

++ → Değişkenin değerini 1 artırır.

-- → Değişkenin değerini 1 azaltır.

Örnek :

```
1 var sayi = 10;
2
3 // Artırma işlemi
4
5 sayi++;
6 // sayi değeri arttırılabilir.
7 console.log(sayi);
8
9 // Azaltma işlemi
10
11 sayi--;
12 console.log(sayi);
13
```

Console Output:

İşlem	Sonuç
Artırma İşlemi	11
Azaltma İşlemi	10

13.3 Atama Operatörleri

= → Atama Operatörü

+= → Toplar ve Atama Yapar

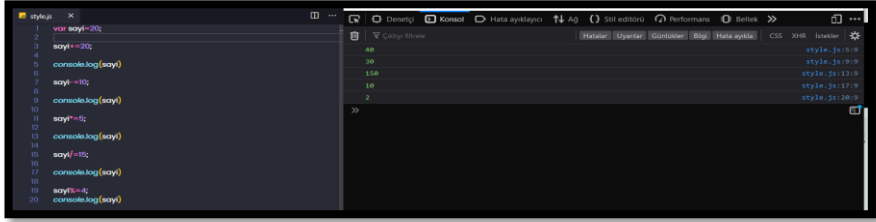
-- → Çıkartır ve Atama Yapar

*= → Çarpar ve Atama Yapar

/= → Bölme ve Atama Yapar

%= → Kalanı Verir ve Atama Yapar.

Örnek:



```
1 var style=20;
2
3 style-->20;
4 console.log(style)
5
6 style-->19;
7 console.log(style)
8
9 style-->18;
10 console.log(style)
11
12 style-->17;
13 console.log(style)
14
15 style-->16;
16 console.log(style)
17
18 style-->15;
19 console.log(style)
20
21 style-->14;
22 console.log(style)
```

13.4 Karşılaştırma Operatörleri

13.4.1 == Eşitse

JavaScript dilinin çalışma anında tür dönüşümü yapabilme özelliği vardır. Eşitlik operatörü ile eşitliğin iki tarafındaki değer **türden bağımsız** olarak aynı ise sonuç **true** değilse **false** olarak dönmektedir.

13.4.2 === Hem değeri hem de türü eşitse

```
<script>
1 == 1 // true
"1" == 1 // true
1 == "1" // true
0 == false // true
0 == null // false
</script>
```

Karşılaştırması yapılan iki değer yada değişken hem tür hemde değer olarak bir biri ile aynı ise karşılaştırma sonucu **true**, tür yada değerlerden biri farklı ise **false** olarak dönmektedir.

```
<script>
3 === 3 // true
3 === '3' // false
</script>
```

13.4.3 != Eşit değilse

Karşılaştırması yapılan iki değer yada değişken arasında türden bağımsız olarak bir birinden **farklı ise true** değilse bir biri ile aynı ise **false** dönmektedir.

```
<script>
1!= 2 // true
1!= "1" // false
1!= 1 // false
1!= true // false
0!= false // false
</script>
```

13.4.4 != Tam Eşit Değilse

Karşılaştırılan veri hem tür hemde değer bakımından bir birinden farklı ise **true**, iki değerde hem tür hemde değer olarak eşitse **false** olarak dönmektedir.

```
<script>
3 != '3' // true
4 != 3 // true
</script>
```

13.4.5 < Küçükse

Karşılaştırması yapılan iki değerden ilk değer ikinci değerden küçükse **true**, ikinci değer küçük yada eşitse **false** sonucunu üretmektedir.

```
<script>
3 < 4 // true
</script>
```

13.4.6 <= Küçük veya eşitse

Karşılaştırması yapılan iki değerden ilk değer ikinci değerden küçük yada eşitse **true**, ikinci değer küçükse **false** sonucunu üretmektedir.

```
<script>
3 <= 4 // true
</script>
```

13.4.7 > Büyükse

Karşılaştırması yapılan iki değerden ilk değer ikinci değerden büyükse **true**, ikinci değer büyük yada eşitse **false** sonucunu üretmektedir.

```
<script>
4 > 3 // true
</script>
```

13.4.8 >= Büyük veya eşitse

Karşılaştırması yapılan iki değerden ilk değer ikinci değerden büyük yada eşitse **true**, ikinci değer büyükse **false** sonucunu üretmektedir.

```
<script>
  4 >= 3 // true
  3 >= 3 // true
</script>
```

13.5 Mantıksal Operatörler

13.5.1 Ve (&&) Operatörü

Belirtilen şartların tümü doğru olduğunda, koşul sağlanmış sayılır. Şartlardan biri bile yanlış ise koşul sağlanmamış sayılır.

Örnek:

Örneğin hem x değişkeninin 50 ile 100 arasında olup olmadığını şu şekilde kontrol edebiliriz. Sayı 50'den büyük ve 100'den küçük ise şartımıza uyuyor. (50 ve 100 dahil olsun)

```
<script>
  if (x >= 50 && x <= 100) {
    // ...
  }
</script>
```

3 sayının 3'ünün de pozitif olmasını istiyorsak;

```
<script>
  if (a>=0 && b>=0 && c>=0 )
</script>
```

13.5.2 Veya (||) Operatörü

Belirtilen şartlardan bir tanesinin doğru olması, koşulun doğru kabul edilmesi için yeterlidir. Şartların tamamı yanlış olduğunda ise koşul sağlanmamış sayılır.

3 sayının en az bir tanesinin pozitif olmasını istiyorsak;

```
<script>
  if (a>=0 || b>=0 || c>=0 )
</script>
```

Sayı 2, 3 veya 4'e bölünebiliyor mu diye bakalım. En az birine bölünmesini istiyoruz.

```
<script>
if ( a%2==0 || a%3==0 || a%4==0 )
</script>
```

13.5.3 Değil (!) Operatörü

İfade true ise false, false ise true değerini döndürür.

```
<script>
n1 = !true // It dönen değeri false
n2 = !false // If dönen değer true
n3 = !"Kedi" // It dönen değer false
</script>
```

Operatörler şimdilik burada bitti. Şimdi ise koşullara geçiyoruz...

14. Koşul Operatörleri

14.1 If-Else Operatörü

Bütün programlama dillerinde olduğu gibi JavaScript dilinde de en çok kullanılan karar ifadelerinden biridir. if parantezi arasındaki koşul doğru ise ifade çalıştırılır. Koşul sağlamıyorsa else kısmı varsa bu kısımdaki ifade çalıştırılır. If-else nin 3 tür kullanımı vardır. Şimdi ise onlara bakalım.

14.1.1 Kullanımı 1:

```
<script>
if (şart) {
    //çalıştırılacak ifade;
}
</script>
```

Örnek: Yaşı 18den büyükse ehliyet alabilirsiniz yazdıran uygulamayı yapın.

```
<script>
  var yas = 20;
  if (yas >= 18) {
    alert("ehliyet alabilirsiniz");
  }
</script>
```

14.1.2 Kullanımı 2:

```
<script>
  if ($art) {
    //şart doğru ise çalıştırılacak ifade;
  }
  else {
    //şart doğru değilse çalıştırılacak ifade;
  }
</script>
```

Örnek:

Yaşı 18den büyükse ehliyet alabilirsiniz, değilse yaşınız ehliyet alamak için uygun değil ve kalan yılı yazdıran uygulamayı yapın.

```
<script>
  var yas = 10;
  if (yas >= 18) {
    alert("ehliyet alabilirsiniz");
  }
  else {
    alert("yaşınız ehliyet almak için uygun değil");
    alert(18 - yas + " kadar yıl sonra ehliyet alabilirsiniz");
  }
</script>
```

14.1.3 Kullanımı 3:

```
<script>
  if (şart1) {
    //şart1 doğru ise çalıştırılacak ifade;
  }
  else if (şart2) {
    //şart2 doğru ise çalıştırılacak ifade;
  }
  else {
    //şart2 de doğru değilse çalıştırılacak ifade;
  }
</script>
```

Örnek:

Girilen değer 100,0 ve negatif aralığındaysa ayrı ayrı uyarı verdiren uygulamayı yapınız.

```
<script>
  var x = 5;
  if (x > 100) {
    alert("x değeri 100den büyüktür.");
  } else if (x > 0) {
    alert("x değeri 0dan büyüktür.");
  } else {
    alert("x negatif bir sayıdır");
  }
</script>
```

14.2 Switch Operatörü

Switch ifadesi farklı koşullara bağlı olarak farklı eylemleri yürütmek için kullanılır. Switch kendine gelen ifadeyi değerlendirip, uygun olan case ile eşleştirir. Uygun olan case ile eşleşen ifade/ifadeler yürütülür.

```
<script>
  switch (ifade) {
    case deger1:
      //ifade, deger1 ile eşleşirse yürütülecek olan kodlar.
      break;
    case deger2:
      //ifade, deger2 ile eşleşirse yürütülecek olan kodlar.
      break;
    //
    //
    case degerN:
      //ifade, degerN ile eşleşirse yürütülecek olan kodlar.
      break;
    default:
      //ifade, hiçbir değer ile eşleşmezse yürütülecek olan kodlar.
      break;
  }
</script>
```

ifade: case değerleri ile eşleştirilecek olan değer

case degerN: ifade ile eşleştirilecek olan kelime/değer

Tanım:

ifade, case değerleri ile eşleştirilecek olan durumu göstermektedir. Eğer ifade **case** değerlerinden biri ile eşleşirse, eşleşen değerden **break;** komutuna kadar olan tüm kodlar çalıştırılır.

Şartlardan sonra **break komutu kullanılmazsa** program aşağı doğru her case kodunu çalıştırır. Break kullanımı unutulmaması gerek önemli bir noktadır.

Örnek:

new Date().getDay() metodu 0-6 arasında değer üretir. 0->pazar 6->cumartesi gününü göstermektedir. (new Date() ve .getDay() metodunu ileride detaylı bir şekilde göreceğiz.)

```
<script>
switch (new Date().getDay()) {
  case 0:
    day = "PAZAR";
    break;
  case 1:
    day = "PAZARTESİ";
    break;
  case 2:
    day = "SALI";
    break;
  case 3:
    day = "ÇARŞAMBA";
    break;
  case 4:
    day = "PERŞEMBE";
    break;
  case 5:
    day = "CUMA";
    break;
  case 6:
    day = "CUMARTESİ";
  }
}</script>
```

15. Döngülere Giriş

NOT: Konuya girmeden önce 🔥Çok Kullanılan Bazı Temel Javascript Fonksiyonları 🔥 Konusuna mutlaka göz atınız burada kullanılacak bazı kodları bilmiyor olabilirsiniz.

Tüm C tabanlı programlama dillerinde olduğu gibi süreklilik arz eden komutlarınızı for döngüsü altında ele alabilirsiniz. JavaScript dilinde de bu yapının kullanımı aynı şekildedir. Bu yazıda **javascript ile for döngüsü kullanımı** hakkında bilgi bulabilir ve bir kaç tane **javascript for döngüsü örneğini** inceleyebilirsiniz.

JavaScript dilinde for döngüsü dışında şu döngüler de bulunmaktadır.

- **for** – Belirli sayıda komutu bloklar arasında yürütmek için kullanılır.
- **for/in** – dizi yada nesnelerin özelliklerini okumak için kullanılır (diğer dillerdeki foreach ile aynı)
- **while** – şart doğru olduğu sürece bloklar arasındaki kodlar yürütülür.
- **do/while** – en az bir kez olmak üzere şart doğru olduğu sürece bloklar arasındaki kodlar yürütülür.

15.1 For Döngüsü

Yazım Kuralı:

```
For (ifade1; ifade2; ifade3) {  
  çalıştırılacak olan kod bloğu  
}
```

ifade1: Döngü başlamadan önce başlangıç değişkeninin tanımlanması (örn: i=0)

ifade2: Döngü çalışma koşulunu göstermektedir. (Örn: i<=100)

ifade3: Başlangıç değişkeninin değişimi (artış/azalış) (Örn: i++)

```
for (var i = 0; i < 100; i++) {  
    //çalıştırılacak olan kodlar  
}
```

Örnek 1: 0-100 arasındaki sayıları yazdırmak için gerekli kodu yazınız.

```
<script>  
for (var i = 0; i <= 100; i++) {  
    document.write("Sayı" + i);  
}  
</script>
```

Örnek 2: Döngü ve şartları bir arada kullanabilirsiniz. 0-100 arasındaki çift sayıları ekrana yazdıran programı yapınız.

```
<script>  
for (var i = 0; i <= 100; i++) {  
    if (i % 2 == 0) {  
        document.write("Sayı" + i);  
    }  
}  
</script>
```

Örnek: 3: Girilen mesajı ve tekrar sayısını kullanıcıdan alıp ekrana yazdıran javascript örneğini yazınız.

```
<script>
var mesaj = prompt("Yazdırılacak Mesajı Girin");
var sayı = prompt("Döngü Sayısı Girin");
for (var i = 0; i < parseInt(sayı); i++) {
    document.write(mesaj + "<br/>");
}
</script>
```

15.2 While Döngüsü

JavaScript while döngüsünde şart doğru olduğu sürece bloklar arasındaki kodlar yürütülür.

Yazım Kuralı:

```
while (şart) {
    çalıştırılacak olan kodlar
}
```

şart: true(doğru) olduğu sürece döngüyü çalıştırır.

şart sağlandığı sürece { }
arası **tekrar tekrar** çalıştırılır.

```
while (şart) {
    ↓
    //ÇALIŞACAK KODLAR
}
```

Örnek: 1:

0'dan 9'a kadar olan sayıları yazdırmak için kullanılan örnek.

Açıklama: Döngüden önce var i=0; ile değişken tanımlanıp ilk değer ataması yapılır. i<10 olduğu sürece döngü çalıştırılacaktır. döngü içinde şartı bozmak için i++ ile i değişkeninde ki değer arttırılmıştır.

```
<script>
var i = 0;
while (i < 10) {
    document.write("Sayı= " + i);
    i++;
}
</script>
```

Örnek: 2:

Döngüdeki en önemli kural; döngü sonsuz döngüye girmemelidir. Bunun için döngüden çıkma komutu(break) yada döngüdeki şartı bozacak değişim yazılmalıdır.

```
<script>
while (true) {
    document.write("merhaba dünya");
    //DİKKAT:program sonsuz döngüye girecek ve sonlanmayacaktır.
}
</script>
```

Örnek 2 düzeltme:

Programı sonsuz döngüden çıkarmak için aşağıdaki gibi bir kod yazılabilir.

```
<script>
var i = 0;
while (true) {
    document.write("merhaba dünya");
    //DİKKAT:program sonsuz döngüye girecek ve sonlanmayacaktır.
    if (i >= 10) break; //i 11 değerine ulaştığında döngüden çıkacaktır.
    i++;
}
</script>
```

15.3 Do-While Döngüsü

Do-While döngüsü while döngüsü ile neredeyse bire bir aynıdır tek farkı istenilen kodun koşul sağlanmasa bile 1 kez çalışmasını sağlar.

```
<script>
var i = 15;
do {
    document.write("Sayı " + i + "<br>");
    i++;
} while (i <= 10);
</script>
```

Yukarıdaki Örnek'te koşul sağlanmıyor olmasına rağmen document.write("Sayı "+i+"
"); ekrana yazılır.

15.4 For in Döngüsü

For.... in döngüsü, bir nesnenin sayılabilir (enumerable) özelliklerinin üzerinde dolaşılmasını sağlar. Her bir özellik için JavaScript, belirlenen ifadeleri çalıştırır. Bir for...in döngüsü aşağıdaki şekilde oluşturulur:

```
<script>
for (değişken in nesne) {
    çalıştırılacak_ifadeler
}
</script>
```

Örnek:

Aşağıdaki fonksiyon, bir nesne ve nesnenin adını parametre olarak alır. Sonrasında nesnenin tüm özellikleri üzerinde gezer ve nesnede bulunan özelliklerin adını ve değerini listeleyen bir string döndürür.

```
<script>
function özellikleriListeOlarakVer(nesne, nesnenin_adi) {
    var sonuç = "";
    for (var i in nesne) {
        sonuç += nesnenin_adi + ":" + i + " = " + nesne[i] + "<br>";
    }
    sonuç += "<hr>";
    return sonuç;
}
</script>
```

Örnek: 2:

```
<script>
var kurslar = { kurs1: "php", kurs2: "Bootstrap", kurs3: "javascript" }
for (nesne in kurslar) {
    document.write(kurslar[nesne] + "<br>");
}
// nesne = değişken ismi isteğe göre değiştirebilirsiniz.
</script>
```

Çıktısı:

```
php
Bootstrap
javascript
```

15.5 İç İçe Döngüler

Bir döngü yapısının içine başka bir döngü yapısının yerleştirilmesiyle elde edilen yapıya iç içe döngü (nested loop) adı verilir.

JavaScript dilinde, if deyimlerini herhangi bir derinliğe kadar iç içe kullanmak nasıl mümkünse, döngü deyimlerini de iç içe kullanmak mümkün olacaktır. İç içe döngülerde hatırlanacak anahtar bilgi, önce en içteki döngülerin tamamlanması gerektiğidir. Bu konuda aşağıdaki kural iç içe döngüler için daima geçerlidir:

İç içe döngülerde en içteki döngü en önce tamamlanır.

15.5.1 Çarpım tablosu uygulaması

Örnek:

```
<script>
document.write("<table border='3'><tr>");
for (var x = 1; x <= 10; x++) {
    document.write("<td>");
    for (var i = 1; i <= 10; i++) {
        document.write(x + "x" + i + "=" + x * i + "<br/>");
    }
    if (x == 5) {
        document.write("</tr><tr>");
    }
    document.write("</td>");
}
document.write("</tr><table>");
</script>
```

Çıktısı:

1x1=1	2x1=2	3x1=3	4x1=4	5x1=5
1x2=2	2x2=4	3x2=6	4x2=8	5x2=10
1x3=3	2x3=6	3x3=9	4x3=12	5x3=15
1x4=4	2x4=8	3x4=12	4x4=16	5x4=20
1x5=5	2x5=10	3x5=15	4x5=20	5x5=25
1x6=6	2x6=12	3x6=18	4x6=24	5x6=30
1x7=7	2x7=14	3x7=21	4x7=28	5x7=35
1x8=8	2x8=16	3x8=24	4x8=32	5x8=40
1x9=9	2x9=18	3x9=27	4x9=36	5x9=45
1x10=10	2x10=20	3x10=30	4x10=40	5x10=50
6x1=6	7x1=7	8x1=8	9x1=9	10x1=10
6x2=12	7x2=14	8x2=16	9x2=18	10x2=20
6x3=18	7x3=21	8x3=24	9x3=27	10x3=30
6x4=24	7x4=28	8x4=32	9x4=36	10x4=40
6x5=30	7x5=35	8x5=40	9x5=45	10x5=50
6x6=36	7x6=42	8x6=48	9x6=54	10x6=60
6x7=42	7x7=49	8x7=56	9x7=63	10x7=70
6x8=48	7x8=56	8x8=64	9x8=72	10x8=80
6x9=54	7x9=63	8x9=72	9x9=81	10x9=90
6x10=60	7x10=70	8x10=80	9x10=90	10x10=100

16. Hata Yönetimi

JavaScript ile hata yakalama konusunu anlatmadan önce hata nasıl oluşur bu kavramı anlamak gerekir. Nesne yönelimli bir programlama yapıyorsanız. Çok fazla satırdan oluşan ve kompleks kodlar yazıyor olmanız oluşabilecek durumlardan biridir. Formlardan veri okurken, ajax ile veri tabanından veri getirirken yada istenilen formatta veriyi oluşturmaya çalışırken hatalar meydana gelebilir. throw komutu ile bu hataları yakalayıp, sayfanın uygun şekilde çalışmasını devam ettirebiliriz.

Örneklere geçmeden önce bazı ifadelerin anlamlarını inceleyelim.

try ifadesi: Hata oluşturabilecek kod blokları yerleştirilir.

catch ifadesi: Hata oluştuğunda hatayı yakalamak için kullanılır.

Örnek:

Hiç tanımlanmamış **test** adında bir fonksiyonu kullanmayı deneyelim.

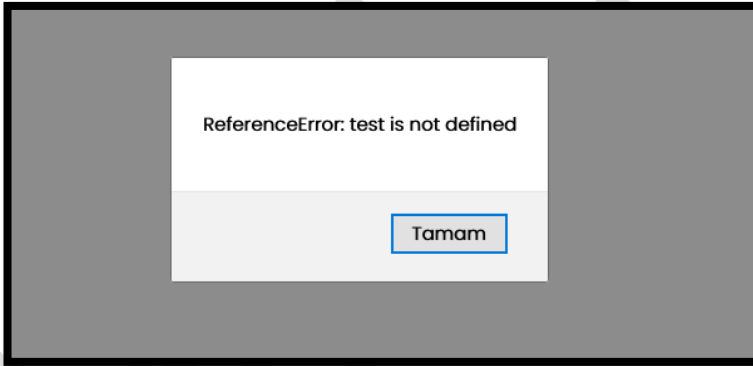
```
<script>
test(); //Uncaught ReferenceError: test is not defined
</script>
```

Uncaught ReferenceError: test is not defined hatası ile karşılaşacağız.

Fonksiyonu try catch bloklarının arasında yazarak fonksiyon tanımlanmadıysa kullanıcıya doğru uyarı vermesini ve programın kaldığı yerde devam etmesini sağlayalım.

```
<script>
  try {
    test();
  }
  catch (hata) {
    alert(hata);
  }
</script>
```

Çıktısı:



try arasında test() fonksiyonunu çalıştırmayı denedi. Fonksiyon tanımlı olmadığı için bir hata fırlattı oluşana hatayı catch ifadesi ile **hata** isminde bir değişkende tutup ekrana hatayı alert ile bilgi olarak verdi.

16.1.1 Throw

Örnek:

throw ile özel hata oluşturma kavramını inceleyelim. TC kimlik numarasını bir form elementi ile kullanıcıdan almayı istiyoruz.

**** Eğer kullanıcı 11 haneden fazla girerse **fazla giriş yaptınız** uyarısı verip form nesnesine odaklanacak

**** Eğer kullanıcı 11' den az girerse **az karakter girişi yaptınız** uyarısı verip formun içine odaklanacak

**** Eğer kullanıcı sayısal karakterler dışında bir değer girerse **hatalı karakter girişi yaptınız** uyarısı verip forma odaklanacak

Yukarıdaki tanımlamalar için throw ifadesi ile aşağıdaki gibi bir script yazıyoruz.

```
<body>
<input type="text" id="tc">
<div id="uyari"></div>
<script>
var nesne = document.getElementById("tc");
var uyari = document.getElementById("uyari");
nesne.onkeypress = function () {
    uyari.textContent = "";
}
nesne.onblur = function () {
    try {
        if (isNaN(nesne.value)) throw "Hatalı karakter girdiniz.";
        if (nesne.value.length < 11) throw "11 karakterden az girdiniz.";
        if (nesne.value.length > 11) throw "11 karakterden fazla girdiniz.";
    }
    catch (e) {
        uyari.textContent = e;
        nesne.focus();
    }
}
</script>
</body>
```

Çıktısı:

15427
11 karakterden az girdiniz.

1428572583577284
11 karakterden fazla girdiniz.

wert
Hatalı karakter girdiniz.

16.1.2 Finally

try catch bloklarından sonra ne olursa olsun çalışması istenilen kodu çalıştırmak için kullanılır.

Örnek:

```
<script>
  var a = 30;
  try {
    document.write(a + b);
  } catch (error) {
    document.write(error);
  } finally {
    document.write("<br>" + a);
  }
</script>
```

Çıktısı:

```
ReferenceError: b is not defined
30
```

b değişkeni 20 sayısı olarak tanımlandığında;

```
50
30
```

17. DİZİLER

Öncelikle **"dizi nedir"** sorusuna cevap vererek başlayalım. Dizi benzer ifadelerin bir araya getirildiği özel değişkendir.

Bir değişken içinde bir veri saklanabilirken, dizilerde birden fazla veri saklamak mümkündür. Aşağıdaki Örnek: kod parçasının ilk satırında diller adında bir dizi oluşturulmuştur. İkinci satırda da idsi Örnek: olan etiket seçilmiş ve dizi içindeki değerler yerleştirilmiştir.

```
<body>
  <div id="ornek">
  </div>
  <script>
    var diller = ["CSS", "HTML", "JavaScript"];
    document.getElementById("ornek").innerHTML = diller;
  </script>
</body>
```

Eğer dizi kullanmasaydık yukarıdaki değerleri şu şekilde yazmak zorunda kalacaktık.

```
<script>
var dil1 = "CSS";
var dil2 = "HTML";
var dil3 = "JavaScript";
</script>
```

17.1 Javascript ile Dizi Nasıl Oluşturulur?

JS dizi oluşturma kuralı şu şekildedir.

```
<script>
var diziadi = ["değer1", "değer2", "değer3", ...];
</script>
```

Örnek: kodda şu şekilde olacaktır.

```
<script>
var donanim = ["Harddisk", "İşlemci", "Ram", "Anakart"];
</script>
```

Ayrıca bilinmesi gereken bir konu da javascript ile veri tipleri dinamik olarak tanımlanır. Bu ifadeden yola çıkarak javascript dizisi içinde kullanılan verilerin tipleri bir birinden farklı olabilir. Şu şekilde dizi tanımlamakta mümkündür.

```
<script>
var örnek = ["Ahmet", false, 12, "Yazı", 78.5];
</script>
```

Not: Sadece bu şekilde değil iç içe dizi ve dizi içinde nesne (diziler de bir nesnedir bu arada) tanımlamak da mümkündür.

17.2 New Anahtar Kelimesi ile Dizi Oluşturma

new anahtar kelimesi ile dizi oluşturmak içinde içinde şu şekilde kod yazmak gerekecektir.

```
<script>
  var isimler = new Array("Ali", "Ahmet", "Yusuf");
</script>
```

Not: Bu şekilde oluşturulan dizi yavaş çalıştığından dolayı önerilmemektedir. “var isimler = [“Ali”, “Ahmet”];” şeklinde dizi oluşturmak daha pratik ve hızlıdır.

17.3 Dizi Nesnelerine Nasıl Ulaşılır?

Dizi içindeki her bir değerinin bir adresi vardır. Dizilerdeki bu adreslere programcı olarak index adını verilmektedir.

İlk değer index(adresi) 0. index olarak ifade edilir ve yine köşeli parantezler içinde index numaraları yazılarak değere ulaşılır.

```
dizi[index];
```

Örnek:

```
<script>
  var diller = ["CSS", "HTML", "JavaScript"];
  document.write(diller[0]); //ekrana CSS yazılacaktır.
  document.write(diller[1]); //ekrana HTML yazılacaktır.
</script>
```

17.4 Dizi içindeki değer nasıl değiştirilir?

Dizi içindeki değeri değiştirmek içinde aynı yöntemle değer değiştirilir.

Örnek:

```
<script>
  var diller = ["CSS", "HTML", "JavaScript"];
  document.write(diller + "<br>");
  diller[2] = "jQuery";
  document.write(diller[2]);
</script>
```

Çıktısı:

CSS,HTML,JavaScript
Jquery

17.5 Çok Boyutlu Diziler

Dizi, kendi içinde başka diziler saklıyor gibi düşünülebilir. Ya da daha kolay anlamak için satır ve sütunlardan oluşan bir tabloya benzetilebilir. Dizinin her satırında birden fazla veri olduğu ve her elemanın bir satır bir de sütun numarası olduğu kabul edildiğinde daha kolay anlaşılacaktır. Gerçekte ise bir dizinin her elemanına başka bir dizi atanmaktadır. Elemanlara ulaşmak içinse önce dıştaki dizinin, sonra da içteki dizinin index numarası belirtilir.

Aşağıdaki Örnekte sınıfım adında bir dizi tanımlanmıştır. Ardından dizinin her elemanına başka bir dizi atanmıştır. Sonra da iç içe for döngüsü kurularak tüm elemanlar ekrana yazdırılmıştır.

```
<script>
var sınıfım = new Array();
sınıfım[0] = new Array(911, "Ahmet", "Yılmaz");
sınıfım[1] = new Array(542, "Özge", "Kara");
sınıfım[2] = new Array(223, "Gizem", "Uysal");
sınıfım[3] = new Array(333, "Aslı", "Bitmez");
var i, j;
for (i = 0; i <= sınıfım.length - 1; i++) {
    for (j = 0; j <= 2; j++) {
        document.write(sınıfım[i][j] + " ");
    }
    document.write("<br />");
}
</script>
```

Burada da görüldüğü gibi dizi elemanlarına ulaşabilmek için hem dıştaki hem de içteki dizinin index numarası verilmiştir.

Örneğin yukarıdaki Örnekte sınıfım[1][2] ifadesi kullanıldığında "Kara" değerini verecektir.

Dizileride burada bitirmiş olduk şimdi ise fonksiyonlara geçiyoruz...

18. Fonksiyonlar (Functions)

18.1 Fonksiyonlar nedir?

Fonksiyon, belirli sayıda verileri kullanarak bunları işleyen ve bir sonuç üreten komut grubudur. Her **fonksiyonun** bir adı ve **fonksiyona** gelen değerleri gösteren parametreleri (bağımsız değişkenleri) vardır. Her **fonksiyonda** parametre olması zorunlu değildir. **Parametresiz fonksiyonlar**, adından da anlaşılacağı gibi herhangi bir parametreye gerek duymadan ve **fonksiyonda** tanımlanan kodların yerine getirilmesi ile çalışan fonksiyonlardır. Aşağıda ki şekil fonksiyonların çalışma prensibini gösteriyor.

18.2 Fonksiyonların genel özellikleri

1. Bir amaca hizmet eden program parçacıklarıdır.
2. Fonksiyon çalıştırıldığında, bir işlemi yerine getirmesi yada bir değer döndürmesi istenecektir.
3. Çağrılmış olan bir fonksiyon ya geriye bir sonuç değeri döndürür ve/veya fonksiyon içerisindeki operasyonlarını işlemlerini tamamlayıp çağrılan yere geri döner.
4. Diğer programlama dillerinde metod,prosedür (yordam) v.b isimlendirme ile adlandırılırlar.
5. Programcılar, programların yapacağı işi fonksiyonlar vasıtasıyla anlamlı parçalara bölerek programlarının modülerlik kazanmalarını sağlar.

18.3 Parametrelili Fonksiyon

javascript dilinde hazırlanan parametrelili bir fonksiyonun genel yapısı şöyledir:

```
<script>
function fonksiyon_Adi(parametre1,parametre2,...parametreN) {
    İŞLEMLER
}
</script>
```

Örnek: bir fonksiyon yazalım.

```
<script>
function merhaba(ad,soyad) {
    document.write("Hoşgeldin" + ad + " + soyad");
}
var ad = prompt("Adınızı Giriniz..");
var soyad = prompt("Soyadınızı Giriniz..");
merhaba(ad,soyad);
</script>
```

Çıktısı:

Adınıza Giriniz...

Soyadınıza Giriniz...

☐ Bu sayfanın ek iletişim kutuları oluşturulmasını önle

Hoşgeldin Musa Yazık

18.4 Parametresiz Fonksiyon

javascript dilinde hazırlanan parametresiz bir fonksiyonun genel yapısı şöyledir:

```
<script>
  function fonksiyon_Adi() {
    İŞLEMLER
  }
</script>
```

Örnek:

```
<script>
function topla() {
  a = 20;
  b = 10;
  return a + b;
}
document.write(topla());
</script>
```

Çıktısı:

30

19. JAVASCRIPTTE NESNELER

JavaScript programlama dilinde 5 tane veri tipi ve 3 tane referansı ifade eden Object tipi vardır. Veri tipleri **Number**, **String**, **Boolean**, **Undefined**, ve **Null**(bu da bir

obje)dir. **fonksiyon, dizi, nesne (object)** gibi refrensları ifade eden Object tipler bulunmaktadır.

Javascript ile çeşitli yöntemler kullanarak nesne oluşturmak mümkündür. Bu yöntemlerde bir kaç tanesini Örnekler ile açıklanmıştır.

1-) JavaScriptte nesne oluşturmak için {} sembolleri kullanılır.

```
var Personel = {};
```

Yukarıda Personel adında bir nesne oluşturduk.

Oluşturulan nesneye bir kaç tane özellik ataması yapalım. Personelin adı,soyad, telefon numarası ve günlük ücreti olsun. Bunları nesneye eklemek için **özellik:değeri** şeklinde kullanacağız.

Eğer birden fazla özellik yada metod tanımlaması yapılacaksa her bir değer arasında virgül (,) sembolü konulmalıdır.

Not: Son özelliğin sonuna virgül konulmaz.

```
<script>
var Personel = {
  ad: "Musa",
  soyad: "Yazlık",
  telefon: "715 400 40 40",
  ucret: 100
};
</script>
```

Metod tanımlaması yapmak içinse yine özellik : değeri gibi kullanılarak, özellik : fonksiyon şeklinde tanımlanmaktadır. Burada JS ile anonim fonksiyon tanımlaması yapılmaktadır.

Eğer Nesne tanımlaması yapılırken özelliklerden biri metod içinde kullanılacaksa (Örnek:te ödeme hesaplamak için ücreti kullanacağız.) nesne içindeki özellik için **this** anahtar kelimesi kullanılır.

```
<script>
var Personel = {
  ad: "Musa",
  soyad: "Yazlık",
  telefon: "715 400 40 40",
  ucret: 100,
  ucretOde: function (gun) {
    return gun * ucret;
  }
};
</script>
```

Javascriptte nesne oluşturmaya bir örnek ile pekiştirelim....

Örnek:

```
<script>
var Ogrenci = {
  ad: "Musa",
  soyad: "Yazlık",
  adSoyad: function () {
    return this.ad + " " + this.soyad;
  }
}
document.write(Ogrenci.adSoyad());
</script>
```

this anahtarı nesne içindeki parçalara(ozellik,metod) ulaşmak için kullanılır.

Çıktısı:

Musa Yazlık

2-) Başka bir JavaScript nesnesi oluşturma yöntemi de constructor pattern(yapıcı desen) olarak tanımlamak mümkündür. Fonksiyon tanımlama yöntemi ile sınıf oluşturup, oluşturulan sınıf **new** anahtarı ile nesne olarak türetilir.

Örnek:

```
<script>
var Uye = function () {
  this.isim;
  this.soyisim;
  this.adSoyad = function () {
    return this.isim + " " + this.soyisim;
  }
}
var Uye1 = new Uye();
var Uye2 = new Uye();
Uye1.isim = "Annamarie";
Uye1.soyisim = "Schiller";
Uye2.isim = "Elisabeth";
Uye2.soyisim = "Bradtke";
document.write(Uye1.adSoyad());
document.write("<br>");
document.write(Uye2.adSoyad());
</script>
```

Annamarie Schiller
Elisabeth Bradtke

Nesnelerde burada bitiyor şimdi ise javascriptin en önemli kısmı olan DOM olaylarına geçiş yapıyoruz...

20. Dom(Document Object Model) Olayları

DOM, Document Object Model'in kısaltılmışı olup, dilleri barındıran bir standarttır. **DOM**, bir dil değildir. JS, PHP, Java, ASP gibi bir çok programlama dilini destekler. **HTML** ile programlama dilleri arasında bir standart oluşturup, programlama dillerinin **HTML** den bilgi alışı-verişinde bulunmasına yardımcı olur. **Document Object Model**, Nesneler ve özelliklerden oluşur.

HTML ile hazırladığımız sayfaya **Belge**, bu **belgenin** içindeki her bir **öğeye** ise **Nesne** denir. **Document Object Model**, belgedeki nesnelerin birbiri ile hiyerarşik bir yapı oluşturmaları ile oluşur.

HTML içerisine yerleştirilmiş **nesnelere** ulaşabilmek için, içinde bulunduğu **nesneyi** çağırabilir veya bir id vererek doğrudan id ile çağırabilirsiniz. **DOM**'un erişim yöntemlerine ne kadar iyi biliyorsak, **HTML**'e o kadar iyi hükmedebiliriz.

UYARI: Aşağıdak göreceğiniz nesnelerin başına document. koyduktan sonra kullanınız sonuçtan aşağıda verilecek nesneler document nesnesi. Mesela **getElementbyid()** olarak değilde **document.getElementById()** olarak kullanınız.

20.1 getElementbyid()

document.getElementById metodu, **id** özelliği belirtilen elementin referansının alınmasını sağlar. Aynı **id** kimliğine sahip başka bir element olması durumunda sadece ilk elementi verir. *Diğerlerine erişim sağlamaz.*

element = document.getElementById(kimlik)

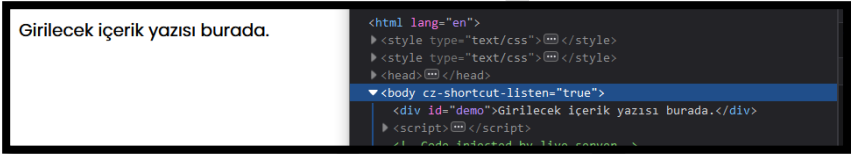
kimlik = Doküman üzerinde elementi bulmak için kullanılacak essiz id özelliğini ifade eder.

element = **element** nesne referansı. Doküman üzerinde belirtilen **id** bulunamazsa **null** döner.

Örnek:

```
<div id="demo"></div>
<script>
  var girilecek_icerik = "Girilecek içerik yazısı burada.";
  document.getElementById("demo").innerHTML = girilecek_icerik;
</script>
```

Çıktısı:



Ek:

```
console.log(document.getElementById("demo"));
```

Asıl seçme işlemi bu şekildedir Ama bu seçmenin yanına alt özellik ekleyerek daha doğru bir seçim yapmış oluyoruz. Yukarıdaki console.log(document.getElementById("demo")); ile elementi tamamlamış oluyoruz

Çıktısı:



20.2 getElementsByName()

Kendisine parametre olarak verilen değer doğrultusunda, belge içerisinde belirtilen name özelliği /niteliğine sahip tüm elemanların referansını elde etmek için kullanılır. Eleman referansları birden fazla ise her bir eleman döngü ile elde edilebilir.

Örnek:

```
Kedi: <input name="animal" type="checkbox" value="Kedi">
Köpek: <input name="animal" type="checkbox" value="Köpek">

<button onclick="myfunction()">Try it</button>

<script>
function myfunction() {
  var x = document.getElementsByName("animal");
  var l;
  for (i = 0; i < x.length; i++) {
    if (x[i].type == "checkbox") {
      x[i].checked = true;
    }
  }
}
</script>
```

Çıktısı:

Kedi: ☒ Köpek: ☒

20.3 getElementByTagName()

document.getElementsByTagName metodu, doküman üzerinde belirtilen etiket ismine ait bütün elementlerin bir listesini verir.

Örnek:

```
<p>Lorem, ipsum dolor.</p>
<p>Lorem, ipsum dolor.</p>
<p>Lorem, ipsum dolor.</p>
<script>
  console.log(document.getElementsByTagName("p"));
</script>
```

Çıktısı:

Lorem, ipsum dolor.
Lorem, ipsum dolor.
Lorem, ipsum dolor.

```
HTMLCollection
  0: <p>
  1: <p>
  2: <p>
  length: 3
  __proto__: HTMLCollectionPrototype { item: item(), namedItem: namedItem(), length: Getter, ... }
```

20.4 getElementByClassName()

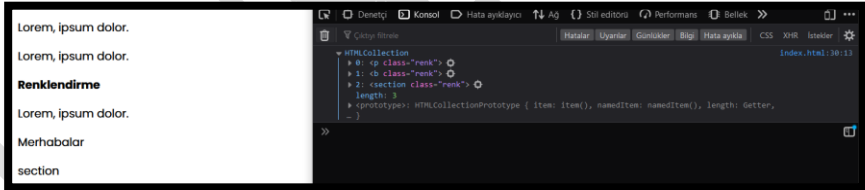
getElementById ile aynı işlevi görür. Tek farkı getElementById istenilen id ye sahip elemanı seçer , getElementByClassName istenilen class a sahip elemanları seçer. Ona göre özellik verilir.

Örnek:

```
<div id="mer">
  <p class="renk">Lorem, ipsum dolor.</p>
  <p>Lorem, ipsum dolor.</p>
  <b class="renk">Renklendirme</b>
  <p>Lorem, ipsum dolor.</p>
</div>

<p>Merhabalar</p>
<section class="renk">
  section
</section>
<script>
  console.log(document.getElementsByClassName("renk"));
</script>
```

Çıktısı:



```
HTMLCollection
  0: <p class="renk">
  1: <b class="renk">
  2: <section class="renk">
  length: 3
  __proto__: HTMLCollectionPrototype ( item: item(), namedItem: namedItem(), length: Getter, ...)
```

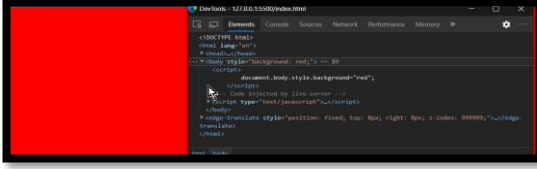
20.5 document.body

Belge içerisindeki body tag'ına özellik vermek amaçlı kullanılır.

Örnek: Arka planı kırmızı yapalım.

```
<script>
  document.body.style.background="red";
</script>
```

Çıktısı:



20.6 document.head

document.body ile aynı işlevi head tag ı için yapar. Belge içerisindeki head tag'ına özellik vermek amaçlı kullanılır.

20.7 characterSet

Belgenin karakter kodlamasının referansını yani içeriğini elde etmek için kullanılır. meta etiketindeki charset özelliğini geriye gönderir.

Örnek:

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>

<body>
  <script>
    document.write(document.characterSet);
  </script>
</body>
```

Çıktısı:

UTF-8

20.8 title

Belge içerisindeki title etiketine özellik vermek ve içeriğini geriye göndürmek amaçlı kullanılır.

Örnek:

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>

<body>
  <script>
    document.write(document.title);
  </script>
</body>
```

Çıktısı:

Document

20.9 documentURI

Belge sayfasının URL adresini almak amaçlı kullanılır.

Örnek:

```
<body>
  <script>
    document.write(document.documentURI);
  </script>
</body>
```

Çıktısı:

http://127.0.0.1:5500/index.html

20.10 location

Belge sayfasının URL adresini almak üzerinde değişiklik yapmak amaçlı kullanılır. documentURI ile neredeyse aynı işlevi görür.

Örnek:

```
<body>
  <script>
    document.write(document.location.href = "https://www.google.com.tr");
  </script>
</body>
```

Çıktısı:

Sayfanın url si çok hızlı değiştiği için çıktısını alamadım. Ama işin özü sayfamızın url si <https://www.google.com.tr> olarak değişti ve google sayfası açıldı.

20.11 domain

Belgenin çalıştığı domain adresini verir.

Örnek:

```
<body>
  <script>
    document.write(document.domain);
  </script>
</body>
```

Çıktısı:

Biz şu anda localde çalıştığımız için verdiği domain 127.0.0.1 oldu siz eğer bir sunucuda çalışırsanız size vereceği adres domain adresiniz olacaktır.

127.0.0.1

20.12 lastModified

Belgenin enson güncelleme tarihinin ve saatinin referansını elde etmek amaçlı kullanılır.

Örnek:

```
<body>
  <script>
    document.write(document.lastModified);
  </script>
</body>
```

Çıktısı:

06/01/2020 12:41:03

20.13 hasFocus()

Belge odağının olup olmadığını kontrol eder ve sonucu true veya false olarak geri döndürür.

Örnek:

```
<body>
  <script>
    var focus = document.hasFocus()
    document.write(focus)
  </script>
</body>
```

Çıktısı:

false

20.14 activeElement()

Belge içerisindeki seçili veya odaklanılmış elemanın referansını elde etmek için kullanılır.

Örnek:

```
<body onclick="sec()">
  <p>Merhaba Dünya</p>
  <input type="text" value="deneme yazı">
  <button>tıkla</button>
  <p id="demo"></p>
  <script>
    function sec() {
      var x = document.activeElement.tagName;
      document.getElementById("demo").innerHTML = x;
    }
  </script>
</body>
```

Çıktısı:

Merhaba Dünya

deneme yazı

BUTTON

Merhaba Dünya

deneme yazı

INPUT

20.15 scripts

Belge içerisindeki script etiketlerine tüm elemanların referansını elde etmek için kullanılır. Eleman referansları birden fazla ise her bir eleman döngü ile elde edilebilir.

Örnek:

```
<script>
  document.write(document.scripts[0].innerHTML);
</script>
```

Çıktısı:

```
document.write(document.scripts[0].innerHTML);
```

20.16 dir

Belge içerisindeki metinlerin yazılış yönünü değiştirmek veya elde etmek için kullanılır.

Örnek:

```
<body>
  <p dir="ltr" id="deneme">Merhaba Dünya</p>
  <div></div>
  <script>
    var dir = document.getElementById("deneme").dir;
    document.getElementsByTagName("div")[0].innerHTML = dir;
  </script>
</body>
```

Çıktısı:

Merhaba Dünya

ltr

20.17 classList

Belge içerisine bulunan her hangi bir html etiketinde bulunan class özelliğinin içeriğini elde etmek için ve ayarlamak için kullanılır. Bu metodun içerisinde bazı alt özelliklerde vardır.

item() → Girilen değere göre class ismini verir.

length → Kaç tane class özelliği olduğunu geriye döndürür.

add() → İstenilen class özelliğini ekler.

remove() → İstenilen class özelliğini siler.

toggle() → İstenilen class özelliği varsa kaldırır yoksa ekler.

contains → İstenilen class özelliğini arama işlemi yapar.

Örnek:

```
<body>
<div id="merhaba" dir="ltr" class="sifir bir iki uc dort bes"> Merhaba</div><br>
<script>
var sonuc1 = document.querySelector("div").classList.item(2); // iki değerini geriye döndürür.
var sonuc2 = document.querySelector("div").classList.length; // 6 sonucunu verecek. Çünkü 6 class özelliği var.
document.querySelector("div").classList.add("deneme"); // deneme class özelliğini ekler.
document.querySelector("div").classList.remove("uc"); // uc class özelliğini siler.
document.querySelector("div").classList.toggle("mer"); // mer class özelliği yok ekleme yapacak.
document.querySelector("div").classList.toggle("bes"); // bes class özelliği var silme yapacak.
var sonuc3 = document.querySelector("div").classList.contains("bir"); // true sonucunu döndürür.

document.write(sonuc1 + "<br>");
document.write(sonuc2 + "<br>");
document.write(sonuc3 + "<br>");

</script>
</body>
```

Çıktısı:

```
iki
6
true
```

```
<body>
  <div id="merhaba" dir="ltr" class="sifir bir iki dort deneme
mer"> Merhaba</div>
```

20.18 tagName

Belirtilen belgede bulunan html etiketinin ismini verir.

Örnek:

```
<body>
<div id="merhaba" dir="ltr" class="sifir bir iki uc dort bes"> Merhaba</div>
<script>
  var sonuc = document.querySelector("#merhaba").tagName;
  document.write(sonuc);
</script>
</body>
```

Çıktısı: merhaba id sine sahip html etiketinin ismi div miş 😊

Merhaba
DIV

20.19 childElementCount

Belge içerisindeki her hangi bir Html elemanının içerdiği alt elemanların sayısını elde etmek için kullanılır.

Örnek:

```
<body>
  <ul>
    <li>merhaba 1</li>
    <li>merhaba 2</li>
    <li>merhaba 3</li>
    <li>merhaba 4</li>
    <li>merhaba 5</li>
  </ul>
  <script>
    var sonuc = document.querySelector("ul").childElementCount;
    document.write(sonuc);
  </script>
</body>
```

Çıktısı: ul etiketinin içerisinde 5 tane li etiketi var.

- merhaba 1
- merhaba 2
- merhaba 3
- merhaba 4
- merhaba 5

5

20.20 hasChildNodes

Belirtilen herhangi bir html etiketinin alt bir düğümü(node) olup olmadığını kontrol eder. Sonucu true veya false olarak geri döndürür.

Örnek:

```
<body>
  <div id="bir"></div>
  <div id="iki">Musa Yazlık</div>
  <div id="uc"><span></span></div>
  <span id="dort"></span>
  <script>
    var sonuc1 = document.querySelector("#bir").hasChildNodes();
    var sonuc2 = document.querySelector("#iki").hasChildNodes();
    var sonuc3 = document.querySelector("#uc").hasChildNodes();
    var sonuc4 = document.querySelector("#dort").hasChildNodes();

    document.write(sonuc1 + "<br>");
    document.write(sonuc2 + "<br>");
    document.write(sonuc3 + "<br>");
    document.write(sonuc4);
  </script>
</body>
```

Çıktısı:

```
Musa Yazlık
false
true
true
false
```

20.21 children

Belirtilen html'e etiketinin alt çocuklarına erişmek için kullanılır. Erişilen alt çocuğun istenilen özelliği ayarlanabilir.

Örnek:

```
<body>
  <ul>
    <li>Merhaba 1</li>
    <li>Merhaba 2</li>
    <li>Merhaba 3</li>
    <li>Merhaba 4</li>
    <li>Merhaba 5</li>
    <li>Merhaba 6</li>
  </ul>
  <script>
    document.write(document.querySelector("ul").children.length + "<br>") // ul etiketinde kaç tane çocuk etiket var onu öğrendim.
    document.write(document.querySelector("ul").children[2].innerHTML) // ikinci elemanın içeriğini aldım. İndis sıfırdan başlar bu yüzden merhaba 3 yazısı geriye döndü.
    document.querySelector("ul").children[3].style.color = "red"; // 3 üncü elemanın yazı rengini kırmızı yaptık.
  </script>
</body>
```

Çıktısı:

- Merhaba 1
- Merhaba 2
- Merhaba 3
- **Merhaba 4**
- Merhaba 5
- Merhaba 6

6

Merhaba 3

20.22 firstElementChild

Belge içerisinde herhangi bir html etiketinin içerdiği ilk çocuğa erişmek için kullanılır.

Örnek:

```
<body>
<ul>
  <li>Merhaba 1</li>
  <li>Merhaba 2</li>
  <li>Merhaba 3</li>
  <li>Merhaba 4</li>
  <li>Merhaba 5</li>
  <li>Merhaba 6</li>
</ul>
<script>
  document.write(document.querySelector("ul").firstElementChild + "<br>") // ilk elemana eriştik ama bu bir nesne alt
  metotlar ile nesnenin özelliklerine erişelim.
  document.write(document.querySelector("ul").firstElementChild.innerHTML + "<br>") //innerHTML ile nesnenin içeriğine
  eriştik.
  document.write(document.querySelector("ul").firstElementChild.tagName + "<br>") // tagName ile Html etiketinin ismini
  öğrendik.
</script>
</body>
```

Çıktısı:

- Merhaba 1
- Merhaba 2
- Merhaba 3
- Merhaba 4
- Merhaba 5
- Merhaba 6

[object HTMLLIElement]

Merhaba 1

LI

20.23 firstChild

Belge içerisinde bulunan her hangi bir html etiketinin ilk alt düğüm(node) çocuğuna erişmek için kullanılır.

Örnek:

```
<body>
  <ul>
    <li>Merhaba 1</li>
    <li>Merhaba 2</li>
    <li>Merhaba 3</li>
  </ul>
</body>

<script>
  document.write(document.querySelector("ul").firstChild + "<br>") // Burada eleman yerine düğüm sayarak işlem yapıldığı
  için ilk düğüm ise bir boşluk karakteridir, oda text elemanı olarak geri döndürülür. İlk html elemanına erişmek
  istiyorsanız firstElementChild özelliğini kullanınız.
</script>
```

Çıktısı:

- Merhaba 1
- Merhaba 2
- Merhaba 3

[object Text]

20.24 lastElementChild

Belge içerisinde herhangi bir html etiketinin içerdiği son çocuğa(elemana) erişmek için kullanılır. firstElementChild ile kullanımı aynı olduğu için Örnek: verilmemiştir.

20.25 lastChild

Belge içerisinde bulunan her hangi bir html etiketinin son alt düğüm(node) çocuğuna erişmek için kullanılır. firstChild ile kullanımı aynı olduğu için Örnek: verilmemiştir.

20.26 nextElementSibling

Belge içerisindeki herhangi bir html elemanının içerisindeki aynı seviyede yani kardeş olan elemanlardan, belirtilecek olan elemandan sonra gelen ilk elemanı elde etmek için kullanılır.

Örnek:

```
<body>
  <div id="elemanlar">
    <p>Lorem, ipsum dolor.</p>
    <div class="isim">Musa Yazlık</div>
    <b>Renkler</b>
    <span>Denemeler</span>
    <br><br><br>
  </div>
  <script>
    document.write(document.querySelector(".isim").nextElementSibling.innerHTML)
  </script>
</body>
```

Çıktısı:

Lorem, ipsum dolor.

Musa Yazlık

Renkler Denemeler

Renkler

20.27 nextSibling

Belge içerisindeki herhangi bir html elemanının içerisindeki aynı seviyede yani kardeş olan düğümden(node), belirtilecek olan düğümden sonra gelen ilk düğümü elde etmek için kullanılır.

Örnek:

```
<body>
  <div id="elemanlar">
    <p>Lorem, ipsum dolor.</p>
    <div class="isim">Musa Yazlık</div><br>
    <b>Renkler</b>
    <span>Denemeler</span>
    <br><br><br>
  </div>
  <script>
    document.write(document.querySelector(".isim").nextSibling)
  </script>
</body>
```

Çıktısı:

Lorem, ipsum dolor.

Musa Yazlık

Renkler Denemeler

[object HTMLBRElement]

20.28 previousElementSibling

Belge içerisindeki herhangi bir html elemanının içerisindeki aynı seviyede yani kardeş olan elemanlardan, belirtilecek olan elemandan önce gelen ilk elemanı elde etmek için kullanılır. nextElementSibling metodu ile kullanımı aynı olduğundan Örnek: verilmemiştir.

20.29 previousSibling

Belge içerisindeki herhangi bir html elemanının içerisindeki aynı seviyede yani kardeş olan düğümden(node), belirtilecek olan düğümden önce gelen ilk düğümü elde etmek için kullanılır. nextSibling metodu ile kullanımı aynı olduğundan Örnek: verilmemiştir.

20.30 offsetParent

Belge içerisindeki herhangi bir html elemanının içerisinde mutlak olarak konumlandırıldığı ilk elemanın referans nesnesini elde etmek için kullanılır.

Örnek:

```
<body>
  <div id="mutlak">Lorem, ipsum dolor.</div>
  <script>
    document.write(document.getElementById("mutlak").offsetParent);
  </script>
</body>
```

Çıktısı:

Lorem, ipsum dolor.
[object HTMLBodyElement]

20.31 contains()

Belge içerisindeki her hangi bir düğümün başka bir düğümün alt düğümü olup olmadığını kontrol eder. true veya false olarak geri bildirim yapar.

Örnek:

```
<body>
  <div id="mutlak">
    <p>Deneme Yazısı</p>Lorem, ipsum dolor.
  </div>
  <div class="mer">Lorem ipsum dolor sit.</div>
  <script>
    var alt = document.getElementsByTagName("p")[0];
    var alt2 = document.getElementById("mer");
    document.write(document.getElementById("mutlak").contains(alt) + "<br>");
    document.write(document.getElementById("mutlak").contains(alt2) + "<br>");
  </script>
</body>
```

Çıktısı:

Deneme Yazısı

Lorem, ipsum dolor.
Lorem ipsum dolor sit.
true
false

20.32 textContent

Belge içerisindeki herhangi bir node un text nodelarının değerini elde etmek veya ayarlamak için kullanılır.

Örnek:

```
<body>
<ul>
  <li>merhaba 1</li>
  <li>merhaba 2</li>
  <li>merhaba 3</li>
</ul>
<button>Tıklama butonu</button>
<br><br><br>
<script>
  document.write(document.querySelector("ul").textContent);
  document.write(document.querySelector("button").textContent);
</script>
</body>
```

Çıktısı:

- merhaba 1
- merhaba 2
- merhaba 3

Tıklama butonu

merhaba 1 merhaba 2 merhaba 3 Tıklama butonu

20.33 compareDocumentPosition()

Belge içerisinde bulunan node'ların birbirine göre konum değerini elde etmek için kullanılır.

2 → Belirtilen ilk düğüm belirtilen ikinci düğümden sonra

4 → Belirtilen ilk düğüm, belirtilen ikinci düğümden önce

Örnek:

```
<body>
  <div id="html">Html</div>
  <div id="css">Css</div>
  <div id="javascript">Javascript</div>
  <div id="php">Php</div>

  <br>
  <br>
  <script>
    document.write(document.querySelector("#css").compareDocumentPosition(document.querySelector("#javascript")) + "<br>")
    document.write(document.querySelector("#javascript").compareDocumentPosition(document.querySelector("#html")))
  </script>
</body>
```

Çıktısı:

Html
Css
Javascript
Php

4
2

20.34 parentElement

Belge içerisindeki her hangi bir html elemanının bir üst elemanını ele etmek için kullanılır.

Örnek:

```
<body>
  <ul>
    <li>Merhaba 1</li>
    <li>Merhaba 2</li>
    <li>Merhaba 3</li>
    <li>Merhaba 4</li>
    <li>Merhaba 5</li>
  </ul>

  <br>
  <br>
  <script>
    document.querySelector("li").parentElement.style.background = "red";
  </script>
</body>
```

Çıktısı:

- Merhaba 1
- Merhaba 2
- Merhaba 3
- Merhaba 4
- Merhaba 5

20.35 parentNode

Belge içerisindeki herhangi bir html node'nın bir üst nodesini ele etmek için kullanılır.

Uyarı: Text nodeları ele alınmaz. Boşluklar, Yorum satırlar vs. Sadece değerli Node ler ele alınır.

Örnek:

```
<body>
<!-- Deneme -->
<div id="deneme">Merhaba Dünya</div>
<br>

<script>
  // body etiketi seçilir.
  document.querySelector("#deneme").parentNode.style.background = "red";
</script>
</body>
```

Çıktısı:

Merhaba Dünya

20.36 appendChild

Bir node'u belge içerisinde belirtilecek olan herhangi bir elemanın içerisine en sonda olmak üzere eklemek için kullanılır.

20.37 revomeChild

Belge içerisindeki herhangi bir html elemanının içerdiği node'u silmek için kullanılır.

Örnek: appendChild ve revomeChild tek örnek.

```
<body>
  <ul id="bir">
    <li>Html</li>
    <li id="css">Css</li>
    <li>JavaScript</li>
  </ul>

  <input type="text">
  <button id="ekle">Ekle</button>
  <button id="sil">Sil</button>
  <script>
    document.querySelector("#ekle").addEventListener("click", function () {
      var li = document.createElement("li");
      li.innerHTML = document.querySelector("input").value;
      document.getElementById("bir").appendChild(li);
    })
    document.querySelector("#sil").addEventListener("click", function () {
      document.getElementById("bir").removeChild(document.querySelector("ul").lastElementChild);
    })
  </script>
</body>
```

Çıktısı: Ekleme / Silme

- Html
- Css
- JavaScript
- Php

Ekle

Sil

- Css
- JavaScript
- Php

Ekle

Sil

20.38 replaceChild()

Belge içerisindeki herhangi bir düğümü değiştirmek için kullanılır.

Örnek:

```
<body>
  <ul id="liste">
    <li>Html</li>
    <li>Css</li>
    <li>JavaScript</li>
  </ul>
  <button onclick="degistir()" id="degistir">Değiştir</button>
  <script>
    function degistir() {
      var yenisugum = document.createTextNode("XLM");
      var deger = document.getElementById("liste").childNodes[3];
      deger.replaceChild(yenisugum, deger.childNodes[0]);
    }
  </script>
</body>
```

Çıktısı:

- Html
- XLM
- JavaScript

Değiştir

20.39 cloneNode()

Herhangi bir html elemanının içerdiği node'u kopyalamak için kullanılır.

Örnek:

```
<body>
  <ul>
    <li>Html</li>
    <li>Css</li>
    <li>JavaScript</li>
  </ul>
  <button>Clon üret</button>
  <div class="clonyaz"></div>
  <script>
    document.querySelector("button").addEventListener("click", function () {
      var eleman = document.querySelector("ul");
      var clon = eleman.cloneNode(true);
      document.querySelector(".clonyaz").appendChild(clon);
    });
  </script>
</body>
```

Çıktısı: Her buttona tıklandığında ul nodu nun bir klonunu buttonun altına oluşturacak. 2 kez tıklandı.



20.40 insertBefore()

Bir nodu belge içerisindeki belirtilecek olan her hangi bir node'un önüne eklemek için kullanılır.

Örnek:

```
<body>
  <ul>
    <li>Html</li>
    <li>Css</li>
    <li>Javascript</li>
  </ul>
  <input type="text">
  <button>Ekle</button>
  <div class="Ekle"></div>
  <script>
    document.querySelector("button").addEventListener("click", function () {
      var input = document.querySelector("input").value;
      var li = document.createElement("li");
      var text = document.createTextNode(input);
      li.appendChild(text);
      var yer = document.querySelector("ul");
      var ekleneceYer = yer.lastChild;
      yer.insertBefore(li, ekleneceYer);
    });
  </script>
</body>
```

Çıktısı: Input içerisine girilen değeri ul etiketinin içinde bulunan li etiketlerinin sonuna yeni bir eleman olarak ekleyecek.

- Html
- Css
- Javascript
- Php
- Mysql

20.41 Attribute Özellikleri

20.41.1 setAttribute()

Belirtilen öge üzerinde bir öznitelik değerini ayarlar.Özellik zaten varsa , değer güncellenir; aksi halde, belirtilen ad ve değerle yeni bir nitelik eklenir.

element.setAttribute(isim, değer);

isim → Gereklidir. Eklemek istediğiniz özneniteliğın adı.

değer → Gereklidir . Eklemek istediğınız öznitelik değeriştir.

Örnek:

```
<body>
<button>javascript</button>
<script>
  var b = document.querySelector("button");
  b.setAttribute("name", "musa");
  b.setAttribute("disabled", "");
</script>
</body>
```

Çıktısı:

```
<html lang="en">
  <head> </head>
  <body cz-shortcut-listen="true">
    <button name="musa" disabled="">javascript</button>
    <script> </script>
```

20.41.2 creatAttribute()

Belge içerisinde istenilen bir elemana istenilen bir özelliği oluşturmak amaçlı kullanılır. Oluşturulan özelliği istenilen elemana eklemek için de setAttributeNode metodu kullanılır. Kısaca creatAttribute özellik(nitelik) oluşturmak için kullanılır.

Örnek:

```
<style>
  .color {
    width: 500px;
    height: 500px;
    background: red;
  }
</style>

<body>
  <div id="deneme"></div>
  <script>
    var ozellik = document.getElementById("deneme");
    var ozellikEkle = document.createAttribute("class");
    ozellikEkle.value = "color";
    ozellik.setAttributeNode(ozellikEkle);
  </script>
</body>
```

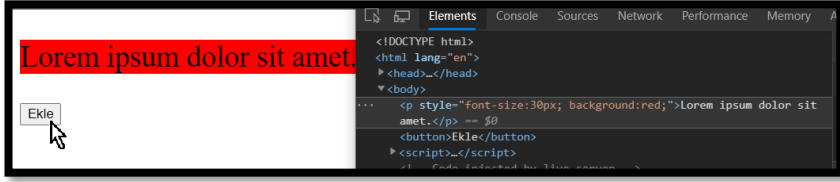
20.41.3 setAttributeNode()

Belge içerisindeki herhangi bir html elemanına createAttribute() metodu ile oluşturulan özellik node'unu ve değerini, node object'ine uygulamak için kullanılır.

Örnek:

```
<body>
  <p>Lorem ipsum dolor sit amet.</p>
  <button>Ekle</button>
  <script>
    document.querySelector("button").addEventListener("click", function () {
      var text = document.querySelector("p");
      var eklenecekozellik = document.createAttribute("style");
      eklenecekozellik.value = "font-size:30px; background:red;";
      text.setAttributeNode(eklenecekozellik);
    })
  </script>
</body>
```

Çıktısı:



Not: setAttribute metodu kullanılması önerilir.

20.41.4 getAttribute

Öğede belirtilen bir öznitelik değerini döndürür. Belirtilen öznitelik değeri yok ise döndürülen değer null veya boş dizi olacaktır.

element.getAttribute(isim);

isim → Gereklidir. Değer almak istediğiniz özniteliğin adı.

Örnek:

```
<body>
  <div id="test" align="center">JavaScript</div>
  <script>
    var test = document.querySelector("#test");
    var align = test.getAttribute("align"); //Çıktı:center
  </script>
</body>
```

20.41.5 getAttributeNode

Belge içerisindeki herhangi bir html elemanının belirtilen özellik node'unun değerini elde etmek veya ayarlamak için kullanılır.

Örnek:

```
<body>
  <a id="test" href="https://www.dsfg.com"></a>
  <p>Lorem ipsum dolor sit amet.</p>
  <script>
    deger = document.querySelector("a").getAttributeNode("id");
    deger2 = document.querySelector("a").getAttributeNode("id").value;
    document.write(deger + "<br>");
    document.write(deger2);
  </script>
</body>
```

- ⇒ `getAttribute` ile arasındaki fark içeriğin özellik nesnesine erişiyoruz. İçeriğini görmek için `value` kullanmamız lazım

Çıktısı:

Lorem ipsum dolor sit amet.

[object Attr]
test

20.41.6 `removeAttribute()`

Bir özniteliği belirtilen öğeden kaldırır. Eleman üzerinde olmayan bir öznitelik kaldırılmaya çalışıldığında bir istisna ortaya çıkmaz.

`element.removeAttribute(özellik);`

özellik → Gereklidir. Kaldırmak istediğiniz niteliğin adı.

Örnek:

```
<body>
  <div id="test" align="left" width="300px"> </div>
  <script>
    var x = document.querySelector("#test");
    x.removeAttribute("align");
  </script>
</body>
```

Çıktısı:

```
> <div id="test" width="300px"> ... </div>
```

20.41.7 `removeAttributeNode()`

Belge içerisindeki herhangi bir html elemanına `getAttributeNode()` metodu ile belirtilmiş özellik `node`'unu ve değerini kaldırmak/silmek için kullanılır.

Örnek:

```
<body>
<p style="font-size: 15px; background-color: red; color: white;">Lorem ipsum dolor sit amet.</p>
<button>Ekle</button>
<script>
  document.querySelector("button").addEventListener("click", function () {
    var text = document.querySelector("p");
    var silinecekozellik = text.getAttributeNode("style");
    text.removeAttribute(silinecekozellik);
  })
</script>
</body>
```

Çıktısı:



20.41.8 hasAttribute()

Belirtilen öğede istenilen özneniteliğın olup olmadıėını kontrol eder. Var ise true yok ise false döndürür.

element.hasAttribute(öznenite);

Öznenite → Gereklidir. Var olup olmadıėını kontrol etmek istediėiniz öznenite adı.

Örnek:

```
<body>
<a id="test" href="https://www.dsfg.com"></a>
<script>
  var x = document.querySelector("#test");
  document.write(x.hasAttribute("href") + "<br>");
  document.write(x.hasAttribute("target"));
</script>
</body>
```

Çıktısı:

true
false

20.41.9 hasAttributes()

Belirtilen öğede herhangi bir özneliğin olup olmadığını kontrol eder. Var ise true yok ise false döndürür.

Örnek:

```
<body>
  <a id="test" href="https://www.dsfg.com"></a>
  <p>Lorem ipsum dolor sit amet.</p>
  <script>
    var x = document.querySelector("#test");
    var y = document.querySelector("p");
    document.write(x.hasAttributes() + "<br>");
    document.write(y.hasAttributes());
  </script>
</body>
```

Çıktısı:

Lorem ipsum dolor sit amet.

true
false

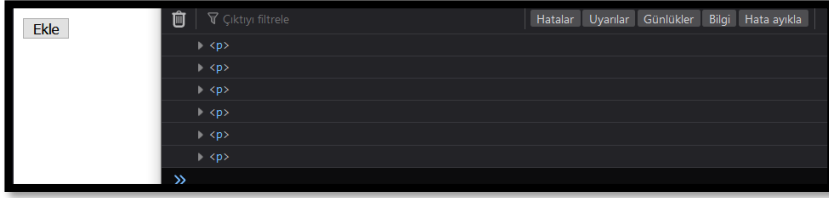
20.42 createElement()

HTML nesnesi oluşturmak için kullanılır. Bellek üzerinde dinamik olarak HTML nesnesi oluşturmak için kullanılır.

Örnek:

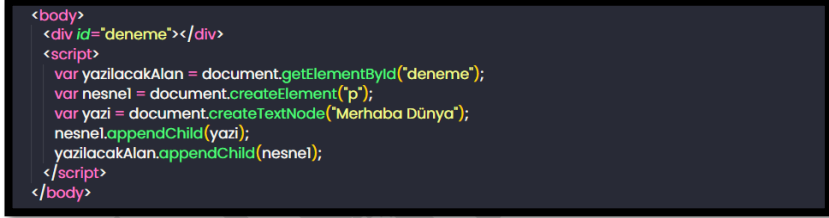
```
<body>
  <button onclick="islem();">Ekle</button>
  <script>
    function islem() {
      var ekle = document.createElement("p");
      console.log(ekle);
    }
  </script>
</body>
```

Çıktısı:



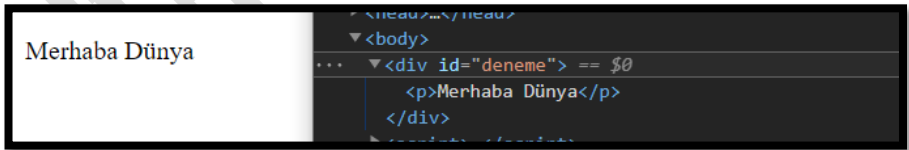
20.43 createElement()

Belirtilen string'i içeren bir düğüm oluşturur. Kısacası oluşturulan elementin içerisine eklenecek string i oluşturur.



Çıktısı:

deneme id sine sahip div etiketinin içerisine p etiketi oluşturduk. createElement metodu ile Merhaba Dünya yazısını oluşturduk. appendChild ile oluşturulan string i p etiketinin içerisine ekledik.



20.44 removeChild()

Belirtilen ögenin belirtilen bir alt düğümünü kaldırır. Kaldırılan düğümü bir Düğüm nesnesi olarak döndürür veya düğüm yoksa null döndürür.

Örnek:

```
<body>
  <ul id="liste">
    <li>Merhaba 1</li>
    <li>Merhaba 2</li>
    <li>Merhaba 3</li>
    <li>Merhaba 4</li>
    <li>Merhaba 5</li>
  </ul>
  <button onclick="sil();">sil</button>
  <script>
    function sil() {
      var ul = document.getElementById("liste");
      ul.removeChild(ul.firstChild);
    }
  </script>
</body>
```

Çıktısı: İki kez sil butonuna tıkladığımızda...

- Merhaba 3
- Merhaba 4
- Merhaba 5

sil

20.45 appendChild()

Belirtilen bir üst düğümün alt öğelerinin sonuna bir düğüm ekler.

Örnek:

```
<body>
  <ul id="liste">
    <li>Merhaba 1</li>
    <li>Merhaba 2</li>
    <li>Merhaba 3</li>
    <li>Merhaba 4</li>
    <li>Merhaba 5</li>
  </ul>
  <button onclick="ekle();">Ekle</button>
  <script>
    function ekle() {
      var li = document.createElement("li");
      var input = document.createTextNode("Merhaba 6");
      li.appendChild(input);
      document.getElementById("liste").appendChild(li);
    }
  </script>
</body>
```

Çıktısı: İki kez ekle butonuna tıkladığımızda...

- Merhaba 1
- Merhaba 2
- Merhaba 3
- Merhaba 4
- Merhaba 5
- Merhaba 6
- Merhaba 6

Ekle

20.46 replaceChild()

Belirtilen düğümün bir alt düğümünü başka bir düğümle değiştirir.

node.replaceChild(yeni, değişecek)

Örnek:

```
<body>
<ul id="liste">
  <li>Merhaba 1</li>
  <li>Merhaba 2</li>
  <li>Merhaba 3</li>
</ul>
<button onclick="degistir()">Değiştir</button>
<script>
  function degistir() {
    var t = document.createTextNode("Musa Yazlık");
    var ekles = document.getElementById("liste").childNodes[1];
    ekles.replaceChild(t, ekles.childNodes[0]);
  }
</script>
</body>
```

Çıktısı: Değiştir butonuna 1 kez tıklandığında...

- Musa
- Merhaba 2
- Merhaba 3

Değiştir

20.47 querySelector()

Belgede belirtilen bir CSS seçiciyle eşleşen ilk öğeyi döndürür. Seçici, birkaç kez kullanılan belgede bir kimlikle eşleşirse (bir "kimlik", bir sayfada benzersiz olması gerektiğini ve birden fazla kullanılmaması gerektiğini unutmayın), ilk eşleşen öğeyi döndürür.

document.querySelector(CSS selectors);

Örnek:

```
<body>
<p>Lorem, ipsum dolor.</p>
<p class="deneme">Lorem ipsum dolor sit amet, consectetur adipisicing elit.</p>
<p>Lorem ipsum dolor sit amet consectetur.</p>

<script>
  var p = document.querySelector('p');
  alert(p.innerHTML);
</script>
</body>
```

Çıktısı:

Lorem, ipsum dolor.

Lorem ipsum dolor sit amet, consectetur adipisicing elit.

Lorem ipsum dolor sit amet consectetur.

Lorem, ipsum dolor.

Tamam

<p> öğesinin ilk Class ="deneme" olan elemanı seçin.

document.querySelector("p.deneme");

id ="test" olan elemanı seçin.

document.querySelector("#test");

20.48 querySelectorAll()

Belgede belirtilen bir CSS seçiciyle eşleşen tüm öğeleri statik bir NodeList nesnesi olarak döndürür. NodeList nesnesi düğümlerin bir koleksiyonunu temsil eder. Düğümlere dizin numaraları erişilebilir. Endeks 0'dan başlar.

İpucu: Belirtilen seçiciyle eşleşen öğelerin sayısını belirlemek için NodeList nesnesinin `length` özelliğini kullanabilirsiniz, ardından tüm öğeler arasında dolaşp istediğiniz bilgiyi alabilirsiniz.

Örnek: div etiketin alt elemanı p'yi seçelim.

`var x = document.querySelectorAll("div > p");`

Örnek: Belgede tüm `<h2>`, `<div>` ve `<span>` öğelerinin arka plan rengini ayarlayalım...

```
<body>
<script>
var x = document.querySelectorAll("h2, div, span");
var i;
for (i = 0; i < x.length; i++) {
  x[i].style.backgroundColor = "green";
}
</script>
</body>
```

Not: querySelector , jquery seçicilere göre daha hızlıdır.

20.49 addEventListener()

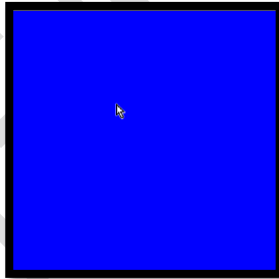
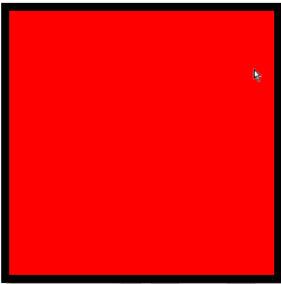
Herhangi bir olayın dinlenerek, gerçekleştiği zaman çalışması istenilen fonksiyonu çalıştırır. 3 parametre alır.

1. parametre olayı belirler. click, mouseover vs.
2. Olay tetiklendiğinde çalışacak fonksiyon
3. İçi içe olan etiketlerin her birinde olay tanımlanmışsa baştan sonra doğru olayları çalıştırıp çalıştırmayacağı. (true, false) . Genelde 3 üncü parametre pek kullanılmaz.

Örnek:

```
<body style="height: 100vh;" id="body">
  <div id="deneme"></div>
  <script>
    var say = 0;
    document.getElementById("body").addEventListener("click", function () {
      if (say == 0) {
        document.body.style.background = "red";
        say++;
      } else {
        document.body.style.background = "blue";
        say--;
      }
    })
  </script>
</body>
```

Çıktısı: Sayfa ilk açıldığında beyaz olacak. Fonksiyon içerisinde verdiğimiz işleme göre body alanına bir kez tıklandığında body alanı kırmızıya ikinci kez tıklandığında body alanı maviye dönecek.



20 removeEventListener

addEventListener ile gerçekleşen olayı sonlandırmak amaçlı kullanılır.

Örnek:

```
<body style="height: 100vh;" id="body">
<script>
var say = 0;
var sayac = 0;
document.getElementById("body").addEventListener("click", tiki1a)
function tiki1a() {
  if (sayac < 5) {
    if (say == 0) {
      document.body.style.backgroundColor = "red";
      say++;
      sayac++;
    } else {
      document.body.style.backgroundColor = "blue";
      say--;
      sayac++;
    }
  } else {
    document.getElementById("body").removeEventListener("click", tiki1a)
  }
}
</script>
</body>
```

Çıktısı: İşlem 5 kez gerçekleştiikten sonra sonlanacaktır. Body alanı kırmızı renk ile kalacaktır.

20.51 Attribute Nesne Özellikleri

20.51.1 length

Belge içerisinde bulunan bir html etiketinin özellik sayısını verir.(id,class,href gibi özellikler)

Örnek:

```
<body>
<div id="merhaba" dir="ltr" class="btn btn-primary">Merhaba</div><br>
<script>
var deneme = document.querySelector("div").attributes.length;
document.write(" Html Etiketinde " + deneme + " özellik bulunuyor");
</script>
</body>
```

Çıktısı:

Merhaba

Html Etiketinde 3 özellik bulunuyor

20.51.2 name

Belge içerisinde bulunan bir html etiketinin özellik ismini verir. Döngü ile sahip olduğu bütün özelliklerin isimleri öğrenilebilir.

Örnek:

```
<body>
<div id="merhaba" dir="ltr" class="btn btn-primary"> Merhaba</div><br>
<script>
var say = document.querySelector("div").attributes.length;
for (let i = 0; i <= say; i++) {
  var deneme = document.querySelector("div").attributes[i].name;
  document.write(" Html etiketinin " + i + "'inci özelliği " + deneme + " özelliiktir."
  + "<br>");
}
</script>
</body>
```

Çıktısı:

Merhaba

Html etiketinin 0'inci özelliği id özelliiktir.
Html etiketinin 1'inci özelliği dir özelliiktir.
Html etiketinin 2'inci özelliği class özelliiktir.

20.51.3 value

Belge içerisinde bulunan bir html etiketinin özelliğinin içeriğini verir.

Örnek:

```
<body>
<div id="merhaba" dir="ltr" class="btn btn-primary"> Merhaba</div><br>
<script>
var say = document.querySelector("div").attributes.length;
for (let i = 0; i <= say; i++) {
  var deneme = document.querySelector("div").attributes[i].name;
  var icerik = document.querySelector("div").attributes[i].value;
  document.write(" Html etiketinin " + deneme + " özelliğinin içeriği: " + icerik +
    "<br>");
}
</script>
</body>
```

Çıktısı:

Html etiketinin id özelliğinin içeriği: merhaba
Html etiketinin dir özelliğinin içeriği: ltr
Html etiketinin class özelliğinin içeriği: btn btn-primary

20.51.4 specified

Belirtilen html etiketinin istenilen özelliğe sahip olup olmadığını kontrol eder. Varsa true yoksa null değerini döndürür.

Örnek:

```
<body>
<div id="merhaba" dir="ltr" class="btn btn-primary"> Merhaba</div><br>
<script>
var sonuc = document.querySelector("div").attributes["id"].specified;
document.write(sonuc);
</script>
</body>
```

Çıktısı:

Merhaba

true

20.51.5 getNamedItem()

Belirtilen html etiketinin özelliğini seçmek amaçlı kullanılır.

Örnek:

```
<body>
<div id="merhaba" dir="ltr" class="btn btn-primary"> Merhaba</div><br>
<script>
  var sonuc = document.querySelector("div").attributes.getNamedItem("class").value;
  document.write(sonuc);
</script>
</body>
```

Çıktısı:

Merhaba

btn btn-primary

20.51.6 setNamedItem()

Belirtilen html etiketine istenilen özelliği ve içeriğini eklemek amaçlı kullanılır.

Örnek:

```
<body>
<div id="merhaba" dir="ltr" class="btn btn-primary"> Merhaba</div><br>
<script>
  var eleman = document.querySelector("div");
  var ozellik = document.createAttribute("style");
  ozellik.value = "background: red;";
  eleman.attributes.setNamedItem(ozellik);
</script>
</body>
```

Çıktısı:



20.51.7 removeNamedItem()

Belirtilen html etiketinin , istenilen özelliğini kaldırmak amaçlı kullanılır.

Örnek:

```
<body>
  <div style="color: ■ white; background-color: ■ red;" id="merhaba" dir="ltr" class="btn
    btn-primary"> Merhaba</div><br>
  <script>
    var eleman = document.querySelector("div").attributes.removeNamedItem("style");
  </script>
</body>
```

Çıktısı: Normalde/Silindikten Sonra



```
• <div id="merhaba" dir="ltr" class="btn btn-primary"> Merhaba
  </div> == $0
•
```

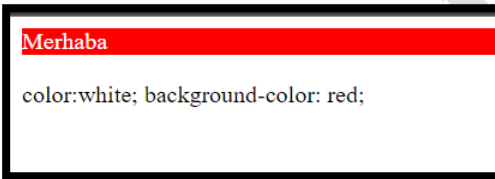
20.51.8 item()

Seçilen html etiketinin belirtilen özelliği seçmek için kullanılır.

Örnek:

```
<body>
<div style="color: ■ white; background-color: ■ red;" id="merhaba" dir="ltr" class="btn
btn-primary"> Merhaba</div><br>
<script>
var eleman = document.querySelector("div").attributes.item("style").value;
document.write(eleman);
</script>
</body>
```

Çıktısı:



20.52 clientLeft

Belge içerisindeki herhangi bir Html elemanının sol kenarlık genişliğini/kalınlığını elde etmek için kullanılır.

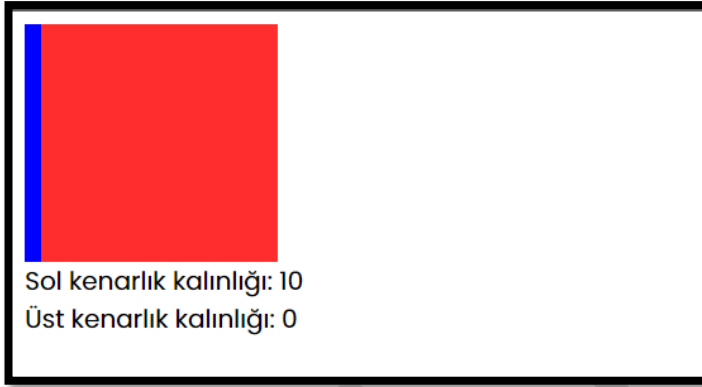
20.53 clientTop

Belge içerisindeki herhangi bir Html elemanının üst kenarlık genişliğini/kalınlığını elde etmek için kullanılır.

Örnek: clientLeft ve clientTop tek örnekte anlatılmıştır.

```
<head>
<meta charset="UTF-8">
<title>Document</title>
<style>
body {
background-color: ■ white;
}
.box {
height: 150px;
width: 150px;
background: ■ rgb(255, 45, 45);
border-left: 10px solid ■ blue;
}
</style>
</head>
<body>
<div class="box">
</div>
<script>
var clientLeft = document.querySelector(".box").clientLeft;
var clientTop = document.querySelector(".box").clientTop;
document.write("Sol kenarlık kalınlığı: " + clientLeft + "<br>");
document.write("Üst kenarlık kalınlığı: " + clientTop);
</script>
</body>
```

Çıktısı: Sol kenarlık 10 px olduğu için 10 değerini verdi. Üst kenarlık olmadığı için 0 değerini verdi.



20.54 clientWidth

Belge içerisindeki herhangi bir html etiketinin genişliğini almak için kullanılır. padding-left + genişlik + padding-right = Toplam genişlik verilir.

20.55 clientHeight

Belge içerisindeki herhangi bir html etiketinin toplam yüksekliğini almak için kullanılır. padding-top + yükseklik + padding-bottom = Toplam yükseklik verilir.

Örnek: clientWidth ve clientHeight tek örnekte anlatılmıştır

```
<head>
<meta charset="UTF-8">
<title>Document</title>
<style>
  body {
    background-color: # white;
  }
  .box {
    height: 150px;
    width: 150px;
    background: # rgb(255, 45, 45);
    padding-left: 10px;
    padding-top: 15px;
    padding-right: 17px;
    padding-bottom: 40px;
  }
</style>
</head>
<body>
<div class="box">
</div>
<script>
  var clientHeight = document.querySelector(".box").clientHeight;
  var clientWidth = document.querySelector(".box").clientWidth;
  document.write("Toplam yükseklik: " + clientHeight + "<br>");
  document.write("Toplam genişlik: " + clientWidth);
</script>
</body>
```

Çıktısı:



20.56 offsetLeft

Belge içerisindeki herhangi bir html elemanının sol üst köşesi ile tarayıcı penceresinin sol üst köşesi arasındaki yatay olan uzaklığı elde etmek için kullanılır.

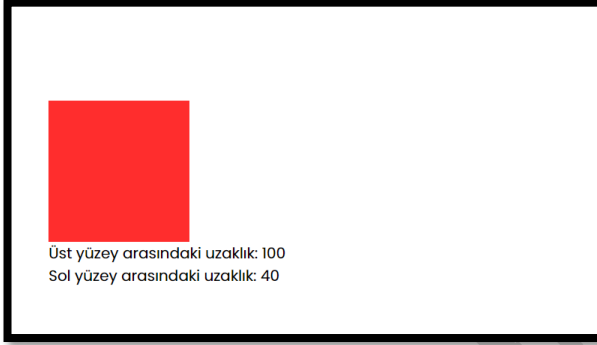
20.57 offsetTop

Belge içerisindeki herhangi bir html elemanının sol üst köşesi ile tarayıcı penceresinin sol üst köşesi arasındaki dikey uzaklığı elde etmek için kullanılır.

Örnek: offsetLeft ve offsetTop tek örnekte anlatılmıştır

```
<head>
<meta charset="UTF-8">
<title>Document</title>
<style>
  body {
    background-color: white;
    margin: 100px 0 0 40px;
  }
  .box {
    height: 150px;
    width: 150px;
    background: rgb(255, 45, 45);
  }
</style>
</head>
<body>
<div class="box">
</div>
<script>
  var offsetTop = document.querySelector(".box").offsetTop;
  var offsetLeft = document.querySelector(".box").offsetLeft;
  document.write("Üst yüzey arasındaki uzaklık: " + offsetTop + "<br>");
  document.write("Sol yüzey arasındaki uzaklık: " + offsetLeft);
</script>
</body>
```

Çıktısı:



20.58 offsetWidth

Belge içerisindeki herhangi bir html etiketinin genişliğini almak için kullanılır. $\text{border-left} + \text{padding-left} + \text{genişlik} + \text{padding-right} + \text{border-right} = \text{Toplam genişlik}$.

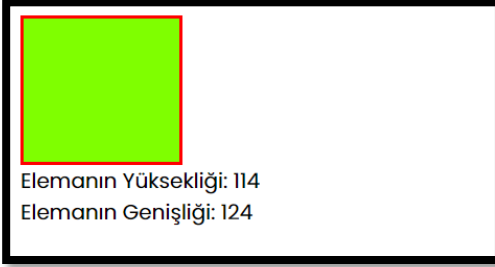
20.59 offsetHeight

Belge içerisindeki herhangi bir html etiketinin yüksekliğini almak için kullanılır. $\text{border-top} + \text{padding-top} + \text{yükseklik} + \text{padding-bottom} + \text{border-bottom} = \text{Toplam yükseklik}$.

Örnek: `offsetWidth` ve `offsetHeight` tek örnekte anlatılmıştır.

```
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<style>
  .box {
    height: 100px;
    width: 100px;
    border: 2px solid red;
    background: chartreuse;
    padding: 5px 10px;
  }
</style>
</head>
<body>
<div class="box"></div>
<script>
  var offsetHeight = document.querySelector(".box").offsetHeight;
  var offsetWidth = document.querySelector(".box").offsetWidth;
  document.write("Elemanın Yüksekliği: " + offsetHeight + "<br>");
  document.write("Elemanın Genişliği: " + offsetWidth);
</script>
</body>
```

Çıktısı:



20.60 scrollLeft

Belge içerisinde her hangi bir html elemanının yatay kaydırılma genişliğini pixel değerini elde etmek için kullanılır. $\text{margin-left} + \text{border-left} + \text{padding-left} + \text{genişlik} + \text{padding-right} + \text{border-right} + \text{margin-right}$

20.61 scrollTop

Belge içerisinde her hangi bir html elemanının yatay kaydırılma genişliğini pixel değerini elde etmek için kullanılır. $\text{margin-top} + \text{border-top} + \text{padding-top} + \text{yükseklik} + \text{padding-bottom} + \text{border-bottom} + \text{margin-bottom}$

Örnek: scrollTop ve scrollLeft tek örnekte anlatılmıştır.

```
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<style>
    .box1 {
        height: 500px;
        width: 500px;
        border: 2px solid red;
        background: #chartreuse;
        padding: 5px 10px;
        overflow: auto;
    }
    .box2 {
        width: 2000px;
        height: 5000px;
        background: #coral;
        margin: 20px;
    }
</style>
</head>
<body>
<div id="box" onscroll="ornek()" class="box">
    <div class="box2"></div>
</div>
<p></p>
<script>
function ornek() {
    var scrollLeft = document.getElementById("box").scrollLeft;
    var scrollTop = document.getElementById("box").scrollTop;
    document.getElementsByTagName("p")[0].innerHTML = "Sol scroll: " + scrollLeft + "
Yükan scroll: " + scrollTop
}
</script>
</body>
```

Çıktısı:



20.62 scrollWidth

Belge içerisindeki herhangi bir html elemanının toplam yatay kaydırma genişlik değerini elde etmek için kullanılır. padding-left + genişlik + padding-right

20.63 scrollHeight

Belge içerisindeki herhangi bir html elemanının toplam dikey kaydırma genişlik değerini elde etmek için kullanılır. padding-top + genişlik + padding-bottom

Örnek: scrollWidth ve scrollHeight tek örnekte anlatılmıştır.

```
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
<style>
    .box1 {
        height: 500px;
        width: 500px;
        background: #chartreuse;
        overflow: auto;
    }
    .box2 {
        width: 5000px;
        height: 5000px;
        background: #coral;
        padding: 10px 5px;
    }
</style>
</head>
<body>
<div id="box" onscroll="ornek()" class="box1">
<div class="box2">Merhaba</div>
</div>
<p></p>
<script>
function ornek() {
    var scrollWidth = document.getElementById("box").scrollWidth;
    var scrollHeight = document.getElementById("box").scrollHeight;
    document.getElementsByTagName("p")[0].innerHTML = "Yatay genişlik: " + scrollWidth + "
    dikey genişlik: " + scrollHeight
}
</script>
</body>
```

Çıktısı:



21. JavaScript DOM Events

Ziyaretçilerin eylemlerine ve komut dosyalarının iç iletişimine tepki vermek için olaylar vardır. Olay, tarayıcıdan bir şey olduğuna dair bir işarettir. Çok çeşitli olaylar var. Bunlardan bazılarını şimdi sizler ile göreceğiz...

21.1 Javascript Fare Olayları (Mouse Events) Türleri

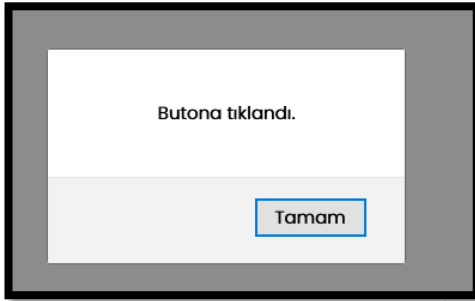
21.1.1 onclick()

Fare ile nesnemizin üzerine sol tuş ile tıkladığımızda belirtilen işlevi görür.

Örnek:

```
<body>
<button onclick="tıkla();">tıkla</button>
<script>
function tıkla() {
  alert("Butona tıklandı.");
}
</script>
</body>
```

Çıktısı: Butona tıkladıktan sonra verilen sonuç...



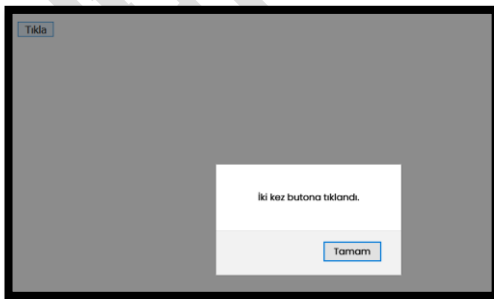
21.1.2 onclick()

Fare ile nesnemizin üzerine iki kez sol tuş ile tıkladığımızda belirtilen işlevi görür.

Örnek:

```
<body>
<button onclick="cift_tikla();">Tıkla</button>
<script>
  function cift_tikla() {
    alert("İki kez butona tıklandı.");
  }
</script>
</body>
```

Çıktısı: Butona iki kez tıkladıktan sonra verilen sonuç...



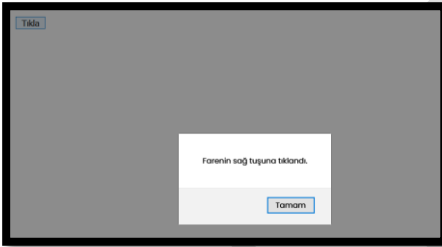
21.1.3 oncontextmenu()

Fare ile nesnemizin üzerine sağ tuş ile tıkladığımızda belirtilen işlevi görür.

Örnek:

```
<body>
  <button oncontextmenu="sag_tikla();">Tıkla</button>
  <script>
    function sag_tikla() {
      alert("Farenin sağ tuşuna tıklandı.");
    }
  </script>
</body>
```

Çıktısı: Butona farenin sağ tuşu ile tıkladığınızda vereceği tepki...



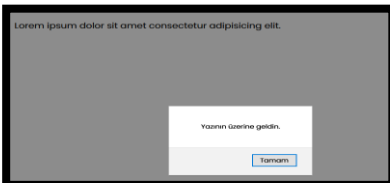
21.1.4 onmouseover()

Fare nesnenin üzerine geldiğinde verilecek tepkiyi çalıştırır.

Örnek:

```
<body>
  <p onmouseover="uzerinegel()">Lorem ipsum dolor sit amet consectetur adipisicing elit.</p>
  <script>
    function uzerinegel() {
      alert("Yazının üzerine geldin.");
    }
  </script>
</body>
```

Çıktısı:



21.1.5 onmouseout()

Fare nesnenin üzerinden çekilince verilecek tepkiyi çalıştırır.

Örnek:

```
<body>
  <p onmouseover="uzerinegel();" onmouseout="gitt();">Lorem ipsum dolor sit amet consectetur
  adipiscing elit.</p>
  <p id="ekle"></p>
  <script>
    function uzerinegel() {
      document.getElementById("ekle").innerHTML = "Üzerine Geldi.";
    }
    function gitt() {
      document.getElementById("ekle").innerHTML = "Gitti...";
    }
  </script>
</body>
```

Çıktısı: Fare nesneni üzerine gelince ve gidince verecek tepki...

Lorem ipsum dolor sit amet consectetur adipiscing elit.

Üzerine Geldi.

Lorem ipsum dolor sit amet consectetur adipiscing elit.

Gitti....

21.1.6 onmouseup()

Fare ile nesnenin üzerine tıklanıp bırakıldığında çalışacak fonksiyonu çalıştırır.

21.1.7 onmousedown()

Fare ile nesnenin üzerine tıklandığında çalışacak fonksiyonu çalıştırır.

Örnek: onmouseup ve onmousedown tek örnekte anlatılmıştır.

```
<body>
<p id="ekle" onmouseup="renk_green()" onmousedown="renk_red();">Lorem ipsum dolor sit amet
consectetur adipisicing
elit.</p>
<script>
function renk_green() {
    document.getElementById("ekle").style.background = "green";
}
function renk_red() {
    document.getElementById("ekle").style.background = "red";
}
</script>
</body>
```

Çıktısı:

Mouse ile tıklandığında...

Lorem ipsum dolor sit amet consectetur adipisicing elit.

Mouse tıklanıp çekildiğinde...

Lorem ipsum dolor sit amet consectetur adipisicing elit.

21.1.8 onmousemove()

Fare nesnemizin üzerinde gezinirken çalışacak fonksiyonu çalıştırır.

Örnek:

```
<body style="height: 100vh; width: 100%; padding: 0; margin: 0;" onmousemove="dolas()">
<p style="margin: 0; font-size: 100px;" id="demo"></p>
<script>
var a = 0;
function dolas() {
    document.getElementById("demo").innerHTML = a += 1;
}
</script>
</body>
```

Çıktısı: fare body alanında ne kadar çok dolaşır ise a değeri de o kadar artar. Her 1 px değişikliği a değerini 1 artırır.

380

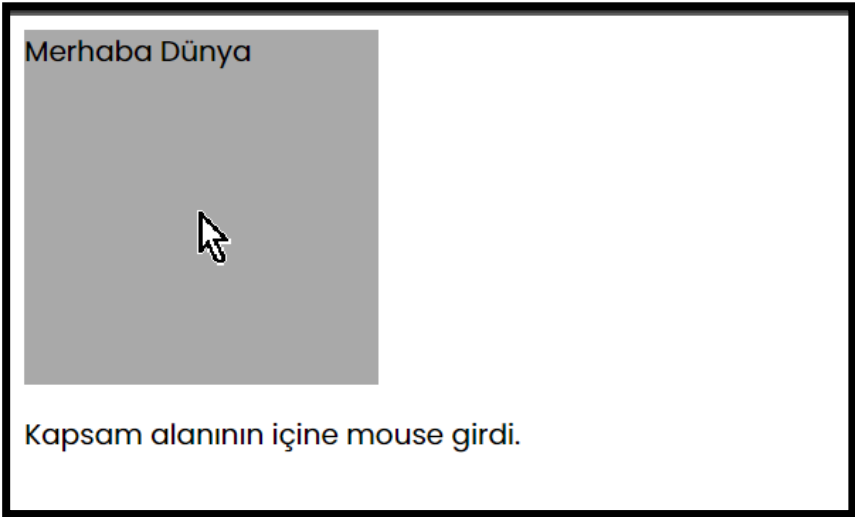
21.1.9 onmouseenter

Belirtilen elemanın mouse ile kapsama alanına girildiğinde belirtilecek olan fonksiyonu(olayı) çalıştırır.

Örnek:

```
<head>
<meta charset="UTF-8">
<title>Document</title>
<style>
.kapsam {
width: 200px;
height: 200px;
background: rgb(169, 169, 169);
}
</style>
</head>
<body>
<div onmouseenter="kapsam()" class="kapsam">
Merhaba Dünya
</div>
<p></p>
<script>
function kapsam() {
document.getElementsByTagName("p")[0].innerHTML = "Kapsam alanının içine mouse girdi.";
}
</script>
</body>
```

Çıktısı:



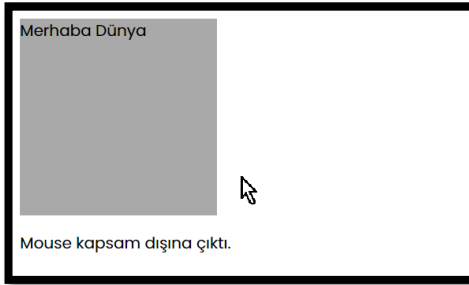
21.1.10 onmouseleave

Belirtilen elemanın içinde bulunan mouse, belirtilen elemanın kapsama alanının dışına çıktığında belirtilecek olan fonksiyonu (olayı) çalıştırır.

Örnek:

```
<head>
<meta charset="UTF-8">
<title>Document</title>
<style>
  .kapsam {
    width: 200px;
    height: 200px;
    background: #169, 169, 169;
  }
</style>
</head>
<body>
<div onmouseleave="kapsamdisi()" class="kapsam">
  Merhaba Dünya
</div>
<p></p>
<script>
  function kapsamdisi() {
    document.getElementsByTagName("p")[0].innerHTML = "Mouse kapsam dışına çıktı.";
  }
</script>
</body>
```

Çıktısı:



21.1.11 onwheel

Belirtilen elemanın üzerindeyken mouse'un tekerleğini aşağı ve yukarı çevirdiğimizde çalışacak fonksiyonu çalıştırır.

Örnek:

```
<body>
<div onwheel="tekerlek()" id="tekerlek">
  Merhaba Dünya
</div>
<script>
  function tekerlek() {
    document.getElementById("tekerlek").style.fontSize = "50px";
  }
</script>
</body>
```

Çıktısı:



21.2 MouseEvent Özellikleri Ve Metotları

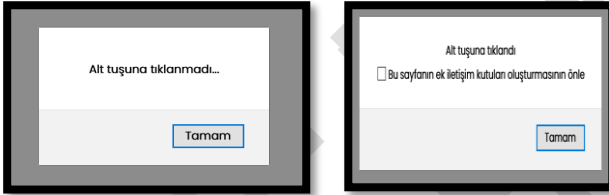
21.2.1 altkey

Bir anahtar olayı tetiklendiğinde "ALT" anahtarının basılı olup olmadığını döndürür.

Örnek:

```
<body>
<button onmousedown="altTikla(event)" id="buton">Tıkla</button>
<script>
function altTikla(event) {
  if (event.altKey) {
    alert("Alt tuşuna tıklandı");
  } else {
    alert("Alt tuşuna tıklanmadı...");
  }
}
</script>
</body>
```

Çıktısı:



21.2.2 shiftkey

Bir anahtar olayı tetiklendiğinde "SHIFT" anahtarının basılı olup olmadığını döndürür. Örnek: kullanımı yukarıdaki altkey gibi.

21.2.3 ctrlkey

Bir anahtar olayı tetiklendiğinde "CTRL" anahtarının basılı olup olmadığını döndürür. Örnek: kullanımı yukarıdaki altkey gibi.

21.2.4 button

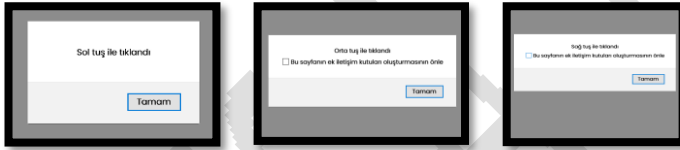
Bir fare olayı tetiklendiğinde farenin hangi tuşuna basıldığını söyler.

0 → Farenin sol tuşu 1 → Farenin orta tekerlek tuşu 2 → Farenin sağ tuşu

Örnek:

```
<body>
<button onmousedown="buttonTikla(event)" id="buton">Tıkla</button>
<script>
function buttonTikla(event) {
    if (event.button == 0) {
        alert("Sol tuş ile tıklandı");
    } else if (event.button == 1) {
        alert("Orta tuş ile tıklandı");
    }
    } else {
        alert("Sağ tuş ile tıklandı");
    }
}
</script>
</body>
```

Çıktısı:



Not : Internet Explorer 8 ve önceki sürümlerin farklı dönüş değerleri vardır

- 1: Sol fare düğmesi
- 2: Sağ fere düğmesi
- 4: Tekerlek düğmesi veya orta düğme (varsa)

21.2.5 which

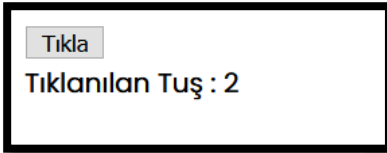
button ile aynı işlevi yapar tek farkı tıklana tuş Çıktısı: farklıdır.

- 1 → Sol fare düğmesi
- 2 → Tekerlek düğmesi veya orta düğme (varsa)
- 3 → Sağ fare düğmesi

Örnek:

```
<body>
<button onmousedown="islem()">Tıkla</button>
<div id="cikti"></div>
<script>
function islem() {
    document.getElementById("cikti").innerHTML = "Tıklanılan Tuş : " + event.which;
}
</script>
</body>
```

Çıktısı: Orta tuşa bastığımızda...



21.2.6 buttons

button ve which gibi farenin tıklana tuşunu verir. Ek olarak farede bulunan geri ve ileri tuşlarında verir.

- 1 → Sol fare tuşu
- 2 → Orta tekerlek tuşu
- 4 → Sağ fare tuşu
- 8 → Geri fare tuşu
- 16 → İleri fare tuşu

Not: Farede geri ve ileri tuşu olmadığından ve button ve which ile işlevi aynı olduğundan Örnek işlem yapmıyorum. Onuda siz yaparsınız değil mi ? 😊

21.2.7 clientX && clientY

clientX mouse olayı tetiklendiğinde geçerli pencereye(tarayıcı sayfasına) göre mouse işaretçisinin yatay(x) koordinatını döndürür. clientY :Mouse olayı tetiklendiğinde geçerli pencereye(tarayıcı sayfaya) göre mouse işaretçisinin dikey(y) koordinatını döndürür.

Örnek:

```
<head>
<meta charset="UTF-8">
<title>Document</title>
<style>
* {
padding: 0;
margin: 0;
}
body {
width: 100%;
height: 100vh;
}
</style>
</head>
<body onmousemove="konum(event);">
<p class="ekle"></p>
<script>
function konum(event) {
var x = event.clientX;
var y = event.clientY;
var konum = "X coords: " + x + ", Y coords: " + y;
document.querySelector(".ekle").innerHTML = konum;
}
</script>
</body>
```

Çıktısı: Mouse un x ve y koordinatına göre konumu.

X coords: 207, Y coords: 24

21.2.8 pageX && page Y

clientX – clientY ile aynı işlevi yapar. Tek farkı scroll kaydırıldığında öğenin sayfadaki X ve Y konumu değişir.

Örnek:

[illegible]

Sayfaya fazladan br etiketi ekleyerek scroll oluşmasını sağladım. Scroll u en aşağıya indirdiğinizde client de Y ordinatında maksimum 400 px i görürken page ile çok daha yükseğini gördük.

Çıktısı:

X coords: 631, Y coords: 1403

21.2.9 detail

Farenin sol tuşuna kaç kez ard arda tıklandığı sayar ve gösterir. Tıklama bir an kesildiğinde sayaç sıfırlanır. Firefox tarayıcısında çalışmamaktadır.

Örnek:

```
<body>
<button onclick="tikla(event);">tıkla</button>
<p id="sonuc"></p>
<script>
function tikla(event) {
    var tika = event.detail;
    document.getElementById("sonuc").innerHTML = tika + " kez tıklandı";
}
</script>
</body>
```

Çıktısı: 16 kez ard arda tıklama sonucu.

tıkla

16 kez tıklandı

21.3 JS- DOM Events - Klavye Olayları

21.3.1 onkeydown

Kullanıcı bir tuşa ilk bastığı zaman gerçekleşecek olayı tetikler.

21.3.2 onkeypress

Kullanıcı bir tuşa bastığında veya basılı tuttuğunda gerçekleşecek olayı tetikler.

21.3.3 onkeyup

Kullanıcı basılı tuşu bıraktığında gerçekleşecek olayı tetikler.

Örnek: onkeyup, onkeydown , onkeypress

```
<body>
<input onkeyup="up()" onkeydown="down()" onkeypress="press()" type="text" id="renk">
<script>
function up() {
    document.getElementById("renk").style.background = "red";
}
function down() {
    document.getElementById("renk").style.background = "blue";
}
function press() {
    document.getElementById("renk").style.background = "green";
}
</script>
</body>
```

21.3.4 keycode

Klavyeden basılan normal veya özel bir tuşun unicode değerini verir.

Örnek:

```
<body>
<input type="text" id="renk" onkeypress="ney(event)">
<div id="ekle"></div>
<script>
  function ney(event) {
    var x = event.keyCode;
    document.getElementById("ekle").innerHTML = "Unicode Değeri : " + x
  }
</script>
</body>
```

Çıktısı: t tuşuna basınca verdiği çıktı.

t
Unicode Değeri : 116

21.3.5 key

Klavyeden basılan tuşu geri döndürür.

Örnek:

```
<body>
<input type="text" id="renk" onkeydown="ney(event)">
<div id="ekle"></div>
<script>
  function ney(event) {
    var x = event.key;
    document.getElementById("ekle").innerHTML = "Tıkladığınız Tuş : " + x
  }
</script>
</body>
```

Çıktısı: v tuşuna basıldığında verdiği çıktı.

v

Tıkladığınız Tuş : v

22. JavaScript Bom Olayları

22.1 Bom Nedir?

BOM, **Browser Object Model**'in kısaltılmışı olup, tarayıcıların kendileri ile ilişkili yöntemleri ve özellikleri işlemek için kullanılan bir tarayıcı nesnesi hiyerarşisidir. Javascript'in tarayıcıyla konuşmasına imkan sağlar. Herhangi bir resmi standardı yoktur. Fakat modern tarayıcılar Javascript etkileşimi için genellikle aynı yöntemleri ve özellikleri kullanmaktadır.

Browser Object Model'i oluşturan nesneler, pencere nesnesini, gezgin nesnesini, ekran nesnesini, geçmişini, konum nesnesini ve belge nesnesini içerir.

Belge Nesnesi, tarayıcı penceresine yüklenen belgenin veya web sayfasının yöntemlerini ve özelliklerini değiştirmek için kullanılan nesnelerden oluşur. Tarayıcı penceresinde yüklü olan web sayfasını temsil eder. Belgeyi oluşturan her bir HTML ögesi veya etiketi de bir nesne olarak kabul edilir. Browser Object Model'i oluşturan herhangi bir nesneyi açıkça oluşturmak gerekli değildir. Bir web tarayıcısı bir web sayfası açtığında nesneler otomatik olarak oluşturulur.

Bom nesnesi 5 ara birimden oluşur. Bunlar;

1. window
2. Screen
3. Location
4. History
5. Navigator

Bu ara birimlerin kendine özel özellikleri vardır. Şimdi her arabirime göre özellikleri tek görelim.

22.2 Window Arabirimi

22.2.1 open()

Yeni browser (tarayıcı) penceresi açmak için kullanılır. Open() metodu 4 parametre alır.

1. Parametre : URL (Açılacak pencerenin adresi)

2. Parametre : Pencerenin adı

3.Parametre : Pencerenin özellikleri

4.Parametre : Pencerenin geçmiş bilgisi

→ Üçüncü parametrede açılacak pencere için verilebilecek özellikler.

width : Açılacak olan pencerenin genişliğini ayarlamak için kullanılır.

height : Açılacak olan pencerenin yüksekliğini ayarlamak için kullanılır.

left : Açılacak olan pencerenin yatay konumunu ayarlamak için kullanılır.

top : Açılacak olan pencerenin dikey konumunu ayarlamak için kullanılır.

→ Her tarayıcıda istenildiği gibi çalışmayan bazı özellikler.

fullscreen : Açılacak olan pencerenin tam ekran modunda gösterilip gösterilmeyeceğini ayarlamak için kullanılır. (yes & no)

location : Açılacak olan pencerede adres çubuğunun gösterilip gösterilmeyeceğini / değiştirilebilir olup olmadığını ayarlamak için kullanılır. (yes & no)

menubar : Açılacak olan pencerede menü çubuğunun gösterilip gösterilmeyeceğini ayarlamak için kullanılır. (yes & no)

scrollbars : Açılacak olan pencerede kaydırma çubuğunun gösterilip gösterilmeyeceğini ayarlamak için kullanılır. (yes & no)

status : Açılacak olan pencerede durum çubuğunun gösterilip gösterilmeyeceğini ayarlamak için kullanılır. (yes & no)

toolbar : Açılacak olan pencerede navigasyon araç çubuğunun gösterilip gösterilmeyeceğini ayarlamak için kullanılır. (yes & no)

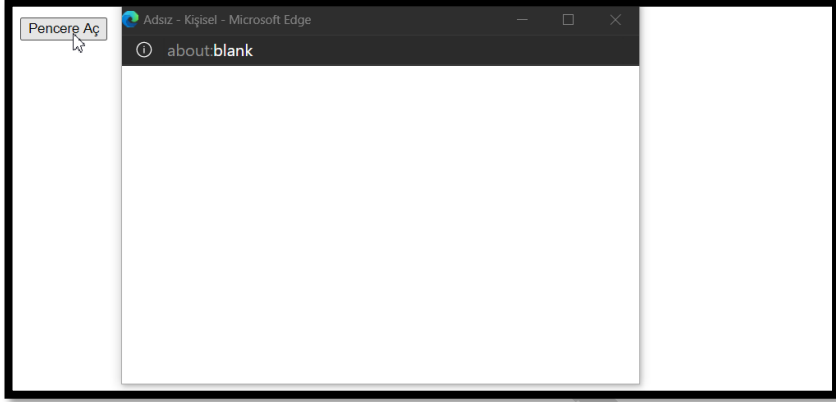
titlebar : Açılacak olan pencerede başlık çubuğunun / başlık çubuğu alanının gösterilip gösterilmeyeceğini ayarlamak için kullanılır. (yes & no)

resizable : Açılacak olan pencerenin boyutlanabilir olup olmadığını ayarlamak için kullanılır. (yes & no)

Örnek:

```
<script>
function PencereAc() {
    var pencere = window.open("", "Yeni Pencere", "width=500,height=300,left=100,top=100");
}
</script>
```

Çıktısı:



22.2.2 opener()

[Window](#) arabiriminin `opener` özelliği, `open()` kullanarak bu pencereyi açmış olan diğer pencereye bir referans değeri döndürür. Başka bir deyişle, eğer A penceresi B penceresini açmışsa, B.opener geriye A döndürür. Hiç bir şey anlamadınız anlıyorum bir örnek ile açıklayayım. 😊

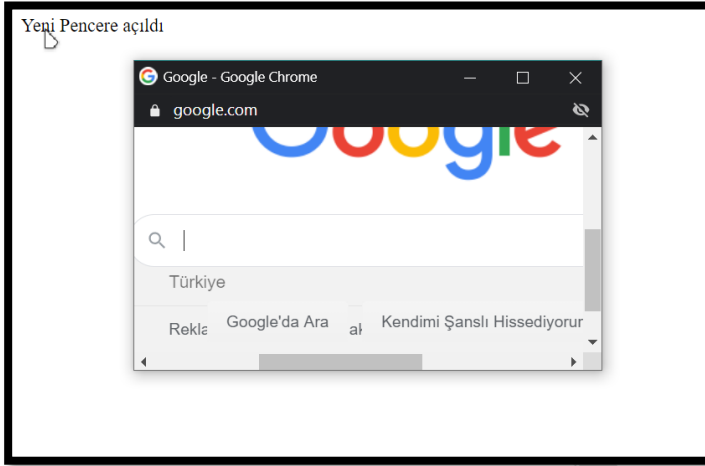
Örnek:

```
<body>
<button onclick="tıkla()">Tıkla</button><br>
<p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Error, fugit.</p>

<script>
function tıkla() {
  var deger = window.open("https://www.google.com", "ikinci pencere", "width=400,height=200,left=100,top=150");
  deger.opener.document.write("Yeni Pencere açıldı");
}
</script>
</body>
```

Çıktısı:

Şimdi olay şu tıkla butonuna tıklanınca ikinci bir sayfa açılarak google sayfası oluşacak. Bu olay tetiklendiğinde `opener` ile bu olayın tetiklenmesini sağlayan sayfaya istediğimiz değişikliği yapmamızı sağlandı. `Opener` kısaltması bu işe yarıyor. `open` ile ikinci bir sayfa açtığımızda birinci sayfada bir değişiklik olsun istiyorsak `opener` kullanmamız gerekli.



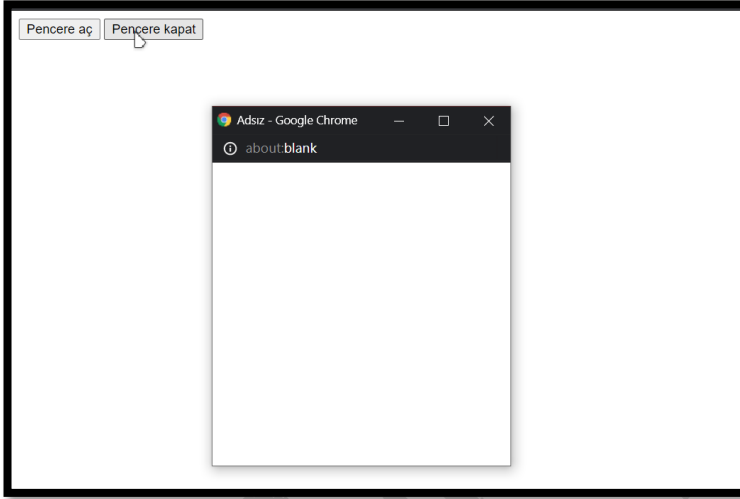
22.2.3 close()

Açık olan bir pencereyi kapatmak amaçlı kullanılır.

Örnek:

```
<body>
<button onclick="ac()">Pencere aç</button>
<button onclick="kapat()">Pencere kapat</button>
<script>
var deneme;
function ac() {
  deneme = window.open("", "deneme sayfası", "height=300,width=300,left=200,top=200");
}
function kapat() {
  deneme.close() // open ile oluşturduğumuz pencereyi kapattık.
  //window.close olarak yazarsak var olan pencere kapatılır.
}
</script>
</body>
```

Çıktısı: Pencere kapat butonuna tıklanınca Adsız adındaki pencere kapanacaktır.



22.2.4 closed

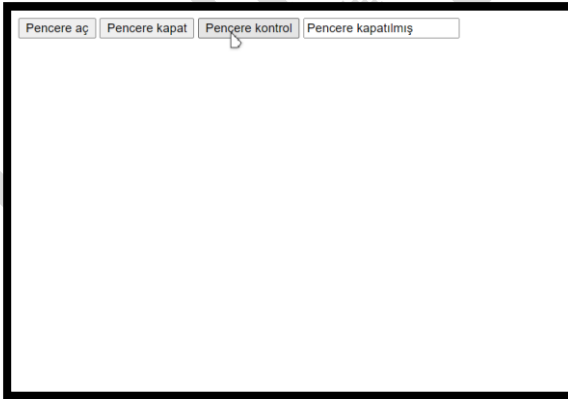
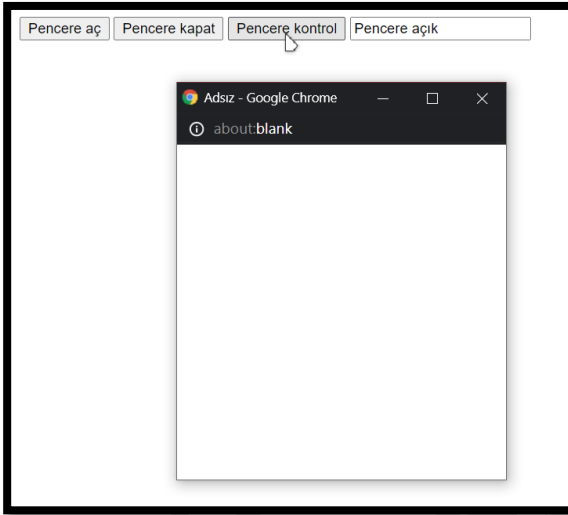
Bir pencerenin kapatılıp kapatılmadığını kontrol ederek sonucu true veya false olarak geri döndürür.

Örnek:

```
<body>
  <button onclick="ac()">Pencere aç</button>
  <button onclick="kapat()">Pencere kapat</button>
  <button onclick="kontrol()">Pencere kontrol</button>
  <input type="text">
</script>
<script>
  var deneme;
  function ac() {
    deneme = window.open("", "deneme sayfası", "height=300,width=300,left=200,top=200");
  }
  function kapat() {
    deneme.close() // open ile oluşturduğumuz pencereyi kapattık.
    // window.close olarak yazarsak var olan pencere kapatılır.
  }

  function kontrol() {
    var sonuc = deneme.closed;
    if (sonuc == true) {
      document.querySelector("input").value = "Pencere kapatılmış";
    } else {
      document.querySelector("input").value = "Pencere açık";
    }
  }
</script>
</body>
```

Çıktısı: 1. resim : Pencere aç butonuna tıkladıktan sonra Pencere kontrol tıklandığında verdiği sonuç. **2. resim** : Ardından pencere kapat tuşuna tıklanıp tekrardan Pencere sonuç butonuna tıklanınca verdiği sonuç.



22.2.5 stop()

Pencerenin yükleme işlemini durdurmak için kullanılır.

Örnek:

```
<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
  <script>
    window.stop()
  </script>
</head>
<body>
  <button onclick="ac()">Pencere aç</button>
  <button onclick="kapat()">Pencere kapat</button>
  <button onclick="kontrol()">Pencere kontrol</button>
  <input type="text">
</body>
</html>
```

Çıktısı: Normalde sayfada olması gereken butonların ve input un olmadığı bir sayfa çıktısı verecektir. Çünkü head etiketleri arasında sayfanın yüklemesini durduran window.stop komutunu koyduk.

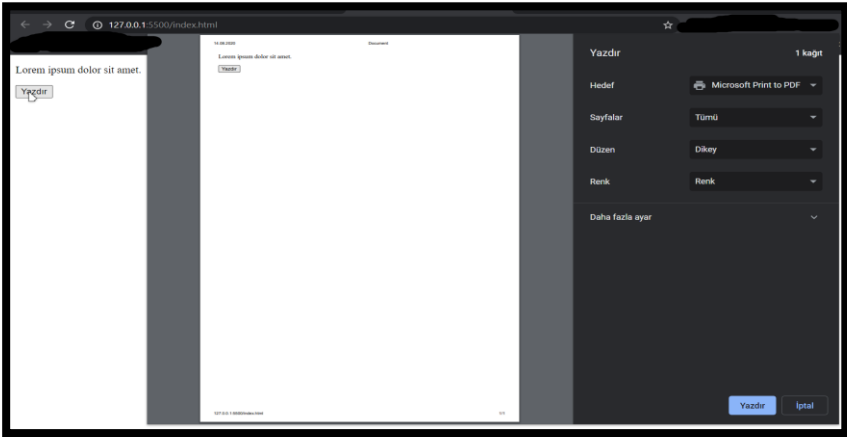
22.2.6 print()

Pencere içeriğini yazdırmak için kullanılır.

Örnek:

```
<body>
<p>Lorem ipsum dolor sit amet.</p>
<button onclick="yazdir()">Yazdır</button>
<script>
function yazdir() {
  window.print();
}
</script>
</body>
```

Çıktısı:



22.2.7 confirm()

Kullanıcıdan onay almak için bir pencere oluşturur. Onayın sonucunu true veya false olarak geri döndürür.

Örnek:

```
<body>
<p>Lorem ipsum dolor sit amet.</p>
<button onclick="gonder()">Gönder</button>
<script>
  function gonder() {
    if (window.confirm("Göndermek istiyor musun?") == 1) {
      document.write("İşlem onaylandı.");
    }
    else {
      document.write("İşlem reddedildi.");
    }
  }
</script>
</body>
```

Çıktısı: Tamam a tıklanınca.

127.0.0.1:5500 web sitesinin mesajı

Göndermek istiyor musun?

Tamamİptal

İşlem onaylandı

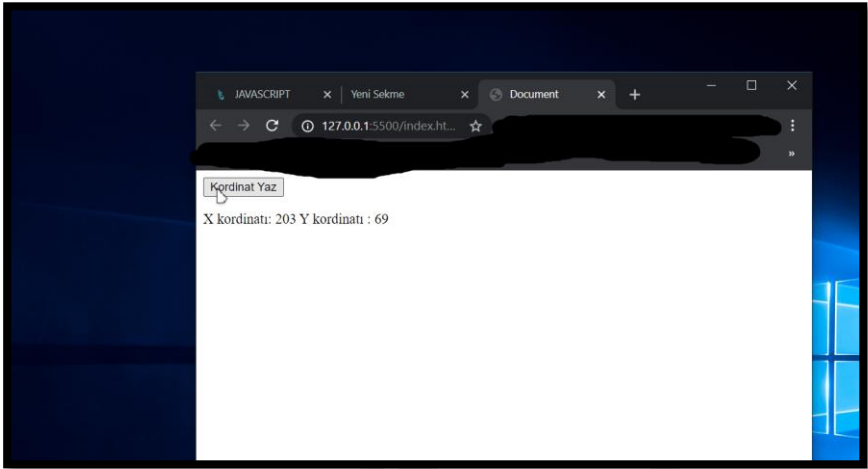
22.2.8 screenX ve screenY

Pencerenin bilgisayar ekranına göre yatay(screenX) ve dikey(screenY) kordinat değerini geriye döndürür.

Örnek:

```
<body>
<button onclick="kordinat()">Kordinat Yaz</button> <br>
<p></p>
<script>
  function kordinat() {
    var xkordinat = window.screenX;
    var ykordinat = window.screenY;
    document.querySelector("p").innerHTML = "X kordinatı: " + xkordinat + " Y kordinatı: " + ykordinat;
  }
</script>
</body>
```

Çıktısı:



22.2.9 innerWidth ve innerHeight

Pencerenin içerik alanının genişlik(innerWidth) ve yükseklik(innerHeight) değerliklerini almak için kullanılır.

Örnek:

```
<body>
  <p id="sonucbir"></p>
  <p id="sonuciki"></p>
  <script>
    var genislik = window.innerWidth;
    var yukseklik = window.innerHeight;
    document.getElementById("sonucbir").innerHTML = "Genislik Değeri : " + genislik;
    document.getElementById("sonuciki").innerHTML = "Yükseklik Değeri : " + yukseklik;
  </script>
</body>
```

Çıktısı:



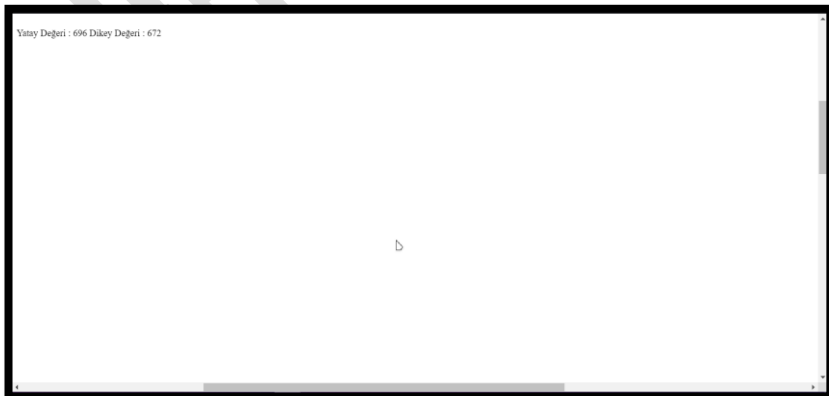
22.2.10 pageXOffset & pageYOffset

Sayfa içerisindeki yatay(pageXOffset) ve dikey(pageYOffset) kaydırma değerlerini verir.

Örnek:

```
<body onmousemove="deneme();" style="width: 3000px; height: 3000px;">  
<p style="position: fixed; id="deger"></p>  
<script>  
  function deneme() {  
    var yatay = window.pageXOffset;  
    var dikey = window.pageYOffset;  
    document.getElementById("deger").innerHTML = "Yatay Değeri : " + yatay + " Dikey Değeri : " + dikey;  
  }  
</script>  
</body>
```

Çıktısı:



22.2.11 scrollX & scrollY

Sayfa içerisindeki yatay(scrollX) ve dikey(scrollY) kaydırma değerlerini verir. pageXOffset ve pageYOffset ile aynı işlemi yaptığı için örnek verilmemiştir.

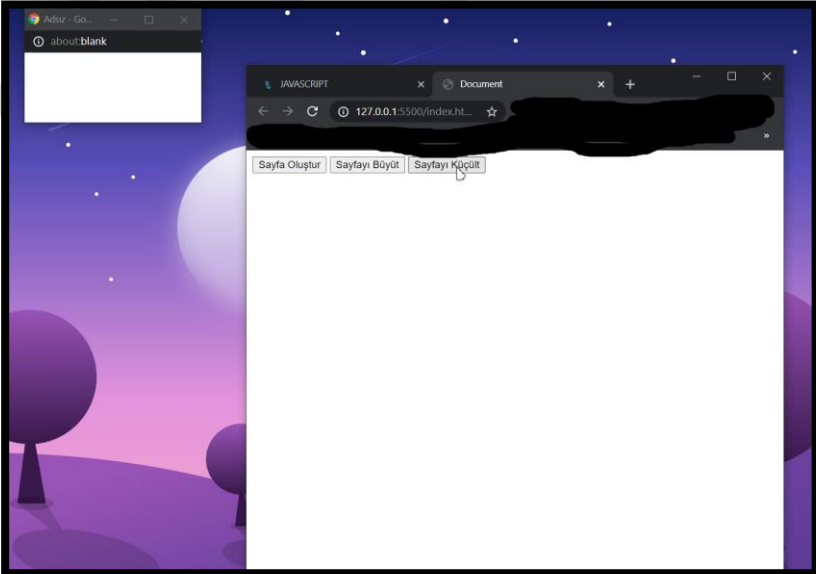
22.2.12 resizeBy()

Sayfayı istenilen değerler doğrultusunda büyütüp küçültmek için kullanılır.

Örnek:

```
<body>
<button onclick="sayfaolustur()">Sayfa Oluştur</button>
<button onclick="sayfayibuyut()">Sayfayı Büyüt</button>
<button onclick="sayfayikucult()">Sayfayı Küçült</button>
<script>
var yenipencere;
function sayfaolustur() {
  yenipencere = open("", "deneme", "width=500,height=500");
}
function sayfayibuyut() {
  yenipencere.resizeBy(100,100);
}
function sayfayikucult() {
  yenipencere.resizeBy(-100,-100);
}
</script>
</body>
```

Çıktısı:



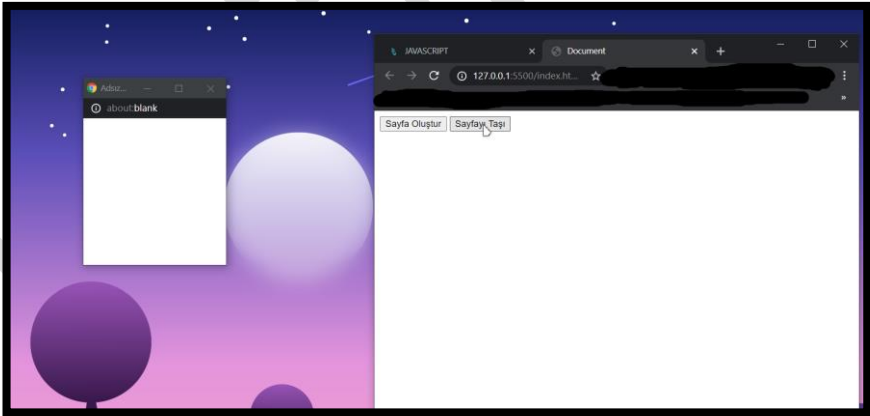
22.2.13 moveBy()

Verilen değerler doğrultusunda açık olan pencerenin bilgisayar ekranına göre pencerenin taşınmasını sağlar.

Örnek:

```
<body>
<button onclick="sayfaolustur()">Sayfa Oluştur</button>
<button onclick="tasi()">Sayfayı Taşı</button>
<script>
var yenipencere;
function sayfaolustur() {
  yenipencere = open("", "deneme", "width=200 , height=200 ");
}
function tasi() {
  yenipencere.moveBy(100,100);
}
</script>
</body>
```

Çıktısı:



22.2.14 scrollBy

Scroll u verilen değerlere göre kaydırmak için kullanılır.

Örnek:

```
<body style="width: 3000px; height: 3000px;">
  <button style="position: fixed;" onclick="kaydir()">Kaydır</button>
  <script>
    function kaydir() {
      window.scrollBy(100, 100);
    }
  </script>
</body>
```

Çıktısı: kaydır butonuna 3 kez tıklanınca.



22.3 Screen Arabirimi

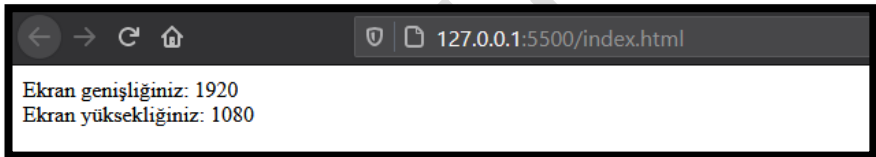
22.3.1 width & height

Kullanıcının sahip olduğu ekranının genişliğini ve yüksekliğini verir. Eğer sizde tam olarak ekran genişliğini yüksekliğini doğru veriyorsa. Ayarlardan tam olarak ekranınıza ait çözünürlüğü mü kullanıyorsunuz kontrol ediniz. Mesela bende 1920 ye 1080 vermesi gerekirken farklı değerler veriyordu. Ben ölçek ve düzeni %125 olarak ayarlamışım. %100 ayarlayınca sonuç 1920x1080 olarak verdi.

Örnek:

```
<body>
  <script>
    document.write("Ekran genişliğiniz: " + screen.width + "<br>");
    document.write("Ekran yüksekliğiniz: " + screen.height);
  </script>
</body>
```

Çıktısı:



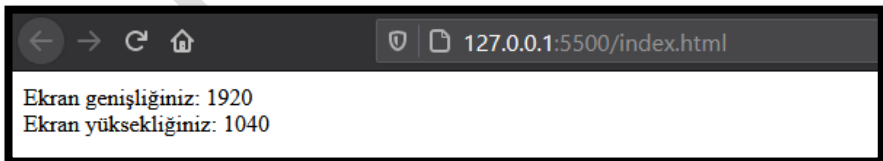
22.3.2 availwidth & availheight

Kullanıcının kullanmış olduğu işletim sisteminde bulunan görev çubunuğunu hesaplamaya dahil etmeden ekran genişliğini ve ekran yüksekliğini verir.

Örnek:

```
<body>
  <script>
    document.write("Ekran genişliğiniz: " + screen.availWidth + "<br>");
    document.write("Ekran yüksekliğiniz: " + screen.availHeight);
  </script>
</body>
```

Çıktısı:



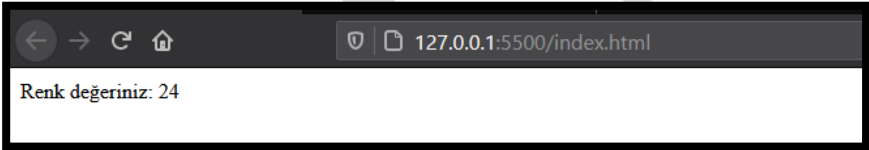
22.3.3 colorDepth

Kullanıcının kullanmış olduğu ekranının renk kalitesini verir. Genelde 24 sonucunu verir ama bu sonuç ekran için kullanılan panelin renk kalitesine göre çok daha yüksek olabilir. Ama dijital ekranlarda genel anlamda 24 bit yani 16 milyon renk derinliği bulunur.

Örnek:

```
<body>
<script>
  document.write("Renk değeriniz: " + screen.colorDepth);
</script>
</body>
```

Çıktısı:



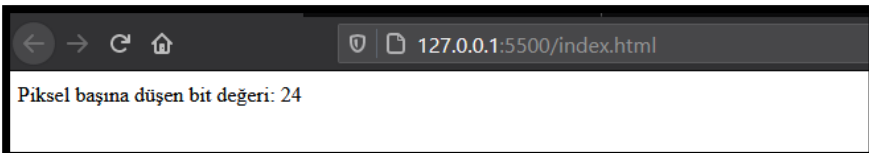
22.3.4 pixelDepth

Kullanıcının kullanmış olduğu ekranının piksel başına düşen bit değerini verir.

Örnek:

```
<body>
<script>
  document.write("Piksel başına düşen bit değeri: " + screen.pixelDepth);
</script>
</body>
```

Çıktısı:



22.4 Navigator Arabirimi

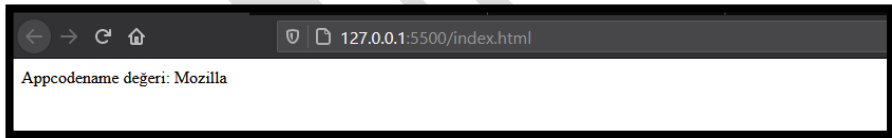
22.4.1 appCodeName

Tarayıcının cod adı referansını verir. Genel anlamda Mozilla değerini verir.

Örnek:

```
<body>
  <script>
    document.write("Appcodename değeri: " + navigator.appCodeName);
  </script>
</body>
```

Çıktısı:



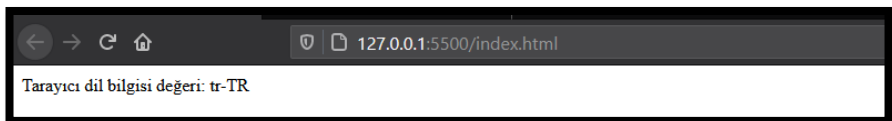
22.4.2 language

Tarayıcının dil bilgisini verir. ISO kısaltmasına göre sonuç verir.

Örnek:

```
<body>
  <script>
    document.write("Appcodename değeri: " + navigator.language);
  </script>
</body>
```

Çıktısı:



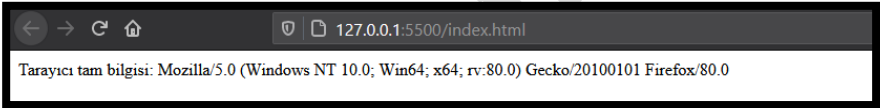
22.4.3 userAgent

Kullanıcının kullandığı tarayıcının tam bilgisini almak için kullanılır.

Örnek:

```
<body>
<script>
document.write("Tarayıcı tam bilgisi: " + navigator.userAgent);
</script>
</body>
```

Çıktısı: Firefox için verdiği sonuç



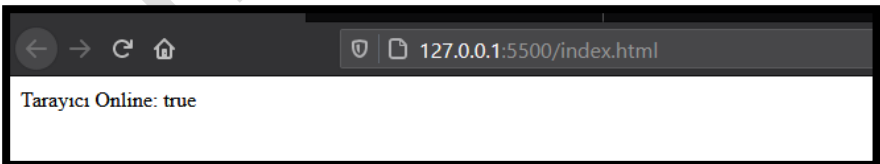
22.4.4 onLine

Tarayıcının çevrim içi modda çalışıp çalışmadığını kontrol eder.

Örnek:

```
<body>
<script>
document.write("Tarayıcı Online: " + navigator.onLine);
</script>
</body>
```

Çıktısı:



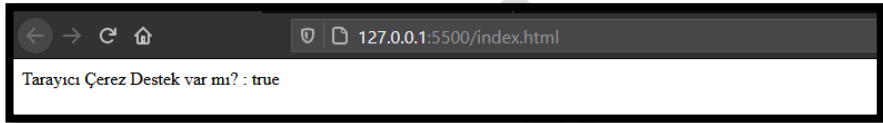
22.4.5 cookieEnabled

Tarayıcının çerezlerinin kullanımının destekleyip desteklemediğini kontrol edip sonucu geriye döndürür. Yani çerezlerin aktif olup olmadığını kontrol eder.

Örnek:

```
<body>
  <script>
    document.write("Tarayıcı Çerez Destek var mı? : " + navigator.cookieEnabled);
  </script>
</body>
```

Çıktısı:



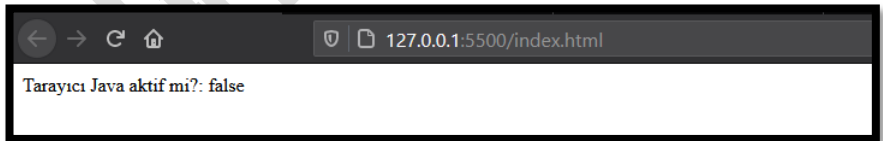
22.4.6 javaEnabled()

Tarayıcıda java ayarının açık olup olmadığını kontrol eder. Açık ise true değerini, kapalı ise false değerini döner.

Örnek:

```
<body>
  <script>
    document.write("Tarayıcı Java aktif mi?: " + navigator.javaEnabled());
  </script>
</body>
```

Çıktısı:



22.5 Location Arabirimi

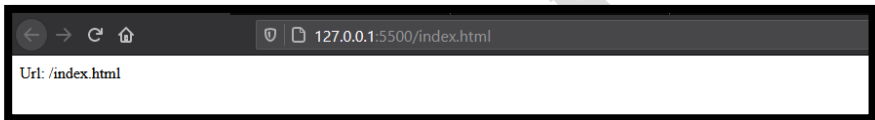
22.5.1 pathname

Bir dosyanın dosya yolu bilgisini domain dahil olmadan elde etmek veya değiştirmek için kullanılır. Mesela <https://musayazlik.com/yazilim/> url den sadece /yazilim/ kısmını geriye döndürecektir. Tam url bilgisini almak için window.location.href kullanabilirsiniz.

Örnek:

```
<body>
<script>
  document.write("Url: " + location.pathname);
</script>
</body>
```

Çıktısı:



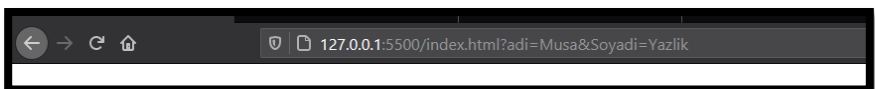
22.5.2 search

Tarayıcı url kısmında bulunan sorgu bilgisini elde etmek veya değiştirmek için kullanılır. Bu biraz php ile ilgili aslında. Php de get yöntemi ile gönderme yaptığımız bilgiler tarayıcıdaki url kısmında ? den sonra gözükür biz buna sorgu bilgisi diyoruz. Search ile ? dahil olmak üzere bu bilgiyi tam olarak verir.

Örnek:

```
<body>
<script>
  var sorgu = location.search="?adi=Musa&Soyadi=Yazlik";
  document.write(sorgu);
</script>
</body>
```

Çıktısı:



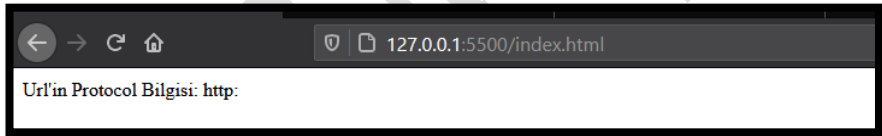
22.5.3 protocol

Url nin protokol bilgisini verir. http,https,file: gibi bir sonuç verir. Ben Visual studio code programında live server eklentisini kullanarak bir local server oluşturduğum için ve ssl si olmadığı için bende http: olarak çıktı verdi. Siz direk index.html dosyasını tarayıcıda açarsanız size file: çıktısını verecektir.

Örnek:

```
<body>
  <script>
    document.write("Url'in Protocol Bilgisi: " + location.protocol);
  </script>
</body>
```

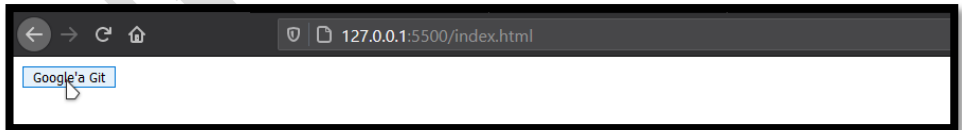
Çıktısı:



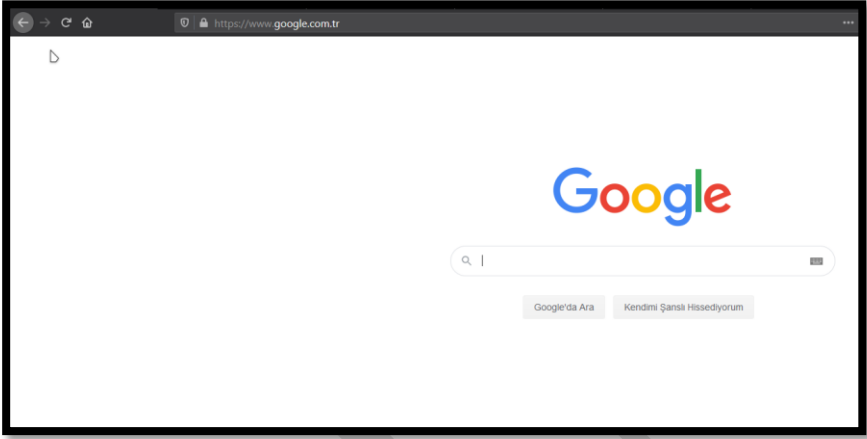
22.5.4 assign()

Tarayıcıda yeni adresin yüklenmesi için kullanılır. Mesela bir buton basıldığında sayfada Google ana sayfası yüklensin isterseniz. O sayfanın içerişi yüklenir.

Örnek:



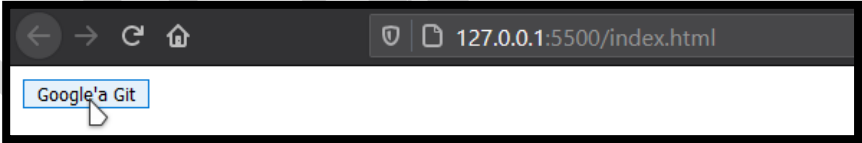
Çıktısı: Google'a Git butonuna tıklanınca.



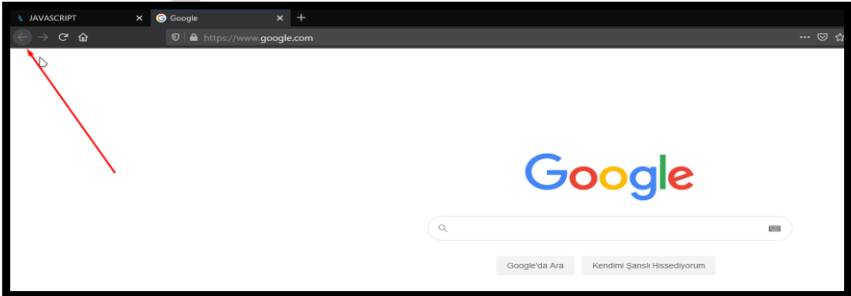
22.5.5 replace()

Assign ile aynı işlevi yapar tek farkı bir önceki sayfanın tüm verileri silinir. Yani bir önceki sayfaya geri dönüş yapamazsınız.

Örnek:



Çıktısı: Gözüktüğü üzere geri tuşu çalışmıyor.



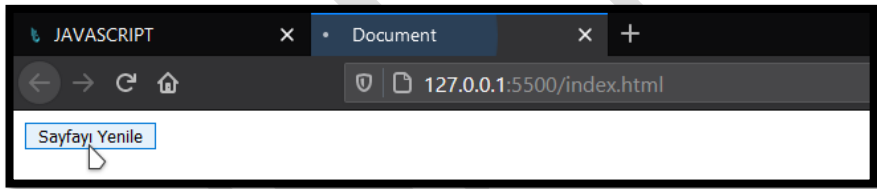
22.5.6 reload()

Sayfayı yenilemek için kullanılır.

Örnek:

```
<body>
  <button onclick="yenile()">Sayfayı Yenile</button>
  <script>
    function yenile() {
      location.reload()
    }
  </script>
</body>
```

Çıktısı: Sayfayı yenilemek için kullanıldığından dolayı pek bir çıktısı yok. 😊



22.6 Zamanlayıcılar

22.6.1 setTimeout()

Belirtilen fonksiyonu süre bitiminde bir kez çalıştıran fonksiyondur. İki parametre alır. Birinci parametreye çalışması istenilen fonksiyon, ikinci parametre ye hangi süre bitiminde çalışması istenildiği yazılır. Verilen süre ms cinsinden verilir.

Örnek:

```
<body>
  <script>
    function yaz() {
      document.write("Süreniz doldu...");
    }
    setTimeout(yaz, 5000);
    setTimeout(() => {
      document.write("<br>"+"Sonra tekrar deneyin...");
    }, 7000);
  </script>
</body>
```

Çıktısı: 5000 ms = 5 saniye sonra süreniz doldu.... yazacak. 2 saniye sonrada sonra tekrar deneyin... yazacak.

Süreniz doldu...
Sonra tekrar deneyin...

22.6.2 clearTimeout

setTimeout ile oluşturulmuş olan zamanlayıcıyı iptal etmek için kullanılır.

Örnek:

```
<body>
<button onclick="durdur()">Durdur</button>
<p id="deneme"></p>
<p id="denemel"></p>
<script>
function yaz() {
    document.getElementById("deneme").innerHTML="Süreniz doldu..";
}
setTimeout(yaz, 5000);
var sonratekar= setTimeout(() => {
    document.getElementById("denemel").innerHTML="Sonra tekrar deneyin...";
}, 7000);
function durdur() {
    clearTimeout(sonratekar);
}
</script>
</body>
```

Çıktısı: Durdurma işlemi yaptığım için "Sonra tekrar deneyin..." yazısını ekrana yazmadı.

Durdur
Süreniz doldu...

22.6.3 SetInterval

Belirtilen fonksiyonu süre bitiminde çalıştıran fonksiyondur. setTimeout dan farkı süre bitiminde çalışması istenilen fonksiyonun bir kez çalışmaz. Verilen süre her bittiğinde yeniden çalışır.

Örnek:

```
<body>
  <script>
    function yaz() {
      document.write("Süre doldu..." + "<br>")
    }
    setInterval(yaz, 2000);
  </script>
</body>
```

Çıktısı:

Süre doldu...
Süre doldu...
Süre doldu...
Süre doldu...

22.6.4 ClearInterval

SetInterval ile verilen fonksiyonu durdurmak veya iptal etmek için kullanılır.

Örnek:

```
<body>
  <button onclick="durdur()">Durdur</button>
  <p id="deneme"></p>
  <script>
    var sayi = 0;
    function yaz() {
      sayi = sayi + 1;
      document.getElementById("deneme").innerHTML = sayi;
    }
    var dur = setInterval(yaz, 1000);
    function durdur() {
      clearInterval(dur);
    }
  </script>
</body>
```

Çıktısı:

Durdur

82

23. Kapanış Yazısı

Evet. Javascript temel anlamda bu kadar. Yaklaşık 12.03.2020 tarihine başladığım javascript öğrenme serüvenine 21.08.2020 tarihinde son veriyorum. Ama bir daha hiç bakmayacağım anlamına gelmiyor. Çünkü yazılım her geçen gün gelişiyor. Bu belgeyi ayda bir de olsa güncellemeye çalışacağım. Belgede yeni bir güncelleme olup olmadığını [web sitemden](#) kontrol edebilirsiniz. Yaklaşık 5 ay gibi sürede Javascriptti temel anlamda bitirmiş sayılırım ve artık aldığım notlarımı sizler ile paylaşma vaktinin geldiğini düşünüyorum. Bu pdf de bulunan her şeyi öğrenmeniz durumunda javascriptti temel anlamda öğrenmiş olursunuz. Küçük bir uyarıda da bulunmak isterim ki hiçbir yazılımcı ürettiği yazılım için %100 hacklenemez garantisi veremediği gibi bende bu belgede bulunan her bilginin doğruluğunu konusunda bir garanti veremem. Ama yazdığım her şeyi yaptığım araştırmalara dayalı olarak yazdığımı söyleyebilirim. Siz yinede google da araştırmaktan çekinmeyin. 😊 Çünkü yazılımın temeli sorgulamaktan geçer. Bundan sonra ne olacağına gelirse. Javascripttin ajax yapısı vardır ki bu yapı içinde ayrı bir not hazırlamaktayım onuda yakında paylaşırım. Ardından Javascripttin kütüphaneleri olan JQuery, React ve Vue.js e bakacağım bunlar ile ilgilide aldığım notları da web sitemden paylaşacağım. 2020 yılı bitmeden de php ye geçiş yapacağımı düşünüyorum.

Benim javascriptti temel anlamda bitirmem 5 ayımı aldı ama siz düzenli olarak günde 5 saat çalışarak 1-2 ay gibi kısa bir sürede temel anlamda javascriptti öğrenebilirsiniz. Çift üniversite okununca pek vaktin olmuyor. 😊 Evet kapanış yazımda bu şekilde. Sizde öğrendiğiniz bir bilgi var ise paylaşmaktan çekinmeyin. Çünkü bilgi paylaştıkça çoğalır...

Sosyal Medya ve İletişim Bilgilerim

Web Sitem: <https://musayazlik.com>

Mail adresim: info@musayazlik.com

Telegram: <https://t.me/musayazlik>

Linkedin: <https://www.linkedin.com/in/musayazlik/>

Facebook: <https://www.facebook.com/musayazlik/>

Instagram: https://www.instagram.com/musa_yazlik/

Twitter: <https://twitter.com/musayazlik>

Github: <https://github.com/musayazlik>

24. Kaynakça

1. [Yazılım Bilişim](#)
2. [Btk Akademi Javascript Eğitimi](#)
3. [Emrah Yüksel Udemy JavaScript Eğitimi](#)
4. [Sadık Turan Udemy JavaScript Eğitimi](#)
5. [Developer Mozilla](#)
6. [W3schools](#)
7. [Webcebir](#)
8. [Mynotlar](#)
9. [Ustaderslik](#)
10. [Tasarımkodlama](#)
11. [Emreseyhan](#)